

SAPIENZA
UNIVERSITÀ DI ROMA

Acrobatic obstacle avoidance for the TITA two-wheel legged robot

Faculty of Information Engineering, Computer Science and Statistics
Master degree in Artificial Intelligence and Robotics

Emiliano Paradiso
ID number 1940454

Advisor
Prof. Giuseppe Oriolo

Co-Advisor
Dr. Nicola Scianca

Academic Year 2025/2026

Thesis defended on 13 April 2026
in front of a Board of Examiners composed by:

Marco Schaerf (chairman)
Roberto Beraldi
Ioannis Chatzigiannakis
Danilo Comminiello
Giulia Fiscon
Giuseppe Oriolo (advisor)
Fabio Patrizi

Acrobatic obstacle avoidance for the TITA two-wheel legged robot
Master degree thesis. Sapienza University of Rome

© 2026 Emiliano Paradiso. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: emilianoparadiso@icloud.com

Dedicato alla
mia famiglia

Abstract

This thesis investigates a model based control framework for a two–wheel legged robot, with emphasis on dynamic locomotion and obstacle avoidance through jumping.

A dynamic framework is developed to describe the robot's motion while accounting for the coupled behavior of the floating base, articulated legs, and wheel contacts. On this basis, a motion generation strategy is designed to produce feasible jump trajectories that satisfy physical constraints such as contact consistency, torque limits, and balance requirements.

The proposed approach is validated through simulation on the TITA robot, showing the robot's ability to execute jumping maneuvers with satisfactory tracking performance and recovery after landing. The results highlight both the potential and the challenges of dynamic locomotion in wheel legged systems. In particular, they show that jumping can significantly extend the mobility capabilities of such robots, allowing them to overcome obstacles and negotiate terrain conditions that are not manageable through pure rolling motion. This thesis therefore contributes toward the development of more agile and adaptive wheel legged robots for future applications in search and rescue, inspection, and field robotics.

Acknowledgments

I would like to express my sincere gratitude to Dr. Nicola Scianca for his invaluable support, guidance, and for the knowledge he shared with me throughout the development of this thesis. This work would not have been possible without his fundamental contribution.
I am also deeply grateful to Prof. Giuseppe Oriolo, first of all for giving me the opportunity to challenge myself with such a stimulating topic, and then for his continuous support, insightful guidance, and stimulating discussions, which greatly contributed to the development of this thesis.
Finally, I would like to thank Prof. Leonardo Lanari for his valuable advice and genuine interest in the outcome of this work.

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Contributions . . . 1
1.3 TITA robot . . . 2
1.4 MuJoCo simulation environment . . . 2
1.5 Literature overview . . . 3
1.6 Chapter organization . . . 4

2 Background **5**
2.1 Trajectory optimization . . . 5
2.1.1 Optimization algorithm . . . 6
2.1.2 Model Predictive Control . . . 7

3 Methods **9**
3.1 Framework Overview . . . 9
3.2 Notation and Symbols . . . 9
3.3 Model Predictive Controller . . . 11
3.3.1 Template model . . . 11
3.4 Whole Body Controller . . . 14
3.4.1 Cost function . . . 15
3.4.2 Equality constraints . . . 17
3.4.3 Inequality constraints . . . 18
3.5 Trajectory generator . . . 19
3.5.1 Cruise trajectory generator . . . 19
3.5.2 Obstacle detector . . . 20
3.5.3 Jump trajectory generator . . . 20
3.6 State filter . . . 23

4 Results **27**
4.1 Simulation results . . . 27
4.1.1 Locomotion . . . 27
4.1.2 Disturbance rejection . . . 32
4.1.3 Sequential jumping over multiple obstacles . . . 35
4.1.4 Maximum height jump . . . 41
4.1.5 High–speed jump . . . 45

A Differential Dynamic Programming **50**
A.1 Optimal control problem . . . 50
A.2 Backward pass . . . 51
A.3 Forward pass . . . 52
A.4 Extensions . . . 54
A.5 Advantages and limitations of DDP . . . 54

B Wheel–ground contact constraint **55**
B.1 Pure rolling of a sphere 55
B.2 Rolling without slipping for a fixed wheel 56

Bibliography **60**

Chapter 1

Introduction

Wheel legged robots combine the efficiency of wheeled locomotion with the versatility of legged systems, making them suitable for tasks that involve both fast motion and obstacle avoidance. However, their hybrid nature also makes control more challenging, especially in dynamic scenarios where balance, motion coordination, and physical constraints must be handled simultaneously.

In this context, model based control offers a suitable framework for the generation of dynamically consistent motions. The objective of this thesis is to investigate a model based control approach for a two–wheel legged robot, with particular focus on dynamic locomotion and obstacle avoidance.

The following sections present the motivation and contributions of the thesis, followed by an overview of the robotic platform, the simulation environment, and the relevant literature.

1.1 Motivation

The main motivation of this thesis lies in the need for control architectures that allow wheel legged robots to operate reliably not only during nominal rolling locomotion, but also in dynamic and unstructured scenarios. In many practical applications, a robot moving on wheels may encounter obstacles that cannot be overcome by rolling alone. In such cases, the ability to execute agile maneuvers, and in particular jumping motions, can significantly extend the operational capabilities of the platform.

Many reduced order predictive control approaches are effective for balance and trajectory generation, but rely on simplifying assumptions that limit their use in highly dynamic tasks. In particular, they are less suitable when vertical motion becomes a dominant component, as in obstacle avoidance through jumping.

This thesis is motivated by the need to bridge this gap. The goal is to develop a model based control framework that preserves the computational efficiency of reduced order prediction while remaining suitable for highly dynamic motions. In particular, the objective is to design a controller able to handle aggressive vertical dynamics, generate feasible jump trajectories online, and coordinate them with a whole body control layer for execution on the full robot.

A further motivation is the integration of different components within a coherent architecture. Dynamic locomotion requires not only trajectory generation and predictive control, but also accurate state estimation and a whole body controller capable of enforcing constraints and tracking the generated references. For this reason, is proposed a unified framework that aims at achieving both real time feasibility and robustness in challenging locomotion tasks.

1.2 Contributions

The main contributions of this thesis can be summarized as follows:

- The development of a model based control architecture for a wheel legged robot, integrating predictive control, whole body control, trajectory generation, and state estimation.



Figure 1.1. TITA wheeled–legged robot.

- The adoption of a simplified template model for the predictive controller that is better suited to handle aggressive vertical dynamics.
- The design of a trajectory generator strategy for obstacle avoidance through jumping, including the generation of dedicated jump reference trajectories.
- The integration of the predictive controller with a whole body controller formulated as a constrained optimization problem, enabling the execution of the planned motions on the full robot while respecting dynamic and actuation constraints.
- The validation of the proposed framework in simulation under multiple scenarios, including nominal locomotion, disturbance rejection, sequential jumping over multiple obstacles, and obstacle avoidance during high speed trajectory tracking.

1.3 TITA robot

The TITA robot from Direct Drive Technology (DDT) is a quasi–direct drive wheeled bipedal robot [1] featuring 8 actuated joints (see Fig. 1.1). The on–board computation is provided by an NVIDIA Jetson Orin NX (16 GB), and the sensor suite includes ultrasonic sensors, a SPAD time–of–flight (ToF) module, the Inertial Measurement Unit, and stereo cameras for perception.

On the software side, the platform runs Ubuntu 22.04 on the Jetson module and exposes a ROS 2 interface. The vendor provides an API with ROS 2 support, including documentation of the available ROS 2 topics and interfaces, as well as motor–level interface source code.

The robot mass is 24.1 kg excluding the battery. In the standing configuration, the overall dimensions are $510 \times 590 \times 490$ mm.

From a modeling standpoint, TITA is naturally described as a floating–base multibody system with 14 degrees of freedom: its overall position and orientation in space are not directly actuated, while actuation is provided only at the internal joints.

1.4 MuJoCo simulation environment

All simulations in this work are performed in MuJoCo, a general–purpose physics engine for simulating articulated systems interacting with the environment [2, 3]. MuJoCo models multibody dynamics in generalized coordinates and provides efficient algorithms for forward and inverse dynamics, with particular emphasis on stable and computationally efficient contact handling [2] which allows to evaluate the proposed approach in a physically consistent simulation setting.

The simulated robot and environment are described through MuJoCo's XML model format, which specifies the kinematic tree, inertial parameters, joint types and limits, actuation models, and contact–related properties [4].

1.5 Literature overview

Wheel legged robots combine the speed and efficiency of wheeled locomotion with the terrain adaptability of legged systems, making them suitable for tasks that require both fast motion and obstacle negotiation. Their control is challenging because the robot must maintain balance, satisfy rolling constraints, and coordinate its motion across different contact regimes, especially when dynamic maneuvers such as jumping are required.

A large part of locomotion control relies on reduced–order models. Among them, the Linear Inverted Pendulum Model has been widely used for gait generation and balance control because it captures the dominant center of mass dynamics while remaining simple enough for online computation [5, 6, 7, 8, 9, 10]. These works provide an important foundation for this thesis, as they motivate the use of simplified predictive models for motion generation. However, they were mainly developed for humanoid walking and balance, and are therefore less suited to highly dynamic maneuvers involving significant vertical motion, contact transitions, and obstacle avoidance.

In wheel legged robots, reduced–order predictive control is often combined with a lower level full body control layer. This hierarchical structure is effective because it separates motion generation on a simplified model from its realization on the full robot. The upper layer computes dynamically consistent reference trajectories, while the lower layer enforces full body dynamics, contact consistency, and actuator limits. This principle is central to the present thesis, which combines nonlinear MPC on a simplified inverted–pendulum–like model with an inverse dynamics controller. A relevant example is the work of Bjelonic et al. [11], where motion planning and hierarchical control are combined for wheel legged robots while explicitly accounting for wheel–ground kinematics and nonholonomic rolling constraints. Compared with that work, the present thesis is not limited to hybrid rolling–walking locomotion, but also investigates dynamic jumping motions for obstacle avoidance on a two-wheel legged robot.

Design–oriented works also provide useful context on recent wheel legged platforms. Zhao et al. [12] present a bio–inspired wheeled bipedal robot together with a controller based on a height–variable wheeled inverted pendulum model and full body dynamics, showing the relevance of simplified model control for agile balancing and locomotion. Similarly, DIABLO [13] introduces a direct–drive wheeled bipedal platform controlled through an inverted–pendulum–based formulation and LQR balancing, highlighting the growing interest in compact and highly backdrivable wheel legged designs.

The works in [14, 15, 16, 17] are particularly relevant to this thesis for their treatment of pure rolling constraints at the acceleration level within wheeled biped control frameworks. Although they do not focus on jumping or obstacle avoidance, they provide useful references for enforcing wheel–ground rolling consistency in the control problem.

The literature most closely related to this thesis is the one addressing jumping in wheel legged systems. Yu et al. [18] proposed a hierarchical jumping framework for hydraulic biped wheel legged robots, where an upper layer reduced model generates center of mass trajectories and a lower layer controller realizes the motion on the full robot. This is strongly related to the present work because it shows that hierarchical reduced–order control can be extended to highly dynamic maneuvers such as jumping. The main difference is that their approach is based on a spring–loaded inverted–pendulum–inspired model for a hydraulic platform, whereas this thesis also relies on an inverted–pendulum–based simplified model, but formulates a nonlinear MPC for an electric direct–drive wheel legged platform.

Another relevant contribution is the work of Hosseini et al. [19], which addresses dynamic hybrid locomotion and jumping for wheel legged robots. Their results show that wheel legged platforms

can use jumps to overcome obstacles while preserving the efficiency of rolling locomotion. This is closely aligned with the motivation of the present thesis, namely extending wheel legged mobility beyond nominal rolling through agile obstacle avoidance maneuvers. The contribution of this thesis, however, lies in the specific hierarchical architecture based on nonlinear MPC and inverse dynamics control, developed for the structure of a two–wheel legged robot.

Reliable state estimation is also an enabling component of these architectures, since predictive control and balance regulation are sensitive to errors in base pose, velocity, and contact estimation. In this context, contact–aided Kalman filtering approaches, and in particular invariant EKF formulations, provide a relevant background for robust state estimation in floating-base robots [20]. Although estimation is not the main focus of this thesis, it represents an important supporting component for the practical deployment of the proposed framework.

Overall, the literature shows a progression from reduced–order predictive balance control to hierarchical architectures combining predictive and full body control for wheel legged locomotion, and more recently toward dynamic jumping and obstacle negotiation. The present thesis is positioned within this evolution by combining nonlinear MPC with inverse dynamics control to enable acrobatic obstacle avoidance on a two–wheel legged robotic platform.

1.6 Chapter organization

The thesis is organized as follows.

Chapter 2 introduces the theoretical background required for the development of the proposed framework. In particular, it reviews the main concepts of trajectory optimization, with specific emphasis on optimization algorithms and Model Predictive Control.

Chapter 3 describes the proposed methodology. The chapter first presents the Whole Body Controller, including the related QP formulation. It then discusses the adopted Model Predictive Controller, which relies on a template model presented in detail. Finally, the trajectory generator is introduced, together with the obstacle detector, the trajectory generator, and the state filter.

Chapter 4 presents the results obtained with the proposed framework. The chapter is devoted to simulation results, including locomotion, disturbance rejection, sequential jumping over multiple obstacles, maximum height jump, and high–speed jump scenarios.

Appendix A provides additional material on Differential Dynamic Programming.

Appendix B reports the derivation of the wheel–ground contact constraint used to enforce pure rolling behavior in the WBC.

Chapter 2

Background

This chapter summarizes the theoretical tools that support the methods adopted in this work. It first introduces optimization–based formulations and then reviews practical solution approaches, focusing on dynamic programming and, in particular, Differential Dynamic Programming. Finally, Model Predictive Control is presented as a receding–horizon strategy that enables closed loop operation through repeated online optimization. The material in this chapter is primarily drawn from [21].

2.1 Trajectory optimization

Model–based control of a complex multi degrees of freedom legged robot is inherently challenging due to its underactuated nature, intermittent contacts, and stringent physical constraints. In this setting, deriving an explicit closed form feedback law is generally infeasible, and control objectives are more naturally expressed as an optimization problem.

A widely adopted approach is therefore to rely on optimization–based techniques, commonly referred to as *trajectory optimization*. Trajectory optimization aims at computing state and control trajectories over a finite time horizon that minimize a predefined cost function while satisfying the system dynamics and additional constraints. The resulting solution provides a dynamically consistent motion plan, i.e., a sequence of states and inputs that achieves the desired task.

In the continuous–time setting, the basic finite–horizon trajectory optimization problem can be written as:

$$\min_{\mathbf{x}(\cdot),\,\mathbf{u}(\cdot)} \quad \int_{t_0}^{t_f} \ell\big(\mathbf{x}(t),\mathbf{u}(t)\big)\,dt + \ell_f\big(\mathbf{x}(t_f)\big) \tag{2.1}$$

$$\text{s.t.} \quad \dot{\mathbf{x}}(t) = f\big(\mathbf{x}(t),\mathbf{u}(t)\big), \quad t \in [t_0, t_f], \tag{2.2}$$

$$\mathbf{x}(t_0) = \bar{\mathbf{x}}_0, \tag{2.3}$$

$$h\big(\mathbf{x}(t),\mathbf{u}(t)\big) = 0, \quad t \in [t_0, t_f], \tag{2.4}$$

$$g\big(\mathbf{x}(t),\mathbf{u}(t)\big) \leq 0, \quad t \in [t_0, t_f]. \tag{2.5}$$

The objective (2.1) consists of a running cost $\ell(\mathbf{x}(t),\mathbf{u}(t))$, integrated over the horizon, and a terminal cost $\ell_f(\mathbf{x}(t_f))$ that penalizes the final state. The optimization is subject to the dynamics (2.2), the initial condition (2.3), and generic equality and inequality constraints (2.4)–(2.5).

A discrete–time counterpart over a horizon of length N is:

$$\min_{\substack{\mathbf{x}_1,\dots,\mathbf{x}_N \\ \mathbf{u}_0,\dots,\mathbf{u}_{N-1}}} \quad \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) \tag{2.6}$$

$$\text{s.t.} \quad \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \tag{2.7}$$

$$\mathbf{x}_0 = \bar{\mathbf{x}}_0, \tag{2.8}$$

$$h(\mathbf{x}_k, \mathbf{u}_k) = 0, \quad k = 0, \dots, N-1, \tag{2.9}$$

$$g(\mathbf{x}_k, \mathbf{u}_k) \leq 0, \quad k = 0, \dots, N-1. \tag{2.10}$$

Here the decision variables are the sequences $\{\mathbf{x}_k\}_{k=1}^{N}$ and $\{\mathbf{u}_k\}_{k=0}^{N-1}$. The running cost is accumulated over $k = 0, \dots, N-1$, and $\ell_N(\mathbf{x}_N)$ is a terminal cost on the final state.

In principle, the solution of this problem over the whole state space, represents an optimal control law $\mathbf{u}(\mathbf{x})$ for every $\mathbf{x} \in \mathcal{X}$.

In most practical legged robotics settings, computing such a solution is computationally prohibitive or conceptually problematic, due to the high dimensional continuous state space or nonlinear and nonconvex constraints. Nonetheless, it is still possible to solve (2.6)–(2.10) for a given initial condition $\bar{\mathbf{x}}_0$ and obtain an optimal open loop solution, namely an optimal trajectory described by a sequence of inputs $\{\mathbf{u}_k^\star\}_{k=0}^{N-1}$ and the associated states $\{\mathbf{x}_k^\star\}_{k=0}^{N}$. This solution is optimal for the specified initial state and horizon, but it does not by itself define a feedback control law over the entire state space.

2.1.1 Optimization algorithm

A classical tool to solve finite–horizon trajectory optimization problems is *dynamic programming* (DP), introduced by [22]. DP relies on Bellman's principle of optimality, which states that an optimal solution can be obtained by decomposing the original problem into a sequence of nested optimal sub–problems.

Cconsider a discrete–time system

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \qquad k = 0, \dots, N-1, \tag{2.11}$$

and the finite–horizon cost

$$J_k(\mathbf{x}_k) = \sum_{i=k}^{N-1} \ell_i(\mathbf{x}_i, \mathbf{u}_i) + \ell_N(\mathbf{x}_N). \tag{2.12}$$

Given an initial condition $\mathbf{x}_0$ and a horizon N, the goal is to determine the control sequence $\{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ that minimizes J_0 subject to the system dynamics. DP exploits the Markov property of the system: once the current state is known, the optimal future control strategy is independent of past states and controls.

This motivates the definition of the *value function*, i.e., the optimal cost–to–go starting from state $\mathbf{x}_k$ at time step k:

$$V_k(\mathbf{x}_k) = \min_{\mathbf{u}_k,\dots,\mathbf{u}_{N-1}} \left(\sum_{i=k}^{N-1} \ell_i(\mathbf{x}_i, \mathbf{u}_i) + \ell_N(\mathbf{x}_N) \right) \quad \text{s.t. } \mathbf{x}_{i+1} = f(\mathbf{x}_i, \mathbf{u}_i). \tag{2.13}$$

Intuitively, $V_k(\mathbf{x}_k)$ represents the minimum cost achievable when starting from $\mathbf{x}_k$ and acting optimally until the end of the horizon.

By applying Bellman's principle, the value function satisfies the following recursion:

$$V_k(\mathbf{x}_k) = \min_{\mathbf{u}_k} \left(\ell_k(\mathbf{x}_k, \mathbf{u}_k) + V_{k+1}\big(f(\mathbf{x}_k, \mathbf{u}_k)\big) \right), \qquad V_N(\mathbf{x}_N) = \ell_N(\mathbf{x}_N). \tag{2.14}$$

For general nonlinear dynamics, computing $V_k(\cdot)$ in closed form is typically infeasible. A direct numerical computation over a discretized state space is possible in principle, but it rapidly becomes intractable as the state dimension grows (the well–known *curse of dimensionality* problem). Therefore, practical methods often rely on local approximations of the value function.

In this work, we adopt *Differential Dynamic Programming* (DDP) [23], [24] an iterative technique for nonlinear optimal control. DDP performs a backward pass and a forward pass on the current estimate of state–control trajectory $(\bar{\mathbf{x}}, \bar{\mathbf{u}})$, i.e. nominal trajectory. In the backward pass, the algorithm approximates the value function as a quadratic function along the nominal trajectory. In the forward pass, a forward integration is performed to produce a new nominal trajectory based on the value function computed in the backward pass. This process is repeated until a desired level of convergence. For a detailed derivation, see Appendix A.

2.1.2 Model Predictive Control

Model Predictive Control (MPC) is a feedback control strategy that computes control actions by repeatedly solving a *finite–horizon* optimal control (trajectory optimization) problem online. At each control cycle, MPC uses the current state estimate as the initial condition, predicts the system evolution over a horizon of N steps through a model, and selects the control sequence that minimizes a cost function while satisfying dynamics and constraints. Only the first control input of the optimal sequence is applied to the system; then the horizon is shifted forward and the optimization is solved again at the next time step. This *receding–horizon* mechanism provides feedback and allows MPC to react to disturbances, modeling errors, and changes in the environment.

A typical discrete–time MPC problem can be written as

$$\min_{\mathbf{u}_0,\dots,\mathbf{u}_{N-1}} \quad \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N) \tag{2.15}$$

$$\text{s.t.} \quad \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \qquad k = 0, \dots, N-1, \tag{2.16}$$

$$\mathbf{x}_k \in \mathcal{X}_{\text{adm}},\ \mathbf{u}_k \in \mathcal{U}_{\text{adm}}, \qquad k = 0, \dots, N-1, \tag{2.17}$$

$$\mathbf{x}_N \in \mathcal{X}_{\text{ter}} \tag{2.18}$$

where $f(\cdot)$ denotes the system model, $\ell(\cdot)$ and $\ell_f(\cdot)$ are the running and terminal costs, and $\mathcal{X}_{\text{adm}}$ and $\mathcal{U}_{\text{adm}}$ represent admissible state and input sets. In practice, MPC is particularly attractive for robotics because it can explicitly handle constraints and multi–objective behaviors while maintaining closed loop operation through repeated replanning.

Moreover, introducing the terminal constraint (2.18) can provide stronger theoretical properties when the terminal set $\mathcal{X}_{\text{ter}}$ is chosen to be *control invariant*, i.e.,

$$\forall\, \mathbf{x} \in \mathcal{X}_{\text{ter}}\ \exists\, \mathbf{u} \in \mathcal{U}_{\text{adm}} \text{ s.t. } f(\mathbf{x}, \mathbf{u}) \in \mathcal{X}_{\text{ter}}. \tag{2.19}$$

Under this assumption, enforcing $\mathbf{x}_N \in \mathcal{X}_{\text{ter}}$ guarantees that, once the predicted terminal state enters $\mathcal{X}_{\text{ter}}$, feasibility can be maintained beyond the horizon by applying admissible inputs that keep the state inside $\mathcal{X}_{\text{ter}}$. This ensures *recursive feasibility*, i.e., if the MPC optimization problem is feasible at time t, then it remains feasible at time $t+1$ (and at subsequent time steps) when solved in a receding–horizon fashion.

A practical challenge of MPC lies in formulating the underlying optimization problem so that it can be solved within the available control budget (typically on the order of 10 ms). To meet such real–time constraints, it is common to adopt simplified robot models that reduce the computational burden while retaining the dominant dynamics relevant for control.

A widely used strategy for real–time nonlinear MPC is the *real–time iteration* (RTI) scheme. Rather than iterating the numerical solver until full convergence at each sampling instant, RTI performs only a single iteration of the chosen optimization algorithm (e.g., SQP, DDP, or related methods) per control cycle. This is enabled by *warm–starting* the solver with the solution obtained

at the previous time step, typically shifted along the horizon. As a result, each control update yields a generally suboptimal solution; however, across successive control cycles the iterates remain close to the optimum and progressively refine the solution while maintaining closed loop feasibility and real–time execution.

Chapter 3

Methods

This chapter presents the methodology adopted in this thesis. First, the overall control framework is introduced. Then, the main components of the proposed approach are described, including the reduced–order predictive controller, the low–level whole body controller, the trajectory generation strategy, the obstacle triggered jump update, and the state estimation method adopted for a possible real robot implementation.

3.1 Framework Overview

A central element of this work is the hierarchical control architecture based on Model Predictive Control (MPC), introduced in Section 2.1.2. In legged and wheel legged systems, solving a predictive controller directly on the full body model is generally too expensive for real–time operation. Although the computational burden can be reduced by lighten the MPC problem formulation, e.g., by shortening the prediction horizon, this may degrade performance and robustness, since the controller reasons over a more limited look–ahead window. For this reason, predictive control is usually carried out on a *simplified* model that captures the dominant locomotion dynamics while remaining fast enough to be solved at the required control rate.

In the proposed framework, MPC generates desired trajectories according to the imposed control objectives using a reduced–order model based on centroidal dynamics. These references are then tracked by a low–level whole body controller (WBC), which accounts for the full body dynamics and enforces actuation and contact constraints when computing the robot commands.

As shown in Fig. 3.1, the proposed pipeline is organized as a cascade of modules. In parallel, a trajectory generator provides the reference trajectories for the robot center of mass (CoM), which are tracked by the control block. During nominal operation, the robot follows the trajectory produced by the cruise trajectory generator. When an obstacle is detected along the path, the obstacle detector triggers the trajectory selector, which switches the reference generation to the jump trajectory generator. The latter then plans a trajectory that includes a jump maneuver to overcome the obstacle, provided that its dimensions lie within the robot's jumping capabilities.

Finally, the chapter also presents the state estimation method considered for a possible deployment on a real robot, where onboard sensor measurements are fused to estimate the robot state required by both the predictive and low–level control layers.

3.2 Notation and Symbols

For the reader's convenience, the main symbols used throughout this chapter are summarized below.

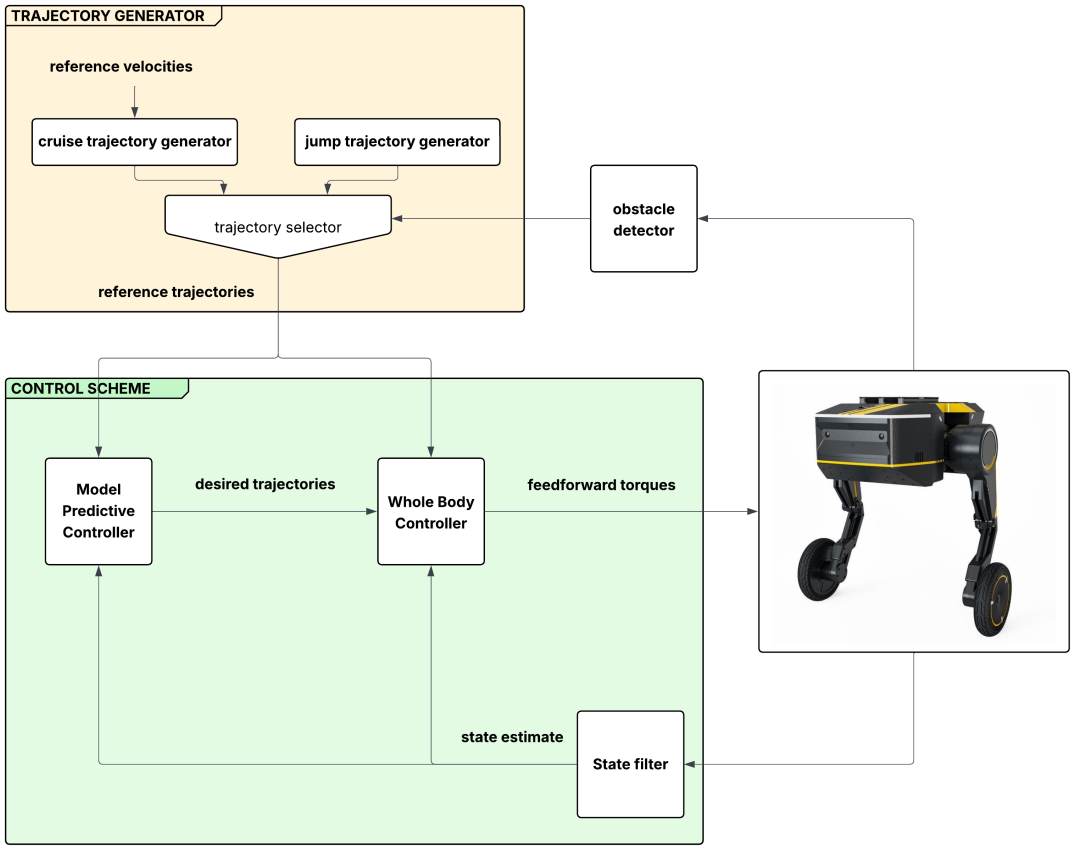


Figure 3.1. Representative block scheme of the control architecture.

Table 3.1. Main symbols used in this chapter.

Symbol	Meaning
Whole body control	
$\boldsymbol{\chi}$	Robot configuration vector
$\mathbf{p}_{fb}, \mathbf{v}_{fb}$	Floating base position and linear velocity
$\mathbf{q}_{fb}$	Floatingvbase orientation represented by a unit quaternion
$\boldsymbol{\omega}_{fb}$	Floatingvbase angular velocity
$\mathbf{q}_j, \dot{\mathbf{q}}_j, \ddot{\mathbf{q}}_j$	Joint position, velocity, and acceleration vectors
$\boldsymbol{\nu}$	Generalized velocity vector
$\dot{\boldsymbol{\nu}}$	Generalized acceleration vector
$\boldsymbol{\xi}$	Decision vector of the whole body QP problem
$\mathbf{f}_{l,i}$	Contact force at the i–th left contact point
$\mathbf{f}_{r,i}$	Contact force at the i–th right contact point
Model predictive control	
$\mathbf{x}$	State vector of the predictive model
$\mathbf{u}$	Control input vector of the predictive model
N	Prediction horizon length
$\mathbf{p}_{com}, \dot{\mathbf{p}}_{com}$	Center of mass position and velocity

Continued on next page

Symbol	Meaning
$\mathbf{c}$	Virtual base point of the simplified model
$v_{c,z}, a_{c,z}$	Vertical components of the velocity and acceleration of the virtual base point
θ	Orientation of the virtual base in the plane
v, a	Planar driving velocity and acceleration of the virtual base
ω, α	Planar steering angular velocity and angular acceleration of the virtual base
d	Distance between the contact points in the simplified model
$\mathbf{p}_{c,l}$	Left contact point in the simplified model
$\mathbf{p}_{c,r}$	Right contact point in the simplified model
$\mathbf{F}_l$	Contact force applied at the left contact point in the simplified model
$\mathbf{F}_r$	Contact force applied at the right contact point in the simplified model
$\mathbf{x}^{\text{ref}}$	Reference state trajectory
$\mathbf{u}^{\text{ref}}$	Reference input trajectory
State estimation	
$\mathbf{x}_{KF}$	Filter state vector
$\mathbf{u}_{KF}$	Filter state vector
$\mathbf{z}_{KF}$	Filter measurement vector

3.3 Model Predictive Controller

To achieve high performance while coping with the intrinsic instability of wheeled legged platforms, we adopt a trajectory optimization framework based on Model Predictive Control. Since the robot operates over highly nonlinear maneuvers, a nonlinear MPC (NMPC) formulation is particularly suitable to capture the relevant dynamics and constraints throughout the motion.

A key challenge in NMPC is obtaining an optimal solution within the available control period. For this reason, we employ a simplified template model that keeps the optimization problem computationally tractable while remaining sufficiently representative of the physical system to enable aggressive behaviors, including jumping. The following sections detail the adopted model for NMPC and the corresponding problem statement.

Following the MPC formulation presented in Section 2.1.2 we pose the finite–horizon optimal control problem as:

$$\min_{\mathbf{u}_0,\dots,\mathbf{u}_{N-1}} \quad \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) \tag{3.1}$$

$$\text{s.t.} \quad \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \tag{3.2}$$

$$\mathbf{x}_0 = \mathbf{x}_{current} \tag{3.3}$$

$$h(\mathbf{x}_k, \mathbf{u}_k) = 0, \quad g(\mathbf{x}_k, \mathbf{u}_k) \leq 0, \qquad k = 0, .., N-1 \tag{3.4}$$

$$h^N_{ter}(\mathbf{x}) = 0 \tag{3.5}$$

3.3.1 Template model

To capture the unstable nature of the wheeled legged system, we rely on an Inverted Pendulum (IP) model. Unlike fixed–height approximations such as the Linear Inverted Pendulum (LIP) [5], the proposed formulation does not constrain the CoM to a constant height. Instead, it preserves

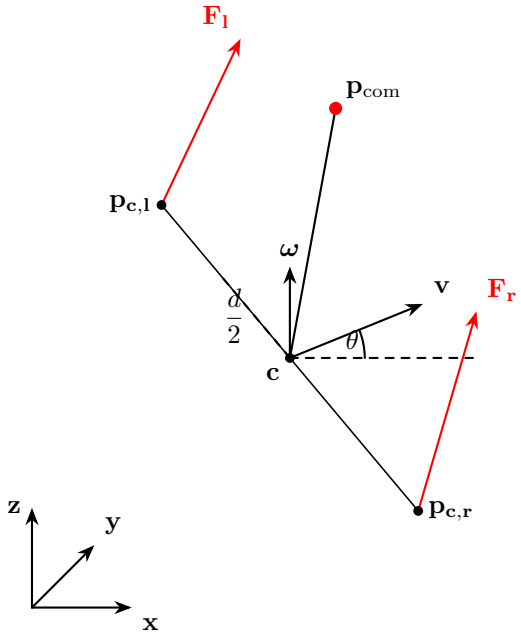


Figure 3.2. Illustration of the DFCIP model.

the more general Variable–Height Inverted Pendulum (VHIP) [8] structure, which is essential for jump maneuvers and other motions involving significant vertical dynamics.

In addition, the presence of wheels introduces nonholonomic constraints due to rolling without slipping. This yields local mobility limitations that must be explicitly considered in the prediction model, resulting in nonlinear kinematic constraints integrated into the NMPC formulation.

The Double Forces Constrained Inverted Pendulum (DFCIP) model is defined by the state and input vectors

$$\mathbf{x} = \begin{pmatrix} \mathbf{p}_{com} & \dot{\mathbf{p}}_{com} & \mathbf{c} & v_{cz} & \theta & v & \omega \end{pmatrix}^T \quad \mathbf{u} = \begin{pmatrix} a & a_{cz} & \alpha & \mathbf{F}_l & \mathbf{F}_r \end{pmatrix}^T$$

The model represents a point mass located above a virtual base, described by the position $\mathbf{c}$ and the orientation θ, whose planar motion is subject to a rolling without slipping constraint. This assumption captures the local mobility limitations induced by the wheeled structure; however it assumes flat ground and neglects stepping phases and wheel camber effects. Such effects can be incorporated in a more general formulation if required.

The vertical base dynamic is modeled as a double integrator, enabling the tracking of height–varying tasks and, in particular, jump maneuvers despite its simplified structure. The CoM dynamic is driven by the ground contact forces $\mathbf{F}_l$ and $\mathbf{F}_r$, applied at left $\mathbf{p}_{c,l}$ and right $\mathbf{p}_{c,r}$ contact points, respectively, located at a lateral offset of $d/2$ from the base point, as illustrated in Figure 3.2.

The DFCIP's equations of dynamic model are:

$$\ddot{\mathbf{p}}_{com} = \frac{1}{m}\left(\mathbf{F}_l + \mathbf{F}_r\right) \tag{3.6}$$
$$\dot{c}_x = v\cos(\theta) \tag{3.7}$$
$$\dot{c}_y = v\sin(\theta) \tag{3.8}$$
$$\dot{c}_z = v_{cz} \tag{3.9}$$
$$\dot{v}_{cz} = a_{cz} \tag{3.10}$$
$$\dot{\theta} = \omega \tag{3.11}$$
$$\dot{v} = a \tag{3.12}$$
$$\dot{\omega} = \alpha \tag{3.13}$$

From these continuous time equations, the discrete time dynamic model in (3.2) is obtained via forward Euler integration, assuming constant inputs over each sampling interval.

When the robot is in contact with the ground, the moment–balance condition together with the contact and stability constraints are enforced. Conversely, during the flight phase, contact forces are set to zero, so that the CoM motion reduces to free fall under gravity and the base yaw rate (ω) is constrained to vanish.

$$\text{in contact: } \begin{cases} \sum_{i\in l,r}(\mathbf{p}_{c,i} - \mathbf{p}_{com}) \times \mathbf{F}_i \quad = 0 \\ F_{zi} \geq 0 \\ v_{cz} = 0 \\ c_x^N = x_{\text{com}}^N \\ c_y^N = y_{\text{com}}^N \end{cases} \tag{3.14}$$

$$\text{in flight: } \begin{cases} \mathbf{F}_i = 0 \\ \omega = 0 \end{cases} \tag{3.15}$$

Finally, the terminal equalities impose that, at the end of the prediction horizon, the planar position of the base coincides with the planar projection of the CoM. This terminal condition is introduced to promote stability.

A final note concerns the adopted solver Crocoddyl [26], which implements the DDP algorithm discussed in Section 2.1.1 but does not provide an explicit interface for imposing equality/inequality constraints. Consequently, constraints that would ideally be imposed as hard constraints are handled in practice as soft constraints, by adding penalty terms to the objective function with suitably large weights.

The running cost in (3.1) is defined as a quadratic tracking objective, penalizing deviations of the predicted state and input from their reference trajectories. In addition, soft constraints are enforced by penalizing their residual in the objective:

$$\begin{aligned} \ell(x_k, u_k) = (\mathbf{x}_k - \mathbf{x}_k^{ref})^T \mathbf{W}_{x,k} (\mathbf{x}_k - \mathbf{x}_k^{ref}) + (\mathbf{u}_k - \mathbf{u}_k^{ref})^T \mathbf{W}_{u,k} (\mathbf{u}_k - \mathbf{u}_k^{ref}) \\ + \mathbf{h}(\mathbf{x}_k, \mathbf{u}_k)^T \mathbf{W}_{h,k}\, \mathbf{h}(\mathbf{x}_k, \mathbf{u}_k), \end{aligned} \tag{3.16}$$

where $\mathbf{h}(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{0}$ denotes the set of equality constraints and $\mathbf{W}_{h,k} \succeq \mathbf{0}$ weights their violation. As part of the input reference, the vertical contact force reference is chosen as $u_{F_{iz},k}^{\text{ref}} = mg$ for $i \in \{l, r\}$, which biases the solution toward nominal weight support during contact. $\mathbf{W}_{x,k}$ and $\mathbf{W}_{u,k}$ are diagonal positive semidefinite weighting matrices, which tune the relative importance of state–tracking accuracy and input effort. Moreover time–varying weights are used to switch the relative penalties between contact and flight phases. The terminal cost is defined in the same manner, omitting the input–dependent term.

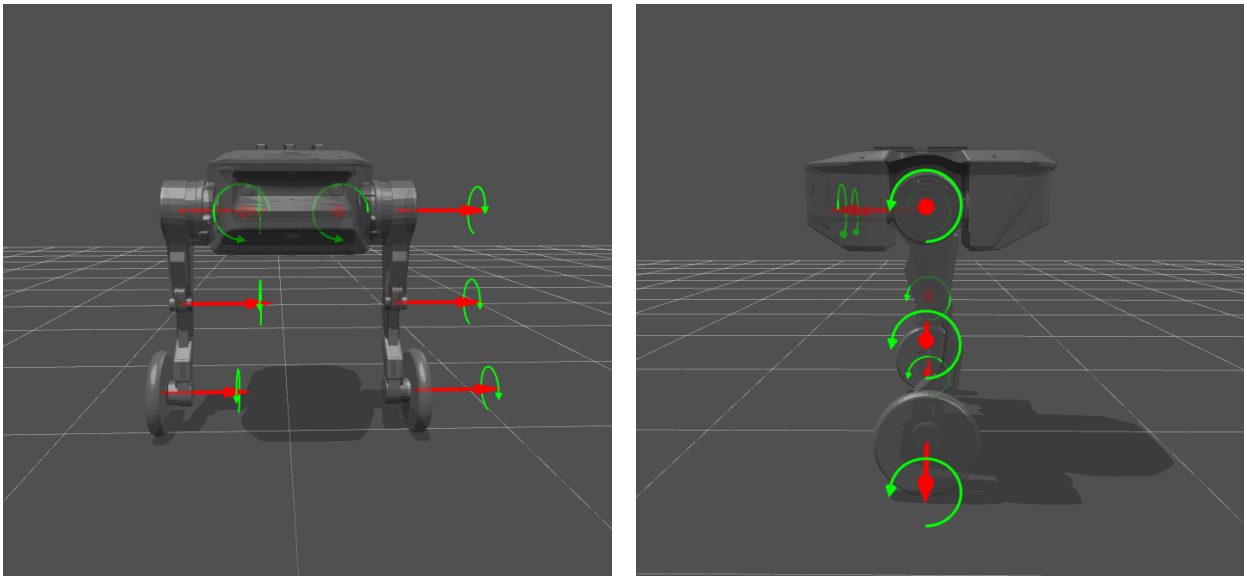
We note that the resulting trajectory of the unicycle–base point is then mapped back as

$$\mathbf{p}_{c,l} = \mathbf{c} + R(\theta) \begin{pmatrix} 0 \\ \frac{d}{2} \\ 0 \end{pmatrix}, \tag{3.17}$$

$$\mathbf{p}_{c,r} = \mathbf{c} - R(\theta) \begin{pmatrix} 0 \\ \frac{d}{2} \\ 0 \end{pmatrix}, \tag{3.18}$$

where $R(\theta)$ is the rotation matrix representing a rotation about the z–axis by an angle θ.

An important simplification of the model is that the wheel planes remain perpendicular to the ground, thereby neglecting tilting effects. As a result, the MPC can only generate desired trajectories that do not induce tilting motion of the wheel plane with respect to the ground. However, this simplification is justified by the focus on standard locomotion behaviors, in which the wheels remain vertically oriented with respect to the ground plane.



(a) (b)

Figure 3.3. Sketch of the TITA kinematic structure, with the joint axes highlighted. (a) Front view. (b) Side view.

3.4 Whole Body Controller

Whole Body Control is a control framework that coordinates all the robot degrees of freedom to achieve a desired motion while respecting the physical constraints imposed by the hardware and by the interaction with the environment. In contrast to reduced–order approaches, WBC leverages the full body dynamics of the system and, when contacts are present, explicitly incorporates contact forces and friction limits to guarantee physically consistent behavior.

The desired behavior is typically specified as a set of tasks that encode motion objectives at different levels. Common tasks include tracking a reference center of mass trajectory, regulating torso orientation, tracking swing–foot motions, and enforcing stance–foot contact conditions (no slip and no penetration). For wheel legged platforms, additional objectives may include wheel motion control and wheel–ground contact constraints. Since these tasks can be redundant or partially conflicting, WBC provides a systematic way to balance them.

A widely adopted and computationally efficient implementation formulates WBC as a *Quadratic Program* (QP). At each control cycle, the QP solve the inverse dynamics by minimizing a weighted least–squares tracking error over the tasks. This leads to a quadratic objective, subject to linear equality and inequality constraints that represent the feasibility limits.

The kinematic structure of the TITA robot is shown in Fig. 3.3. The robot consists of two articulated legs attached to a floating base, resulting in 8 actuated degrees of freedom, in addition to the 6 degrees of freedom of the floating base. Each leg is composed of four actuated joints. Starting from the base, the first joint generates motion in the frontal plane, i.e., it allows lateral leg motion. The second joint corresponds to the hip pitch joint, which drives the forward and backward motion of the upper part of the leg in the sagittal plane. The third joint is the knee joint, which determines the relative motion between upper and lower leg segments. Finally, the fourth joint is the wheel joint, which drives the wheel rotation.

Its configuration vector is defined as

$$\boldsymbol{\chi} = \begin{pmatrix} \mathbf{p}_{fb} \\ \mathbf{q}_{fb} \\ \mathbf{q}_j \end{pmatrix}, \qquad \boldsymbol{\chi} \in SE(3) \times (SO(2))^8, \tag{3.19}$$

where $\mathbf{p}_{fb}$ denotes the floating base position expressed in the inertial frame, $\mathbf{q}_{fb}$ its orientation, represented by a unit quaternion, and $\mathbf{q}_j$ collects the joint coordinates. The configuration space dimension is 14.

The generalized velocity vector is defined as

$$\boldsymbol{\nu} = \begin{pmatrix} \mathbf{v}_{fb} \\ \boldsymbol{\omega}_{fb} \\ \dot{\mathbf{q}}_j \end{pmatrix}, \tag{3.20}$$

where $\mathbf{v}_{fb}$ is the linear velocity of the floating base, $\boldsymbol{\omega}_{fb}$ is its angular velocity, and $\dot{\mathbf{q}}_j$ collects the joint velocities.

To formulate the whole body QP for this system, we consider the optimization variables

$$\boldsymbol{\xi} = \begin{pmatrix} \dot{\boldsymbol{\nu}} \\ \mathbf{f}_{l,i} \\ \vdots \\ \mathbf{f}_{r,i} \\ \vdots \\ \boldsymbol{\tau} \end{pmatrix}, \tag{3.21}$$

which include the generalized acceleration vector $\dot{\boldsymbol{\nu}}$, i.e., the time derivative of the generalized velocity vector, the ground reaction forces at the left and right contact points, denoted by $\mathbf{f}_{l,i}$ and $\mathbf{f}_{r,i}$ for $i = 1, \ldots, n_{\text{contacts}}$, with the ground contact modeled through n_{contacts} contact points, and the joint torques $\boldsymbol{\tau}$. The system is underactuated, so the floating base dynamics are not directly actuated and instead depend on the interaction with the ground through the contact forces.

An important remark is that the generalized accelerations are not independent decision variables, since they are determined by the contact forces and the actuated torques through the dynamic model, which imposes a constraint coupling these quantities. Nevertheless, they are retained as optimization variables in order to introduce regularization on both torques and accelerations.

The QP can then be written in the standard form

$$\min_{\mathbf{x}} \quad \boldsymbol{\xi}^T H \boldsymbol{\xi} + \mathbf{g}^T \boldsymbol{\xi} \tag{3.22}$$

$$\text{s.t.} \quad A\boldsymbol{\xi} = \mathbf{b}, \tag{3.23}$$

$$\mathbf{d}_{\min} \leq C\boldsymbol{\xi} \leq \mathbf{d}_{\max}, \tag{3.24}$$

where the quadratic objective encodes task tracking and regularization, the equality constraints enforce the system dynamics and contact kinematics, and the inequalities impose feasibility limits. A detailed discussion of each component is provided in Sections 3.4.1, 3.4.2, and 3.4.3.

3.4.1 Cost function

The objective in (3.22) is constructed to be quadratic in the decision variables $\boldsymbol{\xi}$ in order to obtain a QP that can be solved efficiently at each control cycle. The matrix H and vector $\mathbf{g}$ are obtained by combining the task–tracking objectives in a weighted least–squares form with appropriate regularization penalties, which promote well–conditioned solutions and discourage unnecessarily large accelerations, contact forces and joint torques.

In this work, the set of tasks includes: CoM position tracking; tracking of the left and right wheel center positions and orientations; base link orientation regulation; joint posture regulation; and angular momentum regulation. Each objective is weighted to reflect its relative importance and to yield stable tracking behavior. More formally, the cost can be interpreted as the combination of (i) acceleration/torque regularization, (ii) contact force regularization, (iii) task–space tracking penalties, (iv) posture regulation and (v) angular momentum penalization.

The regularization terms take the form

$$w_{acc} \left\| \dot{\boldsymbol{\nu}} \right\|^2, \qquad w_f \sum_{i \in \{l_n, r_n\}} \left\| \mathbf{f}_i \right\|^2, \qquad \text{and} \qquad w_\tau \left\| \boldsymbol{\tau} - \boldsymbol{\tau}_{prev} \right\|^2 \tag{3.25}$$

where $w_{\ddot{q}}$, w_τ and w_f are positive weights and $\mathbf{f}_i$ denotes the ground reaction force at contact i. The joint torque regularization penalizes deviations from the previous solution, a strategy adopted to mitigate control chattering.

Task tracking is enforced by penalizing the deviation between the achieved and desired task accelerations. For each task j, the corresponding term is written as

$$w_j \left\| J_j \dot{\boldsymbol{\nu}} + \dot{J}_j \boldsymbol{\nu} - \mathbf{a}_j^{\text{des}} \right\|^2, \tag{3.26}$$

where J_j is the task Jacobian, $\dot{J}_j$ its time derivative and w_j is a positive weight associated to the j–th task. The desired task acceleration is obtained by a feedforward plus PD feedback correction around the desired trajectory:

$$\mathbf{a}_j^{\text{des}} = \mathbf{a}_j^d - K_p \mathbf{e}_j - K_d \dot{\mathbf{e}}_j, \tag{3.27}$$

with $\mathbf{e}_j$ denoting the task–space tracking error (which include translational or rotational components, depending on the task definition) and $\dot{\mathbf{e}}_j$ the corresponding task–space velocity error.

It is important to note that, in the WBC formulation, a general task for tracking the wheel plane orientation is taken into account. However, as mentioned in Section 3.3.1, the model adopted in the MPC formulation relies on a strong simplification, as it does not provide a reference for the wheel plane tilt angle. This plane is therefore explicitly assumed to remain perpendicular to the ground plane, and the resulting reference only specifies its orientation associated with steering.

In addition a joint regulation task is included to maintain a well–conditioned configuration and to exploit kinematic redundancy. This objective penalizes deviations of the joint coordinates from a nominal posture $\mathbf{q}_j^d$, thereby preventing drift, avoiding kinematic singularities, and promoting smooth and physically plausible motions. In this work, the desired joint–space acceleration is defined through feedforward plus PD feedback around the nominal joint trajectory:

$$\ddot{\mathbf{q}}_j^{\text{des}} = \ddot{\mathbf{q}}_j^d - K_{p,j} \left(\mathbf{q}_j - \mathbf{q}_j^d \right) - K_{d,j} \left(\dot{\mathbf{q}}_j - \dot{\mathbf{q}}_j^d \right), \tag{3.28}$$

and enforced in the QP by penalizing the corresponding acceleration tracking error:

$$w_j \left\| \ddot{\mathbf{q}}_j - \ddot{\mathbf{q}}_j^{\text{des}} \right\|^2. \tag{3.29}$$

In practice, this term acts as a secondary objective that shapes the posture without interfering with higher–priority tasks, while also providing regularization when the primary tasks do not fully constrain the solution.

Finally, angular momentum regulation is included to improve whole body stability during dynamic motions. In particular, the centroidal momentum is related to the generalized velocity through the centroidal momentum matrix (CMM), i.e., $\mathbf{h} = \Omega(\boldsymbol{\chi})\boldsymbol{\nu}$. By selecting the angular component of $\mathbf{h}$ and penalizing its magnitude (or tracking a desired value), the controller discourages excessive rotational momentum of the body. This term can be expressed as an additional quadratic penalty of the form

$$w_h \left\| S_h \, \Omega(\boldsymbol{\chi}) \boldsymbol{\nu} - \mathbf{h}^d \right\|^2, \tag{3.30}$$

where S_h is a selection matrix (e.g., extracting angular momentum) and w_h is a positive weight.

3.4.2 Equality constraints

Since the WBC is adopted to solve the inverse dynamics problem, the equality constraints in (3.23) are introduced to explicitly enforce both the system dynamics and the wheel–ground contact kinematics. In particular, they guarantee: (i) consistency with the equations of motion of a floating–base system, and (ii) a wheel–ground contact behavior, modeled as pure rolling (i.e., no slip) imposed at the acceleration level.

System dynamics constraint. The system dynamics are enforced via the Euler–Lagrange equations of motion

$$M(\boldsymbol{\chi})\,\dot{\boldsymbol{\nu}} + \mathbf{n}(\boldsymbol{\chi}, \boldsymbol{\nu}) = S\boldsymbol{\tau} + \sum_{i \in \{l_n, r_n\}} J_{c,i}^{T}(\boldsymbol{\chi})\,\mathbf{f}_i. \tag{3.31}$$

In the above equation, M, $\mathbf{n}$ and $J_{c,i}$ denote, respectively, the inertia matrix, the vector collecting Coriolis/centrifugal and gravity terms and the contact Jacobians. Because the floating base is not directly actuated, the rows of the equations of motion corresponding to the floating–base coordinates have no actuation input, this is encoded by the selection matrix $S \in \mathbb{R}^{n_{coordinates} \times n_u}$ which maps joint torques into generalized coordinates, yielding zero entries in the rows corresponding to the floating base. Although (3.31) describes the nonlinear dynamics of the robot, within the QP formulation it appears linear in the decision variables, since all state–dependent terms are evaluated at the current values of $\boldsymbol{\chi}$ and $\boldsymbol{\nu}$ and are therefore treated as known quantities.

Wheel–ground contact constraint. The interaction between the wheels and the ground is modeled through a kinematic constraint which enforces no slip at the contact interface. Specifically, under the pure rolling assumption, the instantaneous velocity of the wheel material point in contact with the ground is required to be zero. Let $\mathbf{v}_w$ be the linear velocity of the wheel center, $\boldsymbol{\omega}_{\text{wheel}}$ the angular velocity of the wheel rigid body, and $\mathbf{r}$ the vector from the wheel center to the wheel material point in contact with the ground. The rolling without slipping condition at the velocity level is:

$$\mathbf{v}_c = \mathbf{v}_w + \boldsymbol{\omega}_{\text{wheel}} \times \mathbf{r} = \mathbf{0}, \tag{3.32}$$

which equivalently yields

$$\mathbf{v}_w = -\,\boldsymbol{\omega}_{\text{wheel}} \times \mathbf{r}. \tag{3.33}$$

In the proposed QP formulation, constraints are enforced at the acceleration level. A direct differentiation of the velocity constraint requires care, since the acceleration of the material point of the wheel currently in contact with the ground, $\mathbf{a}_c$, is generally not zero under pure rolling. Indeed, for a rigid body,

$$\mathbf{a}_c = \mathbf{a}_w + \dot{\boldsymbol{\omega}}_{\text{wheel}} \times \mathbf{r} + \boldsymbol{\omega}_{\text{wheel}} \times \boldsymbol{\omega}_{\text{wheel}} \times \mathbf{r} \neq \mathbf{0}. \tag{3.34}$$

Several approaches have been proposed in the literature to obtain an acceleration level rolling constraint, including exploiting specific mechanical design of the wheel [14], neglecting contact point acceleration [18], adopting an Euler angle parametrization for the orientation of the wheel–fixed wheel frame to compute the contact point acceleration, [11], constraining relative accelerations at the interface [15], or using a dedicated parametrization of the contact position along the wheel contour [16].

In this work, we follow the derivation proposed in [17], which enforces the contact with the ground while allowing wheel pure rolling, tilting, and steering. The full derivation is reported in Appendix B. The resulting acceleration level constraint can be written as

$$\mathbf{a}_w = -\,\dot{\boldsymbol{\omega}}_{\text{wheel}} \times \mathbf{r} - \,\boldsymbol{\omega}_{\text{wheel}} \times \boldsymbol{\omega}_{\text{virtual}} \times \mathbf{r}, \tag{3.35}$$

where $\boldsymbol{\omega}_{\text{virtual}}$ is the angular velocity of a *virtual* frame that rotates with the wheel except for the component on the wheel rolling axis. Specifically, define the orthonormal basis

$$R_{\text{virtual}} = \begin{bmatrix} \mathbf{t} & \mathbf{n} & \mathbf{s} \end{bmatrix}, \tag{3.36}$$

where $\mathbf{t}$ is tangent to the rolling direction, $\mathbf{n}$ is normal to the wheel plane (e.g., aligned with the wheel joint axis), and $\mathbf{s}$ lies in the wheel plane and is chosen as the axis closest to the ground normal, see Figure B.4. The associated angular velocity is then obtained by projecting the wheel angular velocity onto the $(\mathbf{s}, \mathbf{t})$ plane

$$\boldsymbol{\omega}_{\text{virtual}} = \left(I - \mathbf{n}\mathbf{n}^T\right) \boldsymbol{\omega}_{\text{wheel}}. \tag{3.37}$$

For each wheel $i \in \{l, r\}$, constraint (3.35) is enforced in the QP through the linear and angular components of the wheel Jacobian, mapping $\mathbf{a}_w$ and $\dot{\boldsymbol{\omega}}_{\text{wheel}}$ to the generalized acceleration $\ddot{\mathbf{q}}$.

3.4.3 Inequality constraints

The inequality constraints in (3.24) enforce feasibility limits required for safe and physically consistent motion. They include actuator and joint bounds (torque limits, joint position and velocity limits) as well as contact constraints, which are essential to guarantee a valid interaction with the environment and to prevent slip during stance.

Actuation and joint bounds. Hardware limitations are enforced by bounding joint positions, velocities, and actuator torques. For each joint j, we impose

$$\mathbf{q}_j^{\min} \leq \mathbf{q}_j \leq \mathbf{q}_j^{\max}, \qquad \dot{\mathbf{q}}_j^{\min} \leq \dot{\mathbf{q}}_j \leq \dot{\mathbf{q}}_j^{\max}, \qquad \boldsymbol{\tau}_j^{\min} \leq \boldsymbol{\tau}_j \leq \boldsymbol{\tau}_j^{\max}. \tag{3.38}$$

In the present formulation, the position and velocity limits are enforced indirectly: the QP provides joint accelerations, which are integrated forward in time to compute joint velocities and positions.

Contact wrench constraints. Following the contact feasibility requirements described in [25], a contact wrench must satisfy: *unilaterality* (the contact can only push on the surface), *no–slip* (the tangential force must remain within friction limits), and, when a finite area of contact is present, *no–tilting surface* (the resultant moment must keep the Zero–Moment Point (ZMP) inside the support polygon).

Unilaterality. Let f_{iz} denote the normal component of the i–th ground reaction force expressed in the contact frame, with z oriented along the outward surface normal. Unilaterality is enforced by

$$f_{iz} \geq 0. \tag{3.39}$$

Friction cone. To prevent slip, the tangential components $\mathbf{f}_{it} = [f_{ix} \;\; f_{iy}]^T$ must satisfy the Coulomb friction cone constraint

$$\|\mathbf{f}_{it}\| \leq \mu f_{iz} \qquad \Longleftrightarrow \qquad \sqrt{f_{ix}^2 + f_{iy}^2} \leq \mu f_{iz}, \tag{3.40}$$

where μ is the static friction coefficient. Since this constraint is nonlinear, in the QP it is commonly replaced by a conservative polyhedral approximation, e.g.,

$$-\bar{\mu} f_{iz} \leq f_{ix} \leq \bar{\mu} f_{iz}, \qquad -\bar{\mu} f_{iz} \leq f_{iy} \leq \bar{\mu} f_{iz}, \tag{3.41}$$

with $0 < \bar{\mu} \leq \mu$.

No-tilting surface. When a rigid body establishes a planar surface contact, the resultant ground reaction wrench must be compatible with a stable contact over a finite area. In particular, the point of application of the equivalent ground reaction force (i.e., the ZMP) associated with that contact, must lie within the convex hull of the contact patch. In practice, this condition is enforced by constraining the admissible contact moments (or, equivalently, the ZMP coordinates) so that the resulting wrench remains physically realizable without loss of contact. A convenient implementation models the contact patch through a finite set of point forces (e.g., four forces located at the patch corners) without an explicit torsional component, and derives linear bounds on the corresponding contact wrench.

In humanoid robots, the foot sole provides a relatively large support area, allowing the generation of stabilizing contact moments within the support polygon. In wheeled legged robots, instead, the effective wheel–ground contact patch is typically much smaller and is often approximated as a narrow strip along the wheel width. As a consequence, the capability to generate contact moments is reduced; in particular, moments that require a lever arm along the rolling direction are strongly limited, and the admissible wrench set differs from that of a planar foot contact.

In this work the wheel–ground interaction is modeled as a narrow strip of rectangular shape, and the no–tilting condition is enforced through a four–point contact representation. Specifically, the contact patch is represented by four points at its endpoints where the forces are applied, and the no–tilting constraint is ensured by imposing unilaterality on these forces.

3.5 Trajectory generator

The trajectory generator module shown in Fig. 3.1 generates the reference trajectories used to guide the robot during locomotion tasks. It consists of two main blocks: the cruise trajectory generator and the jump trajectory generator. The two are selected by a trajectory selector, which is triggered by the obstacle detector.

3.5.1 Cruise trajectory generator

The cruise trajectory generator is a block that takes as input the reference velocities v^{ref}, ω^{ref}, and v_z^{ref}, and generates a reference trajectory to be tracked by the control block. In particular, it generates a reference trajectory for the CoM according to a unicycle kinematic model, which is integrated forward in time.

The unicycle kinematics are

$$\begin{cases} \dot{x} = v^{ref}\cos(\phi), \\ \dot{y} = v^{ref}\sin(\phi), \\ \dot{\phi} = \omega^{ref}. \end{cases} \tag{3.42}$$

These equations are integrated to obtain a dynamically feasible planar reference trajectory.

The vertical motion of the CoM is generated separately by integrating a constant vertical velocity v_z^{ref} while enforcing the height bounds

$$z^{\min} \leq z \leq z^{\max}. \tag{3.43}$$

In this work, the CoM height is constrained within the interval $z^{\min} = 0.25\,\mathrm{m}$ and $z^{\max} = 0.42\,\mathrm{m}$ for safety reasons.

Finally, the resulting reference trajectory describes a cruise phase in which the robot moves at the prescribed steady travel velocity while maintaining ground contact throughout the motion.

Accordingly, during the cruise phase, the references over the MPC prediction horizon are built as

$$\begin{cases} x_{com} = x, \\ y_{com} = y, \\ z_{com} = z \\ v^x_{com} = v^{ref} \cos(\phi), \\ v^y_{com} = v^{ref} \sin(\phi) \\ v^z_{com} = v^z_{ref}, \\ c_x = x, \\ c_y = y, \\ c_z = 0 \\ v_{cz} = 0, \\ \theta = \phi, \\ v = v^{ref}, \\ \omega = \omega^{ref}. \end{cases} \tag{3.44}$$

To simplify the notation, the dependence on the discrete time step is omitted.

3.5.2 Obstacle detector

The obstacle detector is a module that relies on sensor informations to estimate the potential risk of collision with a sudden obstacle appearing along the robot's path. Since the robot motion is designed by a trajectory generator that does not explicitly account for the presence of obstacles, this module plays a fundamental role in enabling obstacle avoidance.

The module estimates the distance between the robot and a potential obstacle located in front of it. Based on this estimated distance and the reference forward velocity of the robot, the module determines whether a collision risk exists. If necessary, a jump routine is triggered at an appropriate time in order to avoid the obstacle.

For simplicity, the implementation used in this work relies on the obstacle detection module already available on the robot platform. This module uses an infrared sensor mounted on the front of the robot to measure the distance from nearby obstacles within a maximum range of 1.5 m.

Although this solution provides a simple and effective preliminary implementation, more advanced computer vision techniques could significantly improve the reliability and accuracy of obstacle detection and distance estimation. Such approaches could overcome the limitations of the built–in infrared sensing method adopted in this work.

3.5.3 Jump trajectory generator

When an obstacle appears along the path, the trajectory selector is triggered by the obstacle detector and switches to the jump trajectory generator. This module plans a trajectory that enables the robot to jump over the obstacle, provided that the obstacle dimensions remain within the admissible jumping capabilities of the robot.

Inspired by human–like jumping, the vertical CoM reference motion is divided into three phases: *takeoff*, consisting of a loading step with a downward CoM motion to gather energy, followed by a push–off step that accelerates the CoM upward until toe–off (loss of ground contact); *flight*, during which there is no ground contact and the CoM follows a ballistic trajectory; and *landing*, during which the body, absorbs the impact, and stabilizes. The three phases are detailed below and are designed for $t \in [0, T]$, with total duration $T = T_{\text{takeoff}} + T_{\text{flight}} + T_{\text{landing}}$.

Takeoff. During takeoff, the CoM vertical trajectory is modeled as a cubic polynomial that interpolates the boundary conditions at the start, $(z_{\text{init}}, v_{\text{init}})$, and at the end, $(z_{\text{start_jump}}, v_0)$, using the normalized time variable $\tau \in [0, 1]$,

$$\tau = \frac{t}{T_{\text{takeoff}}} \in [0, 1], \tag{3.45}$$

$$z(\tau) = a_{\text{takeoff}}\,\tau^3 + b_{\text{takeoff}}\,\tau^2 + c_{\text{takeoff}}\,\tau + d_{\text{takeoff}}, \tag{3.46}$$

$$z(0) = z_{\text{init}}, \qquad z(1) = z_{\text{start_jump}}, \tag{3.47}$$

$$z'(0) = v_{\text{init}}\,T_{\text{takeoff}}, \qquad z'(1) = v_0\,T_{\text{takeoff}}, \tag{3.48}$$

$$z''(0) = -g\,T_{\text{takeoff}}^2. \tag{3.49}$$

The last condition is used to scale the timing of the motion so as to preserve feasibility, ensuring that the CoM vertical acceleration is never more negative than gravity.

Flight. In the flight phase, the CoM is in free fall and its vertical motion is governed solely by gravity. Let $t_0 = T_{\text{takeoff}}$ denote the toe–off time, i.e., the beginning of the flight phase. Given the initial conditions (z_{start}, v_0) at $t = t_0$, the vertical position evolves as

$$z(t) = z_{\text{start}} + v_0\,(t - t_0) - \frac{1}{2}g\,(t - t_0)^2. \tag{3.50}$$

To obtain a symmetric flight that reaches the maximum at h_{jump} above the toe–off height, observe that the maximum occurs at

$$t_{\max} = t_0 + \frac{v_0}{g}. \tag{3.51}$$

Substituting $t_{\max}$ into $z(t)$ yields

$$z(t_{\max}) - z_{\text{start}} = \frac{v_0^2}{2g} = h_{\text{jump}}, \tag{3.52}$$

hence the required takeoff vertical velocity is

$$v_0 = \sqrt{2g\,h_{\text{jump}}}. \tag{3.53}$$

Finally, enforcing touchdown at the same height as toe–off, gives

$$z(t_0 + T_{\text{flight}}) - z_{\text{start}} = v_0\,T_{\text{flight}} - \frac{1}{2}g\,T_{\text{flight}}^2 = 0, \tag{3.54}$$

from which the non–trivial solution is

$$T_{\text{flight}} = \frac{2v_0}{g}. \tag{3.55}$$

Therefore, once h_{jump} is selected, the ballistic trajectory is fully determined through v_0 and T_{flight}.

Landing. During landing, the CoM height is shaped through a cubic polynomial in the normalized time variable $\tau \in [0, 1]$, chosen to satisfy the touchdown and settling boundary conditions. In particular,

$$\tau = \frac{t - (T_{\text{takeoff}} + T_{\text{flight}})}{T_{\text{landing}}} \in [0, 1], \tag{3.56}$$

$$z(\tau) = a_{\text{landing}}\,\tau^3 + b_{\text{landing}}\,\tau^2 + c_{\text{landing}}\,\tau + d_{\text{landing}}, \tag{3.57}$$

$$z(0) = z_{\text{start_jump}}, \qquad z(1) = z_{\text{init}}, \tag{3.58}$$

$$z'(0) = v_{\text{start_jump}}\,T_{\text{landing}}, \qquad z'(1) = v_{\text{init}}\,T_{\text{landing}}, \tag{3.59}$$

$$z''(1) = -g\,T_{\text{landing}}^2. \tag{3.60}$$

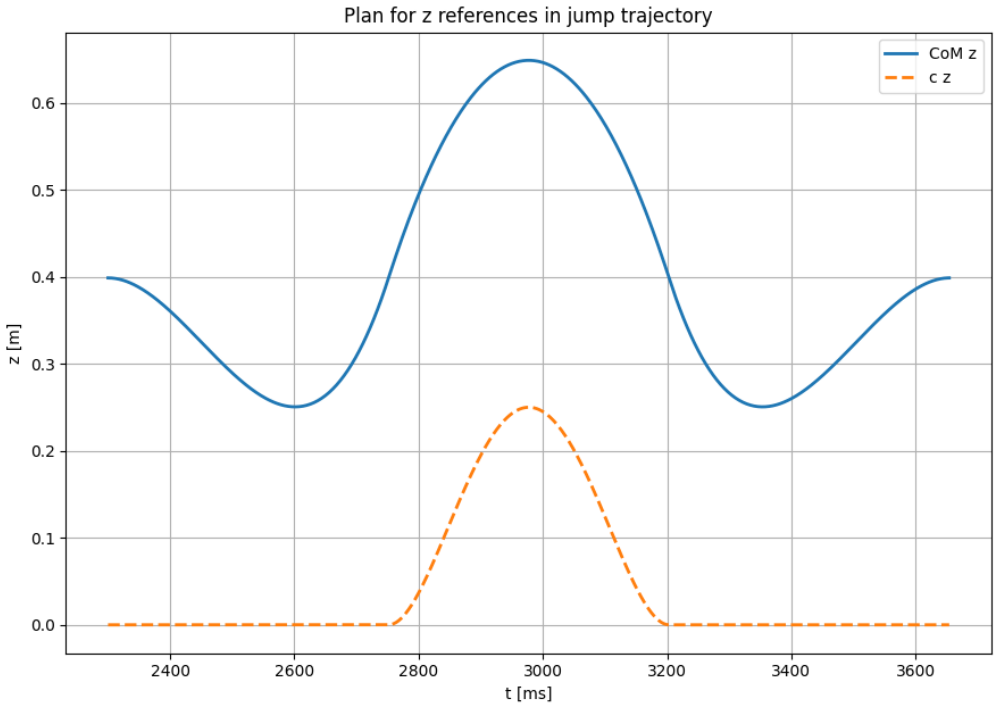


Figure 3.4. Example of the planned vertical trajectories after triggering the jump routine at $t = 2.3\,\mathrm{s}$. The plot reports the CoM height z and the unicycle–base height c_z, with $z_{\mathrm{init}} = z_{\mathrm{start_jump}} = 0.25\,\mathrm{m}$ and $h_{\mathrm{jump}} = z_{\mathrm{max,leg}} = 0.25\,\mathrm{m}$.

The final acceleration constraint is used to tune T_{landing} and keep the trajectory feasible, i.e., to prevent the commanded vertical acceleration from becoming more negative than gravity.

The vertical reference of the unicycle–base point in the template model is kept constant during the takeoff and landing phases. During the flight phase, instead, we introduce a smooth vertical profile to ensure a continuous and well–conditioned leg evolution. In particular, the base–point height is shaped through a quartic polynomial defined over the normalized flight time and constrained to (i) match the pre– and post–jump contact height, (ii) reach a prescribed apex at mid–flight, and (iii) have zero vertical velocity at lift–off and touchdown, so as to avoid abrupt kinematic transitions. The resulting reference is

$$\tau = \frac{t - T_{\mathrm{takeoff}}}{T_{\mathrm{flight}}} \in [0, 1], \tag{3.61}$$

$$z_c(\tau) = a_{\mathrm{leg}}\,\tau^4 + b_{\mathrm{leg}}\,\tau^3 + c_{\mathrm{leg}}\,\tau^2 + d_{\mathrm{leg}}\,\tau + e_{\mathrm{leg}}, \tag{3.62}$$

$$z_c(0) = z_c(1) = z_{0\mathrm{leg}}, \tag{3.63}$$

$$z_c\left(\tfrac{1}{2}\right) = z_{\mathrm{max,leg}}, \tag{3.64}$$

$$z_c'(0) = z_c'(1) = 0. \tag{3.65}$$

Here, $z_{\mathrm{max,leg}}$ is a free design parameter that can be tuned to improve the jump quality. An example of the resulting vertical jump trajectory is shown in Figure 3.4.

The horizontal reference during the jump is still generated from the forward integration of the unicycle model (see Section 3.5.1) so as to obtain planar references for both the CoM and the base point. However, since the jump maneuver has a limited duration, the corresponding linear and angular velocity references are generated through bang–bang acceleration profiles. To this end, scalar reference accelerations $a^{ref}(t)$ and $\alpha^{ref}(t)$ are defined over the whole maneuver duration T.

The horizontal reference in this phase is still obtained by the forward integration of the unicycle model to generate planar references for both the CoM and the base point but since in this case

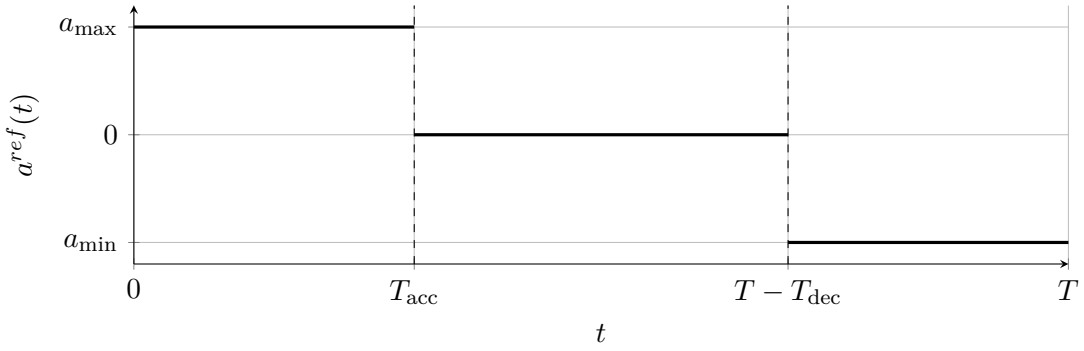


Figure 3.5. Bang–bang linear acceleration profile $a^{ref}(t)$ with acceleration, coasting, and deceleration phases.

the generated trajectory has a limited duration is it possible to generate the reference velocities through a bang–bang acceleration profiles giving scalar acceleration references $a^{ref}(t)$ and $\alpha^{ref}(t)$ over the total trajectory duration T. The profile consists of three intervals: constant acceleration for T_{acc}, a coasting phase with zero acceleration, and constant deceleration for T_{dec} (Fig. 3.5),

$$a^{ref}(t) = \begin{cases} a_{\max} & \text{for } t \in [0,\, T_{\text{acc}}] \\ 0 & \text{for } t \in [T_{\text{acc}},\, T - T_{\text{dec}}] \\ a_{\min} & \text{for } t \in [T - T_{\text{dec}},\, T] \end{cases} \tag{3.66}$$

$$\alpha^{ref}(t) = \begin{cases} \alpha_{\max} & \text{for } t \in [0,\, T_{\text{acc}}] \\ 0 & \text{for } t \in [T_{\text{acc}},\, T - T_{\text{dec}}] \\ \alpha_{\min} & \text{for } t \in [T - T_{\text{dec}},\, T] \end{cases} \tag{3.67}$$

which yield the corresponding velocity references

$$v^{ref}(t) = \begin{cases} a_{\max}\, t & \text{for } t \in [0,\, T_{\text{acc}}] \\ a_{\max}\, T_{\text{acc}} = v_{\text{peak}} & \text{for } t \in [T_{\text{acc}},\, T - T_{\text{dec}}] \\ v_{\text{peak}} + a_{\min}\,(t - (T - T_{\text{dec}})) & \text{for } t \in [T - T_{\text{dec}},\, T] \end{cases} \tag{3.68}$$

$$\omega^{ref}(t) = \begin{cases} \alpha_{\max}\, t & \text{for } t \in [0,\, T_{\text{acc}}] \\ \alpha_{\max}\, T_{\text{acc}} = \omega_{\text{peak}} & \text{for } t \in [T_{\text{acc}},\, T - T_{\text{dec}}] \\ \omega_{\text{peak}} + \alpha_{\min}\,(t - (T - T_{\text{dec}})) & \text{for } t \in [T - T_{\text{dec}},\, T] \end{cases} \tag{3.69}$$

For simplicity, we set $a_{\min} = -a_{\max}$. Therefore, the trajectory is fully specified by the phase durations (T, T_{acc}, T_{dec}) and the peak velocities (v_{peak}, ω_{peak}).

3.6 State filter

For the deployment of the proposed control framework on a real robot, the availability of a reliable state estimation module is essential. In particular, the MPC requires an estimate of the robot center of mass position and velocity, together with the pose and velocity of the virtual base. These quantities are also needed by the whole body controller, since they enter the kinematic and dynamic computations used for motion tracking and constraint enforcement.

In principle, CoM and virtual base quantities can be reconstructed from the floating base and joint states through forward and differential kinematics. However, on a real platform these variables are not directly available with sufficient accuracy, due to sensor noise, modeling errors, and contact uncertainties. As a result, a state filtering procedure is required in order to provide a reliable and consistent estimate of the robot state for the overall control architecture.

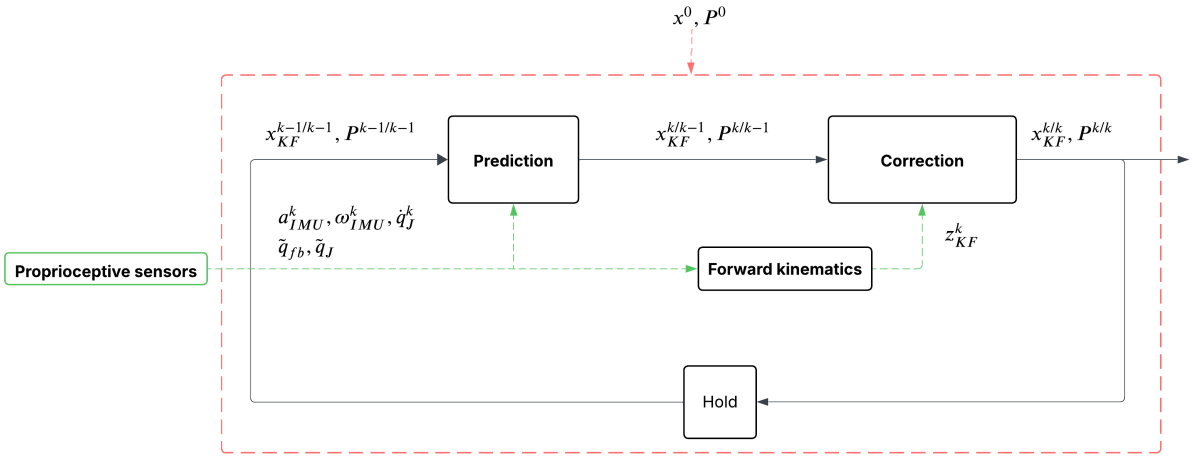


Figure 3.6. Block diagram of the proposed filter architecture.

This section presents the adopted state estimation scheme, inspired by [20], and modified to account for wheel–ground contact kinematics under the pure rolling assumption. In contrast with more general floating base estimators, the proposed formulation introduces additional simplifying assumptions, most notably treating the floating base orientation as a known quantity rather than an estimated state. Under these assumptions, the estimation model becomes linear in both the state and the input variables, which allows the use of a standard Kalman Filter instead of a nonlinear estimator.

The state and input vectors of the filter are defined as follows:

$$\mathbf{x}_{KF} = \begin{pmatrix} \mathbf{p}_{fb} & \mathbf{v}_{fb} & \mathbf{p}_{cl} & \mathbf{p}_{cr} \end{pmatrix}^T \quad \mathbf{u}_{KF} = \begin{pmatrix} \mathbf{a}_{\mathrm{IMU}} & \boldsymbol{\omega}_{IMU} & \dot{\mathbf{q}}_{\mathrm{J}} \end{pmatrix}^T \tag{3.70}$$

The available proprioceptive measurements are the IMU specific acceleration $\mathbf{a}_{\mathrm{IMU}}$, the IMU angular velocity $\boldsymbol{\omega}_{\mathrm{IMU}}$ and the joint encoder positions and velocities, $\tilde{\mathbf{q}}_J$ and $\dot{\mathbf{q}}_J$. It is also provided the base orientation expressed as a quaternion $\tilde{\mathbf{q}}_{fb}$. In the proposed model, the IMU frame is assumed to coincide with the floating base frame.

More precisely, the accelerometer measures the specific force expressed in the IMU frame, while the gyroscope provides the angular velocity of the base expressed in the base frame. The quantities $\tilde{\mathbf{q}}_{fb}$ and $\tilde{\mathbf{q}}_J$ are treated in the proposed model as known noise–free parameters, and are therefore used to construct both the prediction and correction models rather than being included in the estimated state.

This is a strong simplifying assumption, since the floating base orientation is not estimated within the proposed filter but is instead taken from an internal estimation module already available on the robot. Under this assumption, the process and measurement models become linear with respect to the filter state and inputs, thus allowing the use of a standard linear Kalman filter.

An overview of the proposed filter architecture is provided in Fig. 3.6 in the form of a block diagram.

prediction model

The continuous–time prediction model propagates the floating base position and velocity using the inputs:

$$\dot{\mathbf{p}}_{fb} = \mathbf{v}_{fb}, \tag{3.71}$$

$$\dot{\mathbf{v}}_{fb} = \mathbf{R}_{IMU}(\tilde{\mathbf{q}}_{fb})\mathbf{a}_{IMU} + \mathbf{g}. \tag{3.72}$$

where $\mathbf{R}_{IMU}(\tilde{\mathbf{q}}_{fb}) \in SO(3)$ denotes the rotation matrix associated with the quaternion $\tilde{\mathbf{q}}_{fb}$, mapping vectors from the base frame to the inertial frame, and $\mathbf{g}$ is the gravity vector expressed in the inertial frame.

The above expression follows from the fact that the accelerometer measures specific force rather than the absolute linear acceleration. Therefore, once rotated into the inertial frame, the gravity contribution must be added back to recover the floating–base linear acceleration.

When wheel $i \in \{l, r\}$ is in contact with the ground, wheel odometry can be exploited by propagating the corresponding contact point position $\mathbf{p}_{c,i}$ under the pure rolling assumption. The contact point velocity is constrained to lie along the wheel rolling direction $\hat{\mathbf{t}}_i$ and its magnitude is given by the wheel angular speed about the wheel axis $\hat{\mathbf{n}}_i$:

$$\dot{\mathbf{p}}_{c,i} = \hat{\mathbf{t}}_i(\tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J)\ v_{w,i}, \tag{3.73}$$

$$v_{w,i} = r_{wheel}\, \hat{\mathbf{n}}_i(\tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J)^T\, \mathbf{J}_{wi}^{\text{ang}}(\mathbf{0}, \tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J) \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\omega}_{IMU} \\ \dot{\mathbf{q}}_J \end{bmatrix}, \tag{3.74}$$

where r_{wheel} denotes the wheel radius.

The raw accelerometer and gyroscope data are modeled as being subject to additive thermal noise processes. The matrix $\mathbf{J}_{wi}^{\text{ang}}$ is the angular part of the Jacobian associated with wheel i, and $\hat{\mathbf{t}}_i$ and $\hat{\mathbf{n}}_i$ are the unit vectors of the wheel virtual frame defined in (3.36). Equation (3.73) is applied only when the corresponding wheel is in contact with the ground. One might alternatively adopt a direct wheel encoder–based model, however:

$$\dot{\mathbf{p}}_{c,i} = \hat{\mathbf{t}}_i(\tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J)\ \dot{q}_{wi}\, r_{\text{wheel}} \tag{3.75}$$

would implicitly assume that the wheel rolling rate coincides with the measured wheel joint rate, i.e., $\omega_{\text{roll},i} = \dot{q}_{wi}$. For wheeled legged robots this assumption can be violated when internal leg motions induce relative rotations at the wheel joint that are not associated with actual rolling of the wheel, making the odometry unreliable in those situations.

For this reason, the model in (3.73) is adopted: it uses differential kinematics to extract the wheel rolling angular velocity through the wheel Jacobian, thereby providing a more consistent estimate of the contact point velocity during rolling. The drawback is an increased sensitivity to noise in $\tilde{\mathbf{q}}_{fb}$ and $\boldsymbol{\omega}_{IMU}$, since these quantities also affect the Jacobians and the projection directions.

Finally, the prediction model is discretized via forward Euler integration assuming constant inputs over the sampling interval.

Correction model

The correction step exploits the relative position between the floating base and the wheel–ground contact points, reconstructed through forward kinematics from the floating base orientation and joint configuration. In particular, for each wheel $i \in \{l, r\}$, the forward kinematics provides the contact point position relative to the base as

$$\mathbf{p}_{c,i} - \mathbf{p}_{fb} = \text{FWKinematics}_i(\mathbf{0}, \tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J)\,, \tag{3.76}$$

where $\text{FWKinematics}_i(\cdot)$ returns the position of the I–th wheel–ground contact point with respect to the floating base, expressed in the inertial frame.

Accordingly, the measurement vector used in the correction step is defined as

$$\mathbf{z}_{KF} = \begin{pmatrix} \text{FWKinematics}_L(\mathbf{0}, \tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J) \\ \text{FWKinematics}_R(\mathbf{0}, \tilde{\mathbf{q}}_{fb}, \tilde{\mathbf{q}}_J) \end{pmatrix}. \tag{3.77}$$

From the state definition, the same relative quantities can be written as

$$\mathbf{z}_{KF} = \begin{pmatrix} \mathbf{p}_{c,l} - \mathbf{p}_{fb} \\ \mathbf{p}_{c,r} - \mathbf{p}_{fb} \end{pmatrix} = \mathbf{H}_{KF}\, \mathbf{x}_{KF}, \tag{3.78}$$

with observation matrix

$$\mathbf{H}_{KF} = \begin{pmatrix} -I_3 & 0 & I_3 & 0 \\ -I_3 & 0 & 0 & I_3 \end{pmatrix}. \tag{3.79}$$

Therefore, the correction step enforces consistency between the estimated state and the contact points relative position reconstructed from the robot kinematics. Since the resulting correction model is linear with respect to the filter state, it can be carried out through the standard Kalman filter update.

Finally, it is pointed out that the filter is used only to estimate the floating base position and linear velocity, while the base orientation and angular velocity are taken directly from the proprioceptive measurements.

Chapter 4

Results

This chapter presents the results obtained by testing the proposed control framework in simulation. While validation on the real platform would have been of great interest, the challenges associated with hardware implementation are substantial and thus motivate its consideration as future work.

In the conducted tests, the controller is evaluated in different scenarios, including locomotion, jumping maneuvers, and disturbance rejection, in order to assess its performance and robustness.

As a practical implementation note, in order to achieve a reliable control rate the controller was implemented in C++. The nonlinear MPC problem is solved using *Crocoddyl*[26], an optimal control library that efficiently implements the DDP algorithm.

The implemented NMPC has a prediction horizon of 50 knots and an integration interval of 0.01 s. Under these design choices, the controller runs at an average frequency of approximately 500 Hz.

4.1 Simulation results

The proposed control framework was evaluated in simulation using the MuJoCo physics engine (see Section 1.4). MuJoCo provides a high–fidelity simulation environment that allows accurate modeling of the dynamics of complex systems, including contacts and actuator behavior. The objective of these experiments is to assess the performance of the control architecture in representative locomotion, disturbance rejection and obstacle avoidance scenarios and to evaluate the interaction between the Model Predictive Controller and the Whole Body Controller.

4.1.1 Locomotion

The first experiment is aimed at evaluating the locomotion capabilities enabled by the proposed control framework. In this scenario, the robot is required to track a reference trajectory generated by the cruise trajectory generator described in Section 3.5.1, thereby performing a basic locomotion task while maintaining balance. An illustration of the resulting platform motion is reported in Fig. 4.1.

The reference trajectory is defined for both the robot center of mass and the unicycle base according to the generator described in Section 3.5.1. In particular, the trajectory is generated using a forward reference velocity of $v_{peak} = -0.8$ m/s and an angular reference velocity of $\omega_{peak} = 0.5$ rad/s. In addition, a constant vertical reference velocity $v_z^{ref} = -0.1$ m/s is applied until the lower admissible height limit is reached.

Figures 4.2 and 4.3 report the evolution of the MPC state and input variables together with their corresponding predictions along the control horizon.

From these results it can be observed that the control inputs evolve smoothly over time, indicating stable behavior of the predictive controller. Small discrepancies between predicted and realized trajectories can be observed. These differences are mainly caused by the loop closure

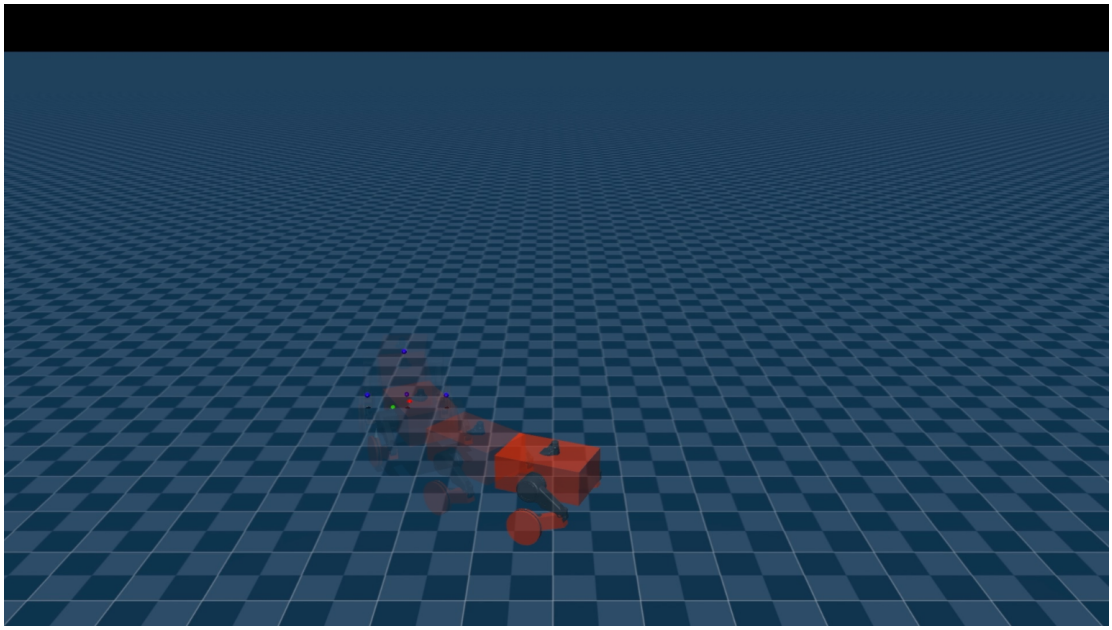


Figure 4.1. Capture of the robot locomotion during the experiment.

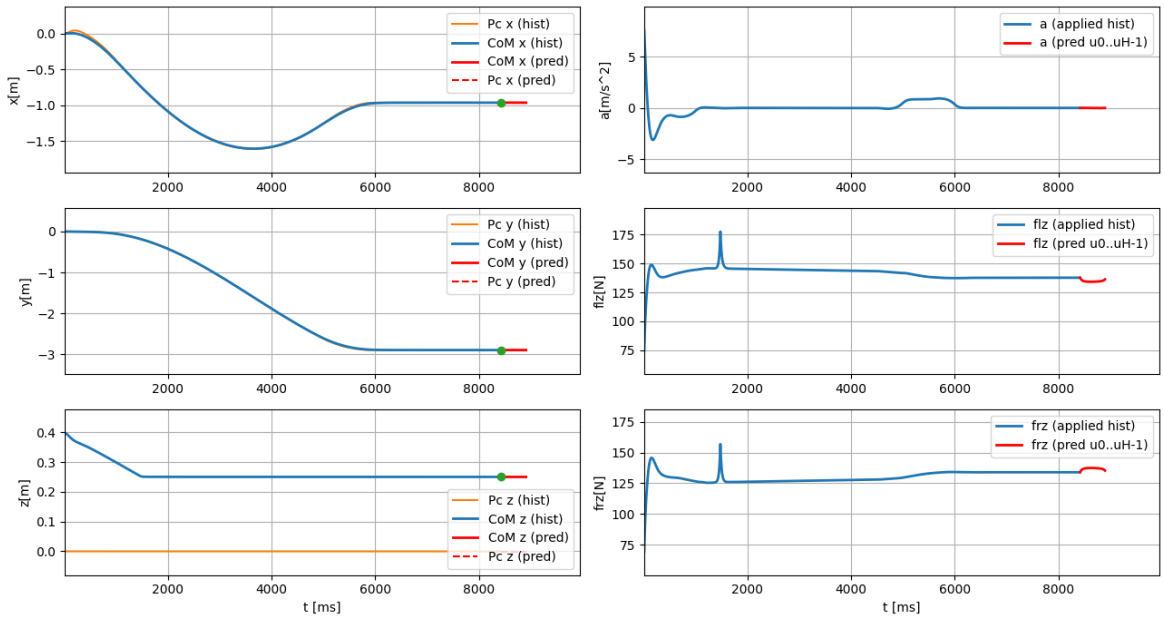


Figure 4.2. MPC predictions during the locomotion task, on the left, the history (solid lines) and predicted trajectories (dashed lines) of the CoM position (blue) and unicycle base motion (orange). The right column reports the base accelerations and the vertical components of the contact forces.

between the simplified template model used by the MPC and the full robot dynamics controlled by the WBC.

Despite these discrepancies, the MPC demonstrates robust behavior and consistently generates feasible trajectories that stabilize the robot while tracking the reference motion. Furthermore, the predicted contact forces remain physically consistent, confirming that the optimization respects the contact constraints imposed in the model.

The trajectories generated by the MPC are subsequently tracked by the Whole Body Controller, and the corresponding tracking errors are shown in Figure 4.4. Throughout the locomotion task, the WBC maintains a very small tracking error, with a maximum deviation of approximately 2 mm,

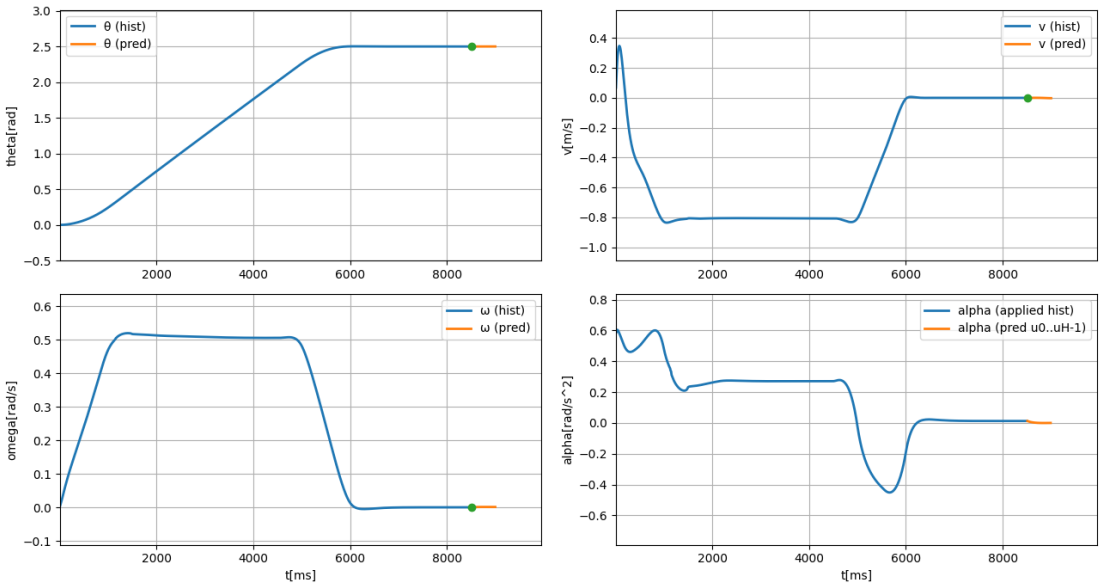


Figure 4.3. The evolution of the scalar velocity together with the orientation, angular velocity, and angular acceleration of the unicycle base.

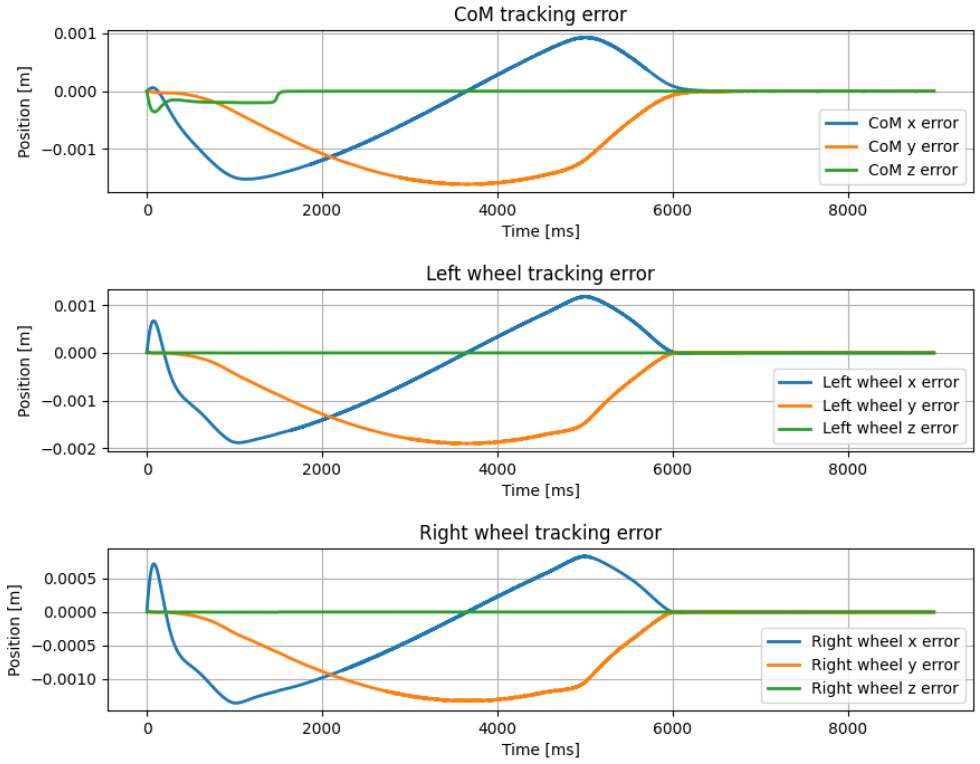


Figure 4.4. WBC tracking error of the CoM and of the left and right wheel centers during the locomotion task.

thereby confirming its effectiveness in accurately reproducing the motion planned by the MPC.

The effectiveness of the hierarchical control structure becomes more evident when comparing the executed CoM trajectories with the corresponding reference trajectories, as shown in Figure 4.5. The overall plots indicate a deviation of less than 2 cm from the cruise trajectory. This behavior can be attributed to the fact that the MPC is primarily designed to generate dynamically feasible maneuvers while guaranteeing the stability of the overall platform. As a consequence, the reference

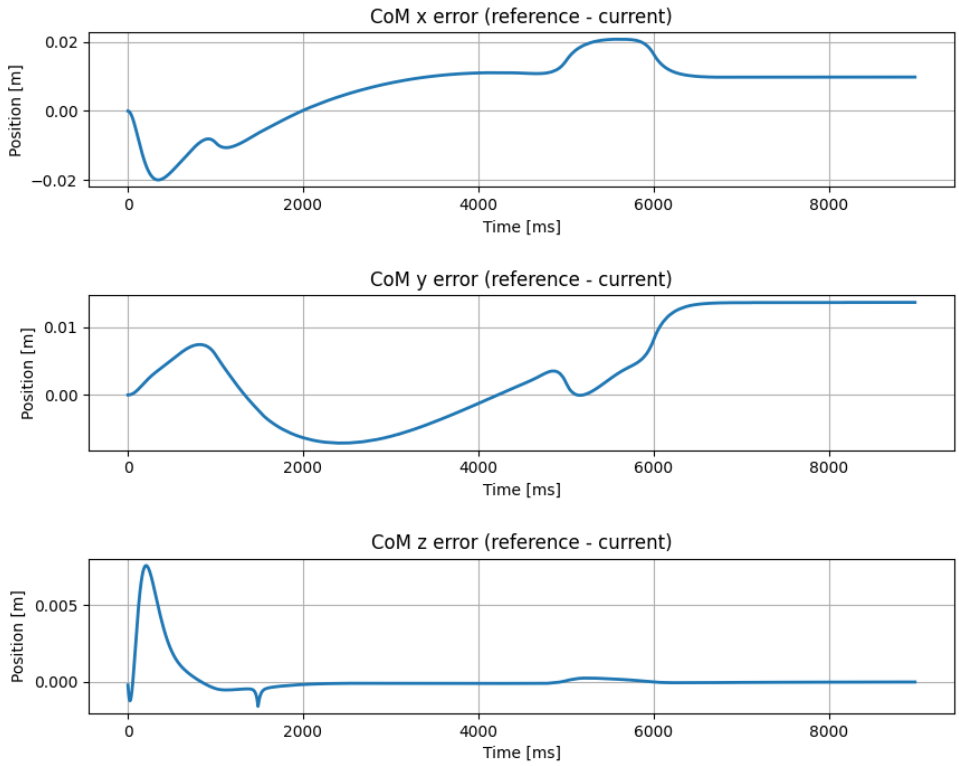


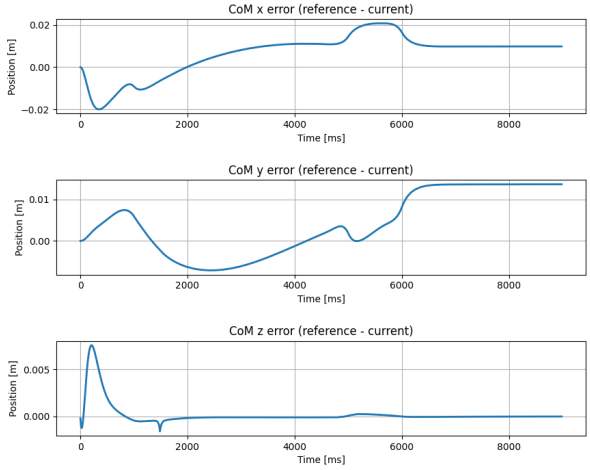
Figure 4.5. Deviation of the CoM executed trajectories from the generated cruise reference trajectory.

trajectory is tracked as closely as possible, provided that system stability is not compromised. Accordingly, a small residual tracking error may still be observed under steady–state conditions. This occurs because the robot can converge to a configuration in which the tracking error cannot be completely eliminated without violating the imposed constraints, such as the no lateral sliding condition. Nevertheless, this effect becomes less significant when more aggressive tracking weights are selected in the MPC and WBC cost functions.

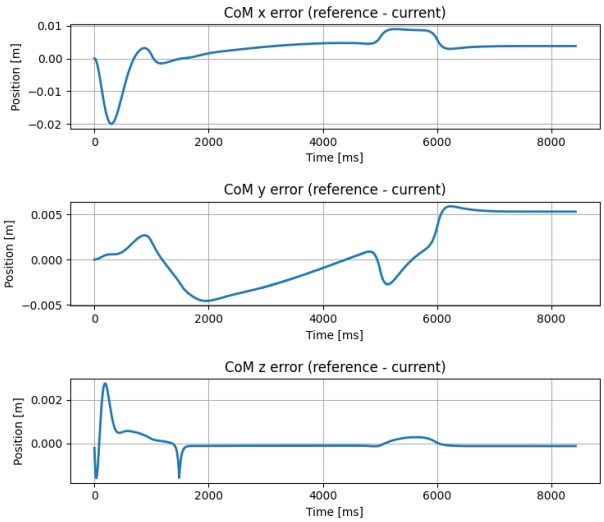
A quantitative comparison of the tracking performance obtained with standard, medium, and aggressive weight tuning is presented in Figure 4.6a, Figure 4.6b, and Figure 4.6c, respectively. Starting from the baseline set of weights adopted for generic locomotion tasks, corresponding to the performance shown in Figure 4.6a, it can be observed in the considered case that progressively increasing the weights associated with reference CoM tracking in the MPC cost function, together with assigning larger weights to the tracking tasks in the WBC QP, results in a reduction of the steady–state tracking error. This trend is evident in Figure 4.6b and Figure 4.6c.

Finally, the control effort required to execute the locomotion task is reported in Figure 4.7. The plot illustrates the evolution of the control inputs over the control cycles. The control signals remain smooth and bounded, indicating that the locomotion behavior can be achieved without excessive actuator demands. This confirms that the generated motion is dynamically feasible and that the control framework operates within realistic actuation limits.

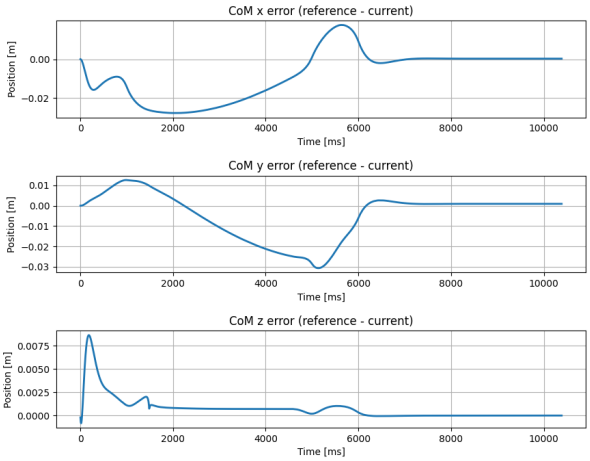
The timing performance of each iteration cycle is illustrated in Figure 4.8. The results show that the overall controller operates within approximately 2 ms per iteration, with only one spike above this value, and achieves an average control period of 819.1 μs. Although, on average, occasional timing spikes are present, they are primarily caused by the computational load associated with solving the MPC optimization problem, which represents the main challenge in terms of computational efficiency. Nevertheless, the tested controller is able to run reliably within the required real–time constraints, enabling deployment in locomotion tasks with an average control frequency of approximately 500 Hz.



(a) Standard tracking weights in MPC and WBC.



(b) Medium tracking weights in MPC and WBC.



(c) Aggressive tracking weights in MPC and WBC.

Figure 4.6. Deviation of the executed CoM trajectories from the generated cruise reference trajectory for standard, medium, and aggressive tracking weights in MPC and WBC.

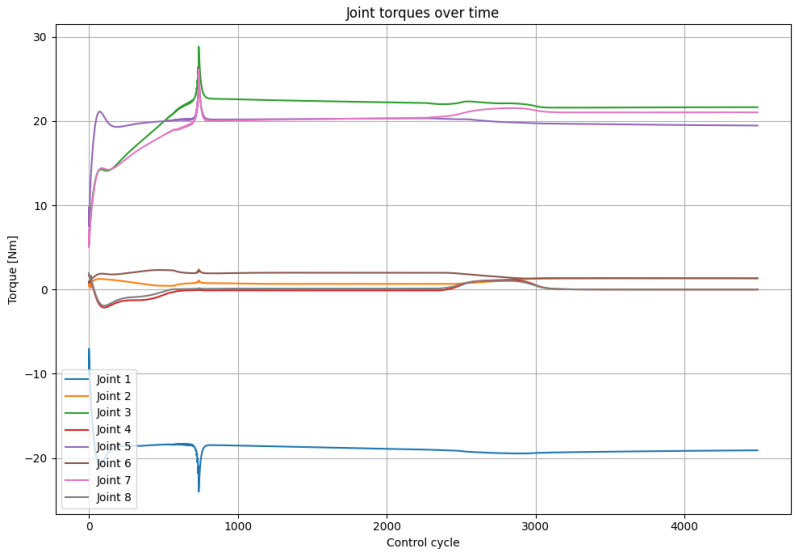


Figure 4.7. Control effort applied by the controller during the locomotion task.

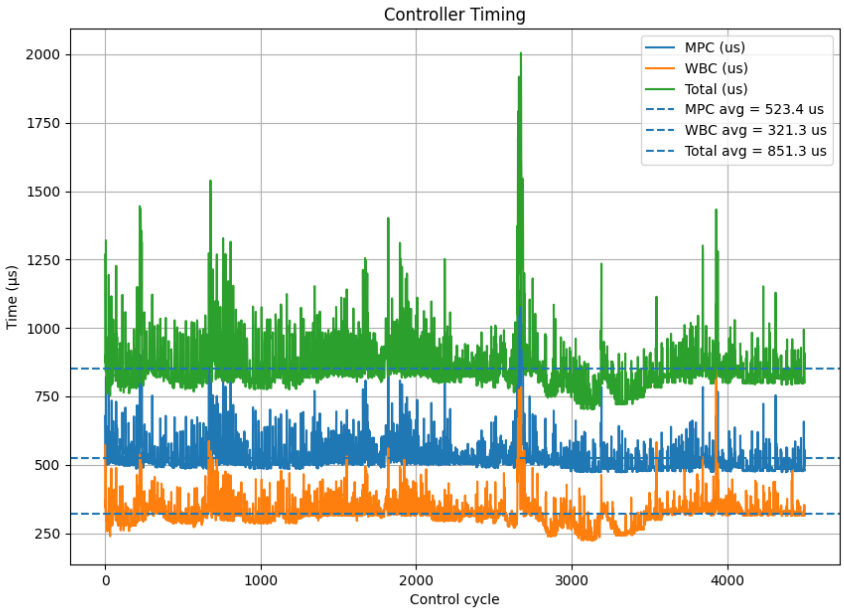
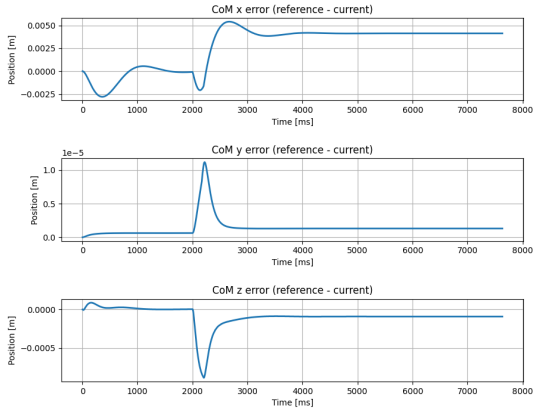


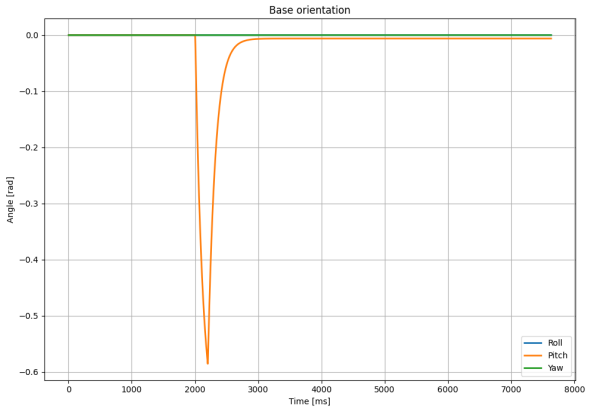
Figure 4.8. Controller timing per control cycle. In blue the MPC computational time is shown, in orange the WBC time, and in green the overall controller period.

4.1.2 Disturbance rejection

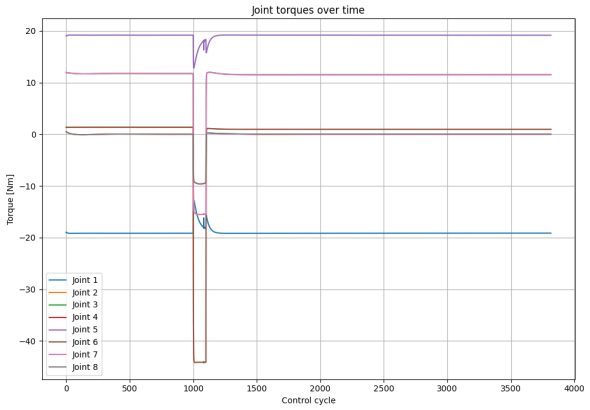
In the second experiment, the robot is required to maintain its initial configuration while being subjected to a set of impulsive disturbances of 200 ms duration, with the aim of evaluating the robustness of the control framework and its capability to reject external perturbations. To this end, representative disturbances acting along the principal directions, namely forward, lateral, and vertical, are considered.



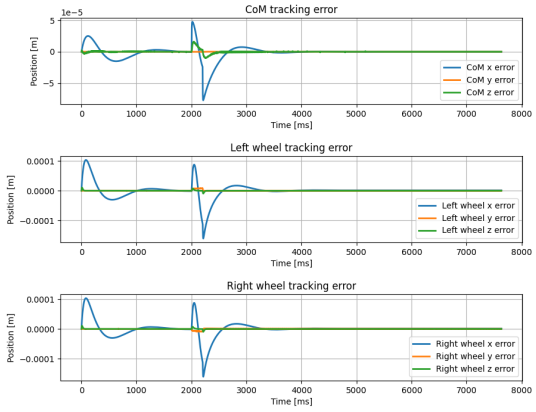
(a) Reference tracking error under the applied disturbance.



(b) Platform orientation expressed in roll–pitch–yaw Euler angles.



(c) Control effort generated to reject the disturbance.



(d) WBC tracking error during the disturbance rejection task.

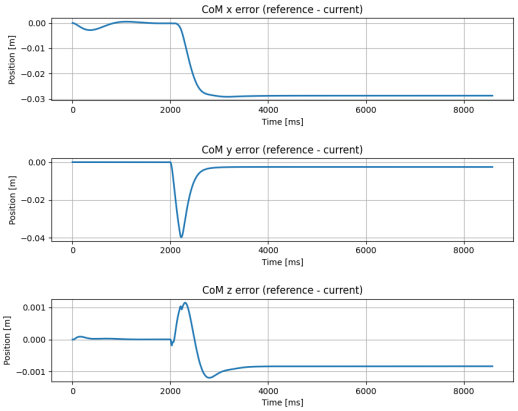
Figure 4.9. Tracking performance, platform orientation, control effort, and WBC tracking error for the disturbance rejection test with applied force $f_d^1 = (200 \;\; 0 \;\; 0)^T$ N.

Forward disturbance

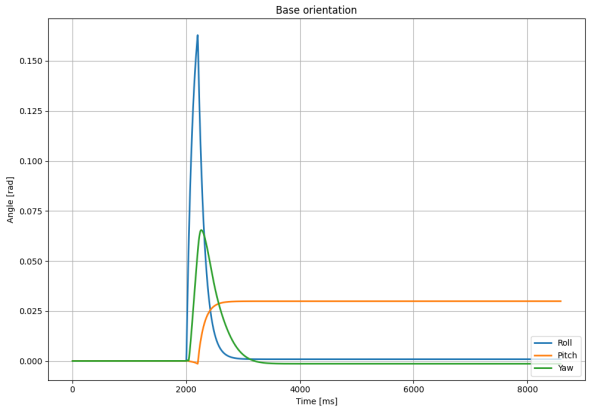
The first considered disturbance is a horizontal force $f_d^1 = (200 \;\; 0 \;\; 0)^T$ N, applied to the robot base for 200 ms in the forward direction. The controller exhibits robust behavior in rejecting this perturbation, while requiring a limited control effort, as shown in Figure 4.9c. The tracking performance with respect to the reference trajectory and the WBC tracking error are reported in Figure 4.9a and Figure 4.9d, respectively. The evolution of the platform orientation, expressed in roll–pitch–yaw Euler angles, is shown in Figure 4.9b.

After the transient phase, the platform converges to a configuration characterized by a small nonzero steady–state error in the CoM position with respect to the reference, together with a nonzero pitch angle. These residual errors remain limited and originate from the same effects discussed in the previous experiment. Since the control architecture does not guarantee perfect convergence to a zero–error steady–state configuration even under nominal conditions, such offsets cannot be fully eliminated in the presence of disturbances.

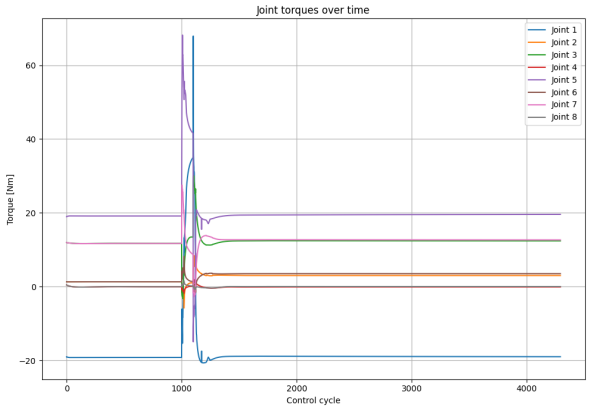
Nevertheless, despite this residual offset, the control framework proves capable of effectively counteracting a disturbance of this magnitude in the forward direction and of preserving the balance of the platform.



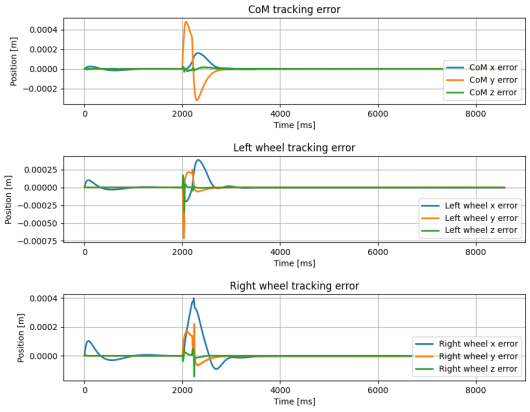
(a) Reference tracking error under the applied disturbance.



(b) Platform orientation expressed in roll–pitch–yaw Euler angles.



(c) Control effort generated to reject the disturbance.



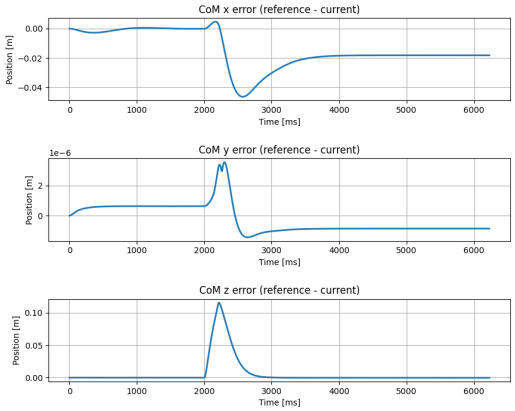
(d) WBC tracking error during the disturbance rejection task.

Figure 4.10. Results of the disturbance rejection test corresponding to the applied force $f_d^2 = (0 \;\; 160 \;\; 0)^T$ N: reference tracking error, platform orientation, control effort, and WBC tracking error.

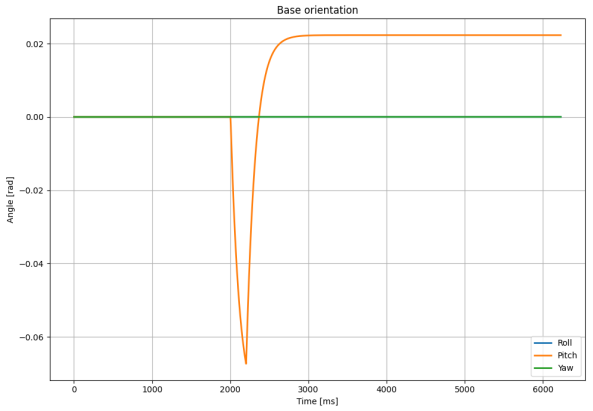
Lateral disturbance

The second considered disturbance is a horizontal force $f_d^2 = (0 \;\; 160 \;\; 0)^T$ N, applied to the robot base for 200 ms along the lateral direction. Also in this case, the control framework is able to react promptly to the perturbation and maintain the stability of the platform.

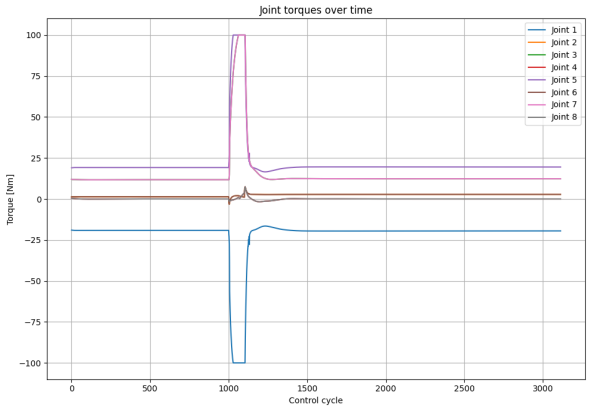
The tracking performance with respect to the reference trajectory, the platform orientation, the control effort, and the WBC tracking error are reported in Figure 4.10a, Figure 4.10b, Figure 4.10c, and Figure 4.10d, respectively. In contrast to the previous case, this disturbance primarily excites the lateral dynamics of the robot. Since lateral motion is more strongly constrained by the platform design, the perturbation may lead to a residual steady–state offset in the CoM position. This behavior can be explained by the fact that the applied external force may induce a slight violation of the no–sliding condition at the contact level, resulting in a small displacement of the contact point along a constrained direction. Consequently, the robot settles into a configuration that is less favorable for fully recovering the nominal reference. Nevertheless, the results confirm that the controller is capable of rejecting the disturbance and recovering a stable equilibrium without loss of balance.



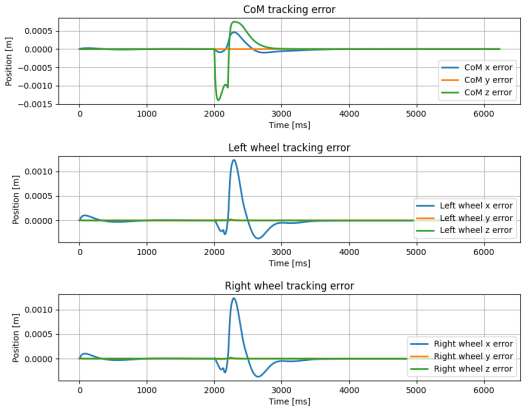
(a) Reference tracking error under the applied disturbance.



(b) Platform orientation expressed in roll–pitch–yaw Euler angles.



(c) Control effort generated to reject the disturbance.



(d) WBC tracking error during the disturbance rejection task.

Figure 4.11. Results of the disturbance rejection test corresponding to the applied force $f_d^3 = (0 \;\; 0 \;\; -1000)^T$ N: reference tracking error, platform orientation, control effort, and WBC tracking error.

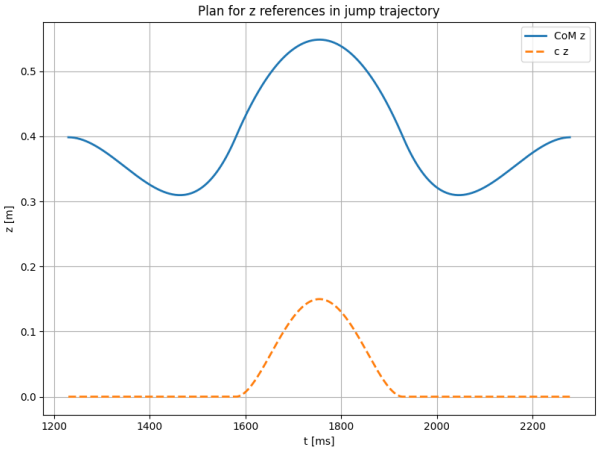
Vertical disturbance

The third considered disturbance is a vertical force $f_d^3 = (0 \;\; 0 \;\; -1000)^T$ N, of considerable magnitude applied to the robot base for 200 ms. The tracking performance with respect to the reference trajectory, the platform orientation, the control effort, and the WBC tracking error are reported in Figure 4.11a, Figure 4.11b, Figure 4.11c, and Figure 4.11d, respectively.

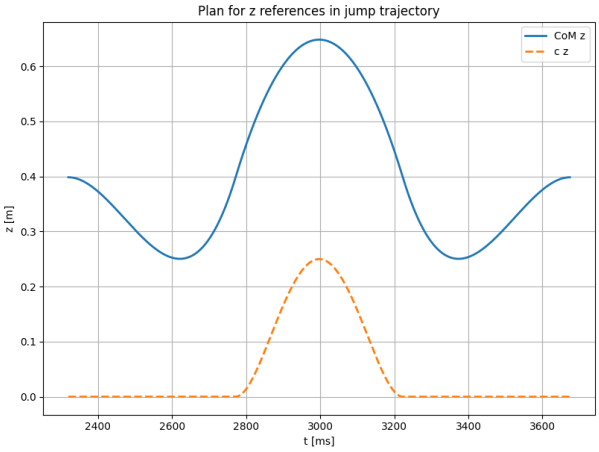
Unlike the horizontal disturbance cases, the applied force acts along the vertical direction and mainly results in a temporary compression of the robot structure. Consequently, the controller responds by increasing the contact forces to counterbalance the external load while keeping the platform stable. The plots show that the perturbation is effectively rejected and that the robot is able to recover a stable equilibrium after the transient. Only a contained deviation from the nominal reference is observed, confirming the robustness of the control framework also under significant impulsive vertical loads.

4.1.3 Sequential jumping over multiple obstacles

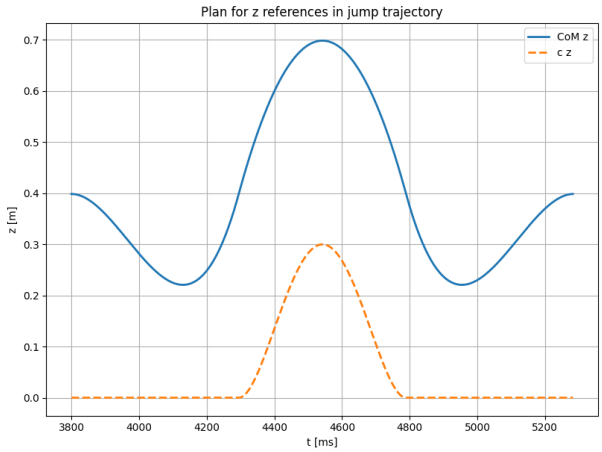
In this experiment, the robot is evaluated on its ability to perform a sequence of three dynamic jumps over obstacles of increasing height while tracking a reference trajectory. The nominal reference trajectory is generated under the assumption of an obstacle–free environment with $v^{ref} = 1.2$ m/s and $\omega^{ref} = 0$ rad/s.



(a) The planned jump trajectory for the second obstacle reaches a maximum leg height of 15 cm.



(b) The planned jump trajectory for the second obstacle reaches a maximum leg height of 25 cm.

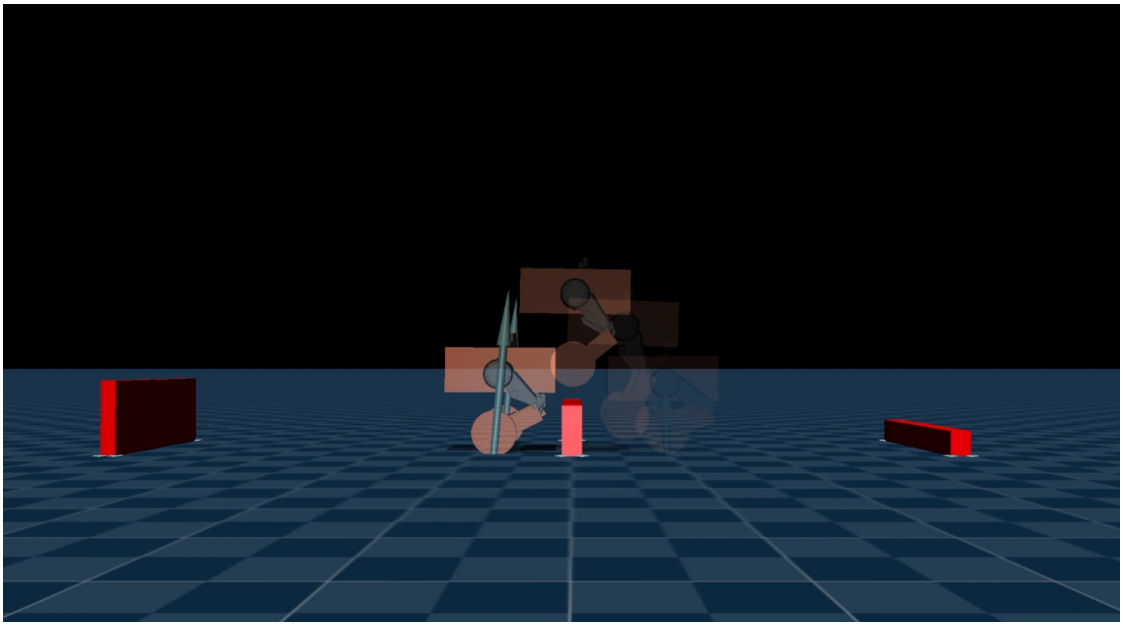


(c) The planned jump trajectory for the second obstacle reaches a maximum leg height of 30 cm.

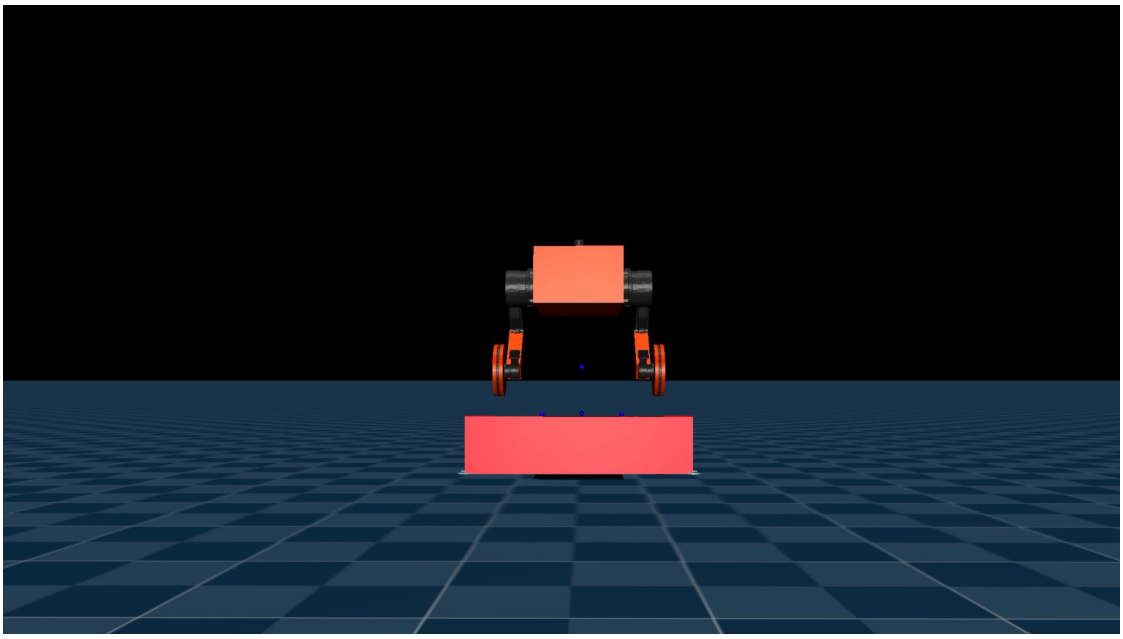
Figure 4.12. Vertical jump trajectories generated by the jump trajectory generator corresponding to three obstacles of increasing height.

To emulate the sudden appearance of obstacles, the jump trajectory generator described in Section 3.5.3 is triggered when the robot reaches a predefined distance from each obstacle. This module generate a feasible reference for the MPC state so as to generate a jump of the required height. The resulting trajectories are shown for the first obstacle, with dimensions $8 \times 80 \times 10$ cm^3, in Figure 4.12a; for the second obstacle, with dimensions $8 \times 80 \times 20$ cm^3, in Figure 4.12b; and for the third obstacle, with dimensions $8 \times 80 \times 30$ cm^3, in Figure 4.12c. An illustration of the jump motion is reported in Figures 4.13a and 4.13b for reference.

In order to execute a jump maneuver, the robot must go through three main phases. The first is the *takeoff* phase, in which the robot reaches a tucked configuration to increase its elastic potential energy and then pushes against the ground to increase the CoM acceleration. During this phase, a subtle but important precaution is introduced: for approximately 100 ms before toe–off, the weight associated with angular momentum penalization in the WBC control objective is increased, in order to prevent the platform from starting the flight phase with a nonzero angular momentum. This is necessary because, once the robot is in free fall, such angular momentum cannot be recovered. The second phase is the *flight* phase, during which the CoM follows a ballistic trajectory and the legs execute smooth and continuous motions that prepare the robot for landing. The final phase is the *landing* phase, in which the robot cushions the impact and returns to the nominal height maintained before the jump.



(a) Side–view illustration of the jumping motion, reconstructed from a sequence of snapshots acquired during the *takeoff*, *flight*, and *landing* phases.

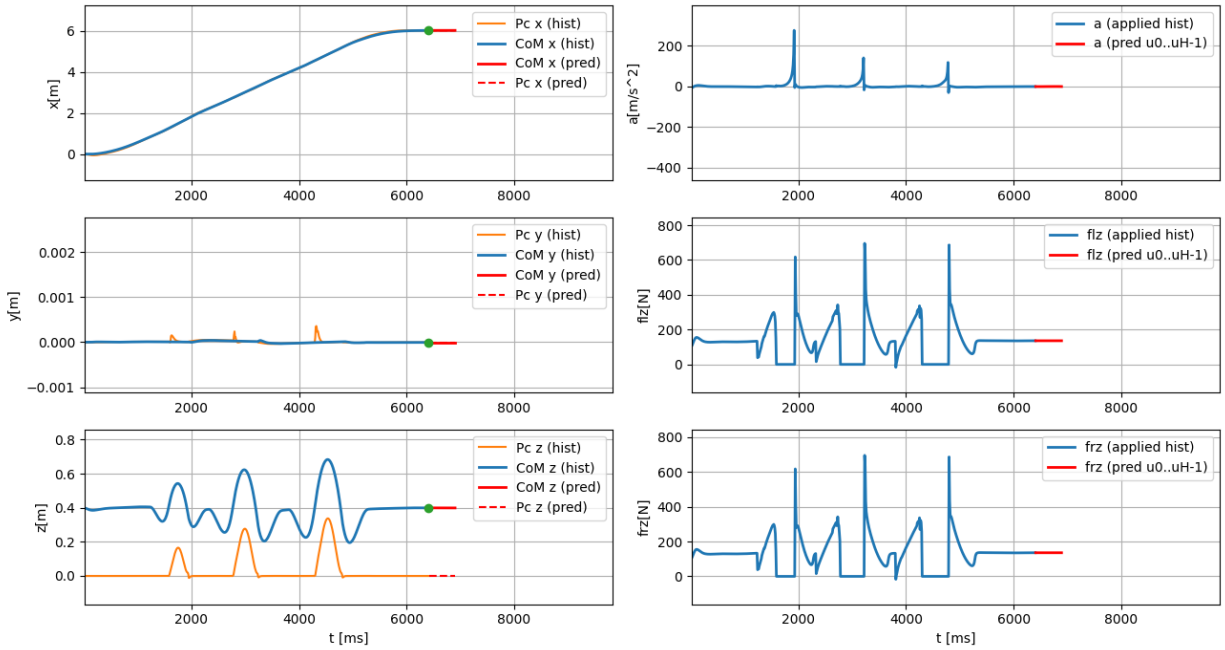


(b) Front view of the robot at the maximum height reached during the *flight* phase.

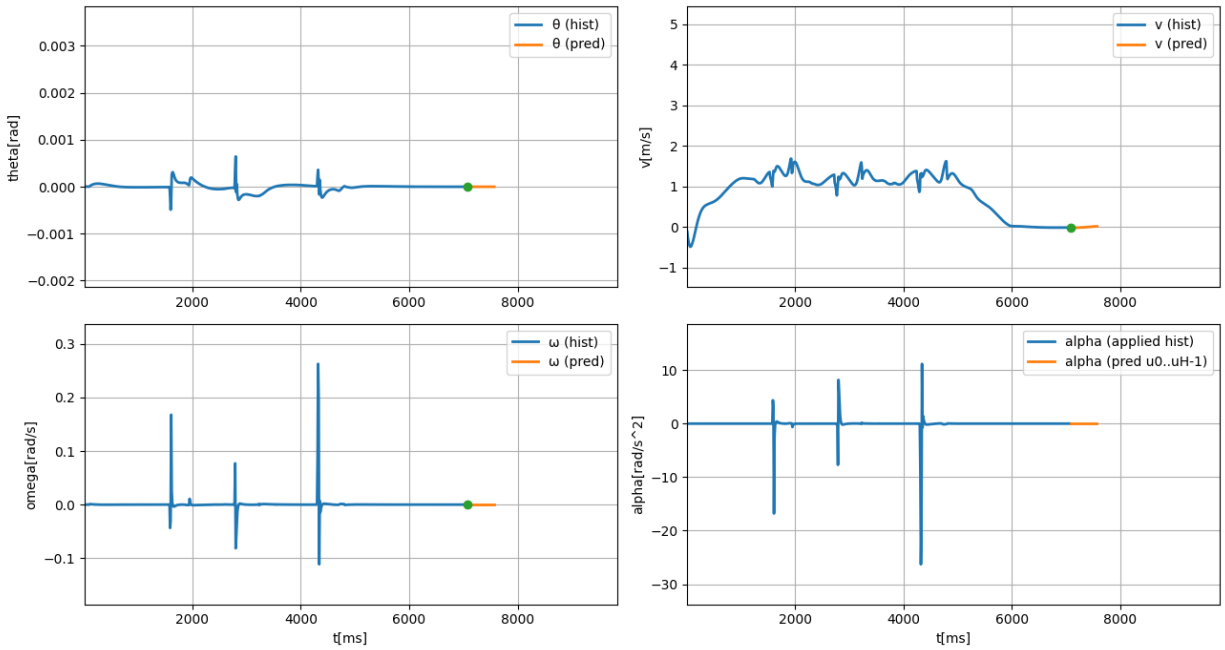
Figure 4.13. Illustration of the jump motion from side and front viewpoints.

The MPC predictions reported in Figures 4.14a and 4.14b show a behavior consistent with the expected flight dynamics. In particular, during the flight phase the contact forces are driven to zero, while the vertical dynamics of the unicycle base is allowed to follow the reference swing trajectory. This confirms that the predictive controller correctly captures the transition from the contact phase to the ballistic phase of the maneuver. From the tracking error reported in Figure 4.15, it can be observed that the WBC is able to accurately track the trajectories generated by the MPC also during the jumps.

The tracking error of the CoM with respect to the reference cruise trajectory is reported in Figure 4.16, while the evolution of the platform orientation in roll–pitch–yaw Euler angles is shown in Figure 4.17. From these plots, it is evident that the robot deviates from the nominal reference



(a) Evolution of the CoM and unicycle–base together with the corresponding MPC predictions. The right column also reports the base accelerations and the vertical components of the contact forces.



(b) Evolution of the scalar velocity, orientation, angular velocity, and angular acceleration of the unicycle base, together with the corresponding MPC predictions.

Figure 4.14. MPC state and input predictions during the execution of the three jump maneuvers.

trajectory in correspondence with the jump maneuvers, particularly along the vertical direction, since the cruise reference is generated with a constant CoM height. Nevertheless, once the jump is completed and the obstacle is overcome, the robot resumes tracking of the cruise trajectory. Similarly, the platform orientation exhibits a pitch variation in proximity to each jump, which is effectively recovered after the maneuver.

A final remark concerns the control effort required to execute the maneuver. As shown in Figure 4.18, the joint torques reach their highest values during the *takeoff* and *landing* phases,

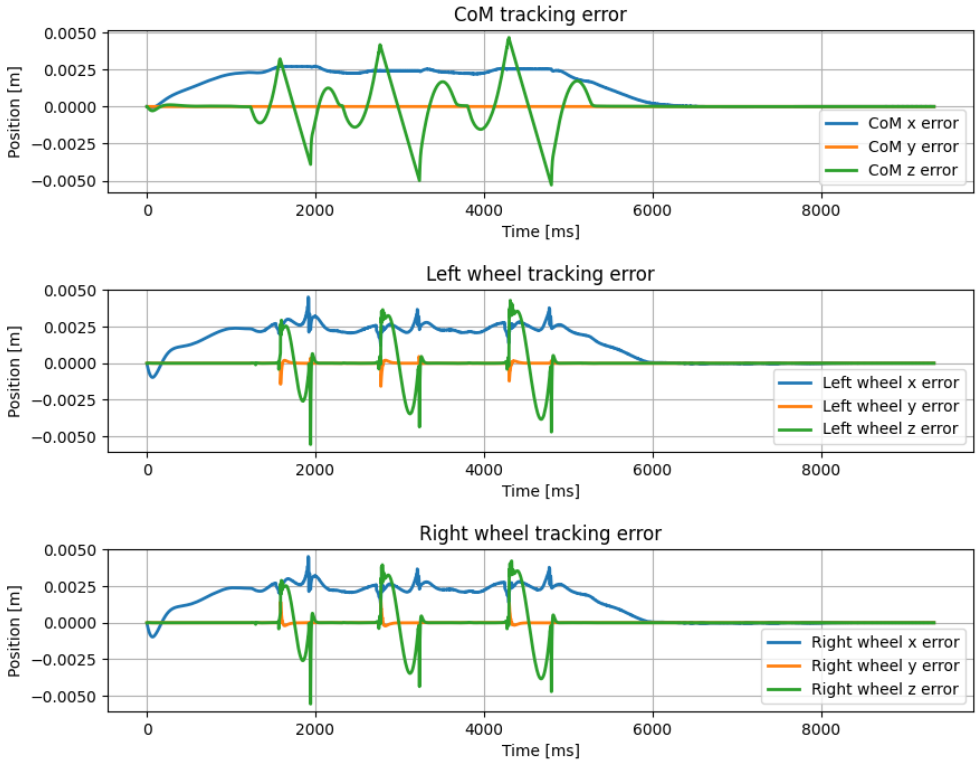


Figure 4.15. WBC tracking error with respect to the MPC generated desired trajectories during the execution of the three jump maneuvers.

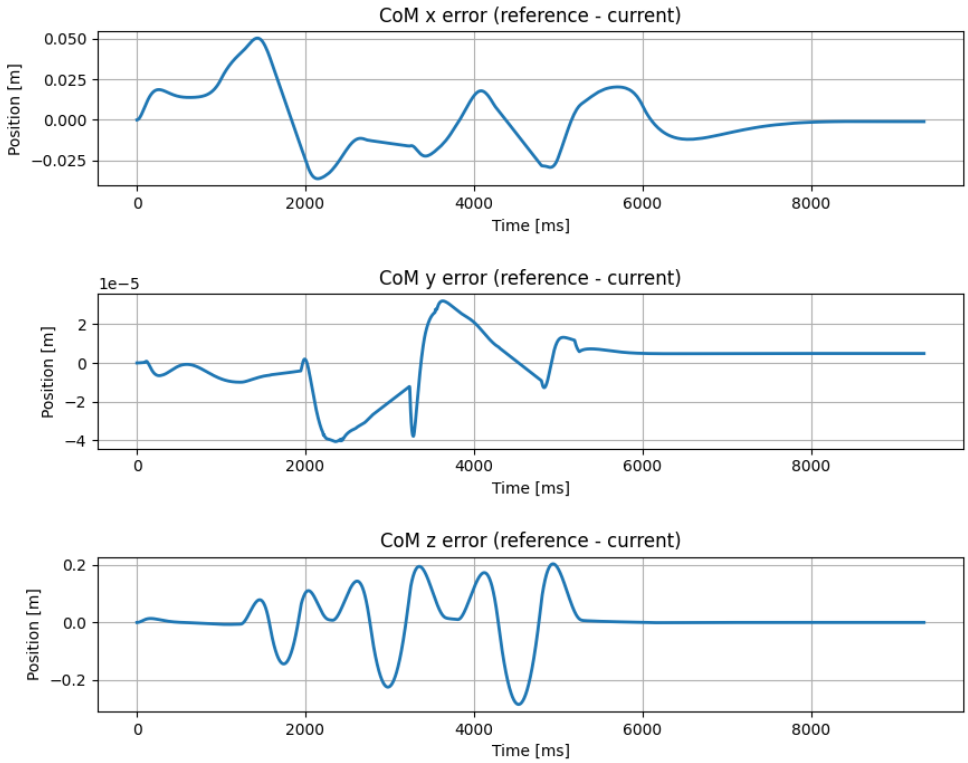


Figure 4.16. Tracking error of the CoM with respect to the cruise reference trajectory during the execution of the three jump maneuvers.

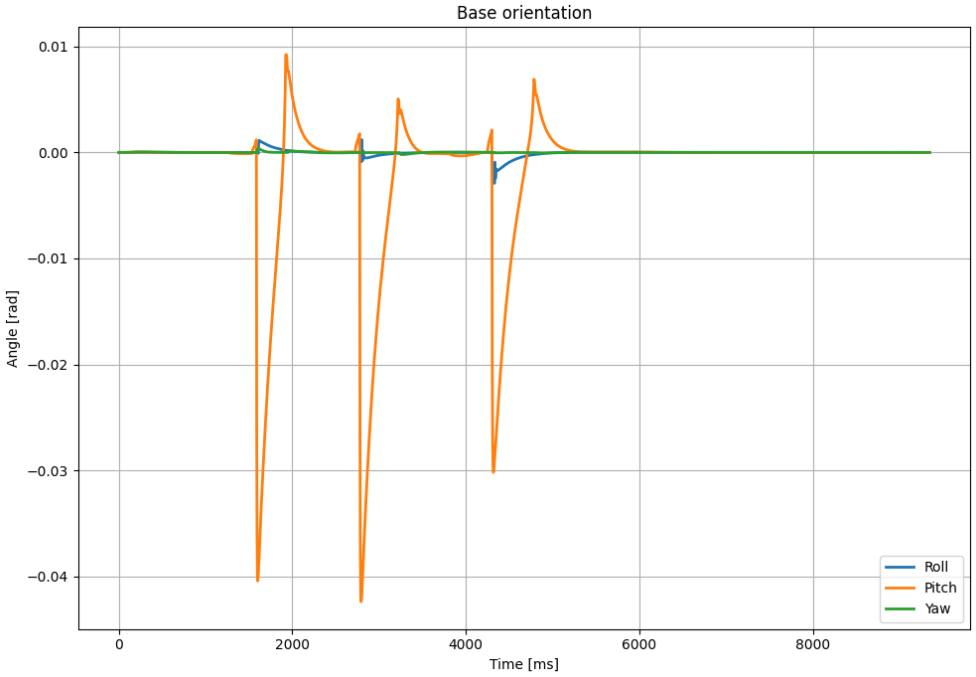


Figure 4.17. Evolution of the platform orientation, expressed in roll–pitch–yaw Euler angles, during the execution of the three jump maneuvers.

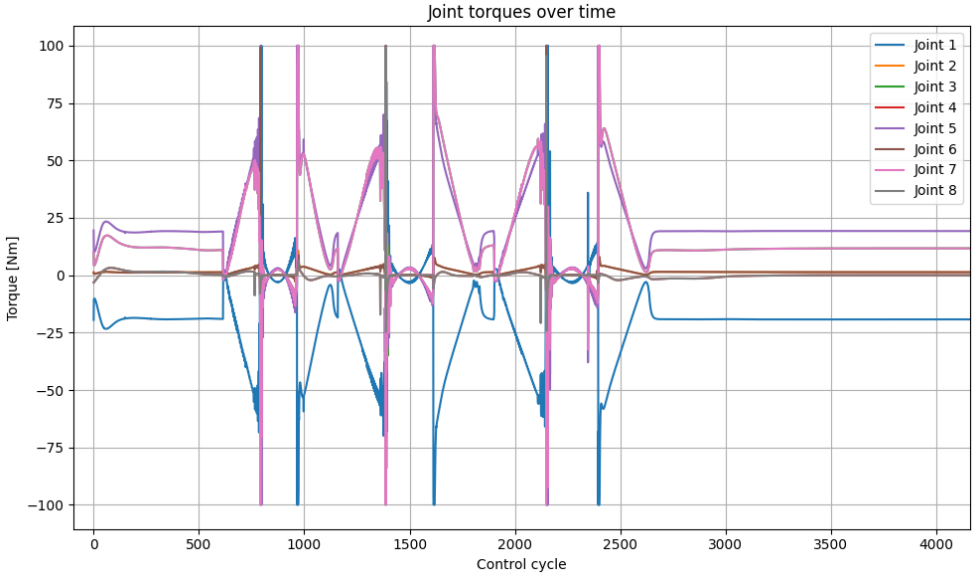


Figure 4.18. Joint torque profiles during the execution of the three jump maneuvers. The largest torque peaks occur during the *takeoff* and *landing* phases.

especially for the shoulder joints (joint 1 and joint 5) and elbow joints (joint 3 and joint 7), which are primarily responsible for generating the vertical acceleration needed for the jump. In these phases, the torque values approach saturation, while still remaining within the physical limits of the motors. The timing results shown in Figure 4.19 confirm that the jump maneuvers are executed within the available control period, thus demonstrating the computational feasibility of the proposed framework also in the presence of highly dynamic motions.

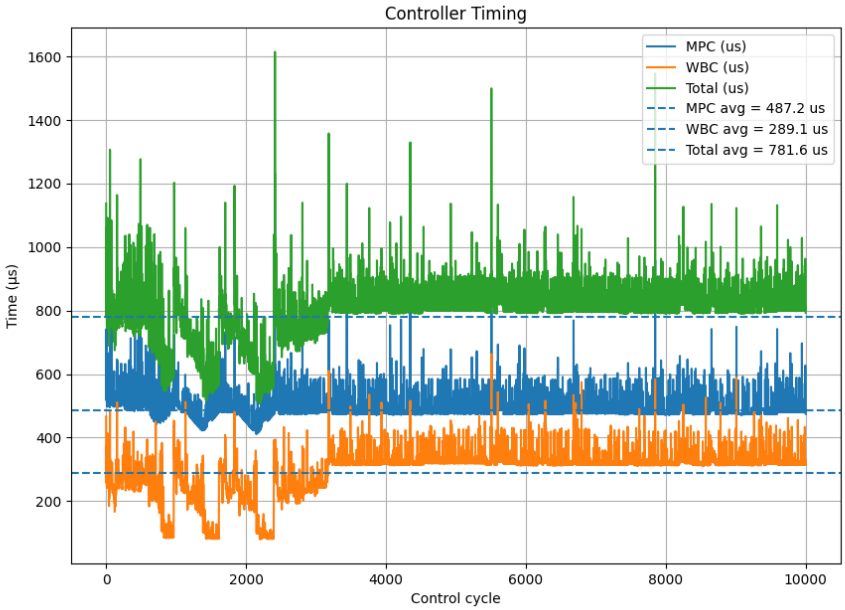


Figure 4.19. Computation times of the control framework during the execution of the three jump maneuvers. All timing values remain within the available control period, confirming the real–time feasibility of the proposed approach.

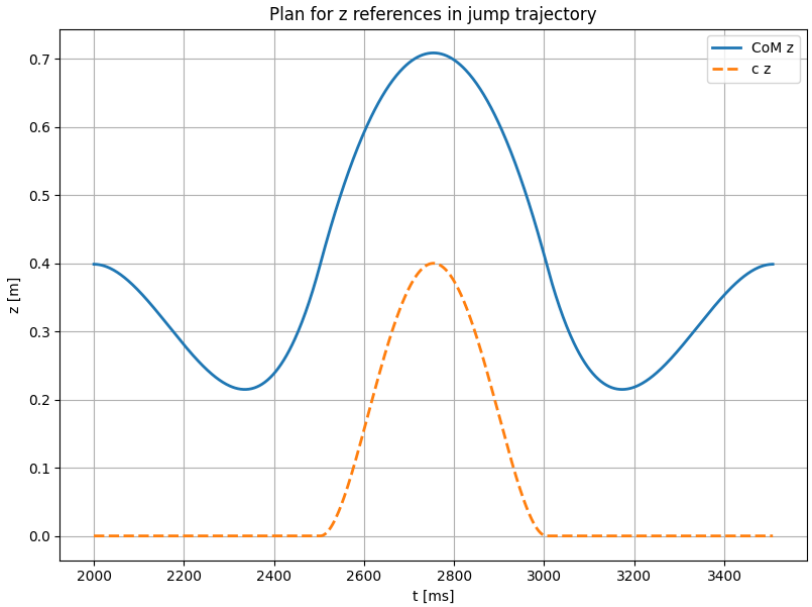
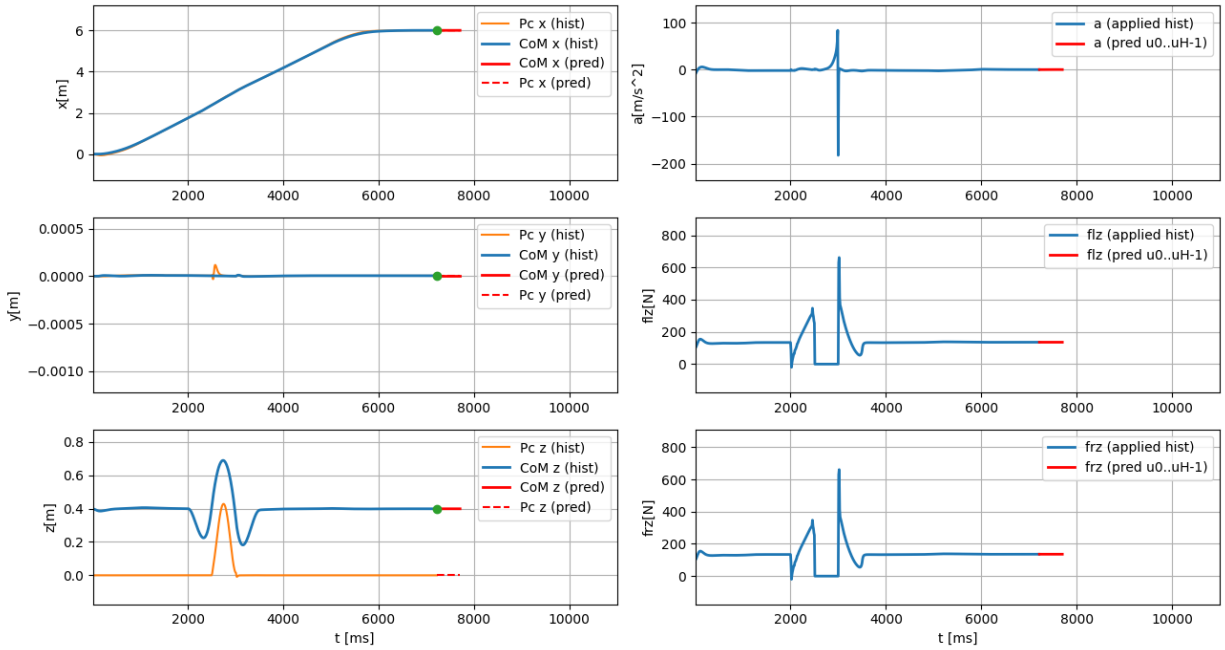


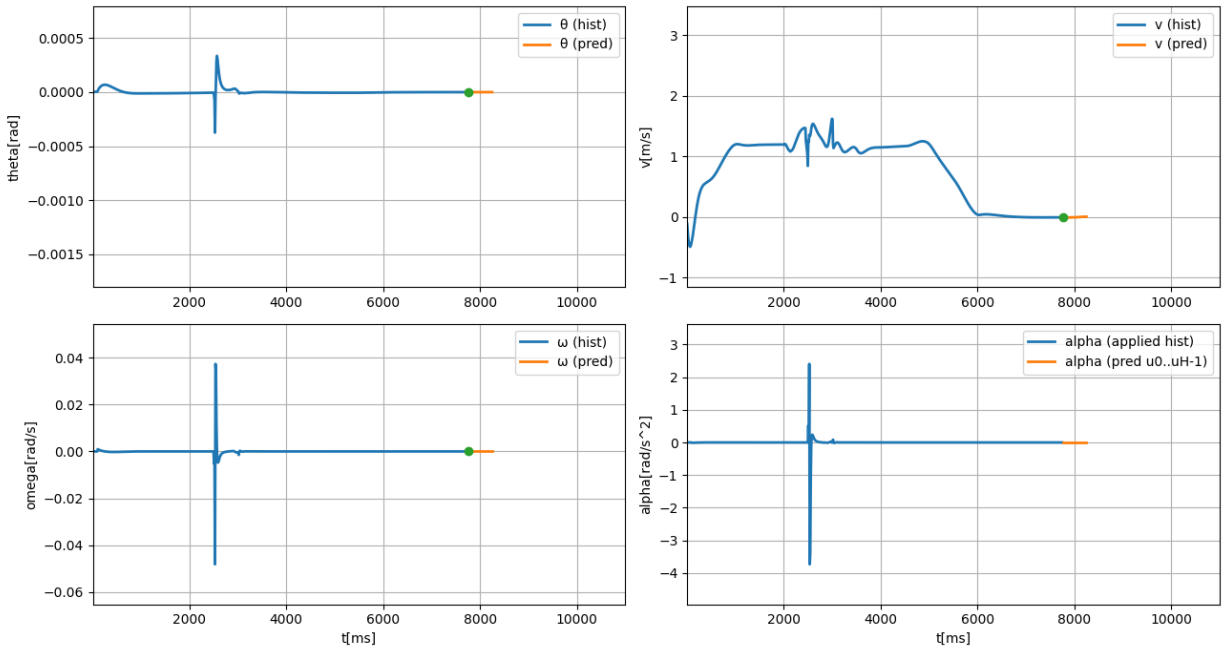
Figure 4.20. Reference vertical jump trajectory for the maximum–height jump experiment. The reference is designed to allow the robot to overcome an obstacle of dimensions $8 \times 2 \times 40$ cm^3.

4.1.4 Maximum height jump

This experiment evaluates the maximum jump height achieved by the robot in simulation. Following the same cruise trajectory described in the previous experiment, the robot performs a jump to avoid a rectangular box obstacle of dimensions $2 \times 80 \times 40$ cm^3. To generate the jump reference trajectories, a maximum CoM vertical displacement of 30 cm is planned, while the legs perform a larger excursion, reaching a peak displacement of 41 cm, thereby allowing the robot to successfully avoid the obstacle. The planned jump trajectory is reported in Figure 4.20. Also in this case,



(a) Evolution of the CoM and unicycle–base states together with the corresponding MPC predictions. The right column also reports the base accelerations and the vertical components of the contact forces.



(b) Evolution of the scalar velocity, orientation, angular velocity, and angular acceleration of the unicycle base, together with the corresponding MPC predictions.

Figure 4.21. MPC state and input predictions during the execution of the maximum–height jump maneuver.

the MPC predictions exhibit a behavior consistent with the expected flight dynamics, as shown in Figures 4.21a and 4.21b. The tracking performance of the WBC remains accurate also in this experiment, as shown in Figure 4.22.

The CoM tracking error with respect to the nominal cruise trajectory is reported in Figure 4.23, while the evolution of the platform orientation is shown in Figure 4.24. As in the previous jump experiments, the largest deviations from the nominal reference occur in correspondence with the jump maneuver, especially near takeoff and landing. Nevertheless, once the obstacle has been

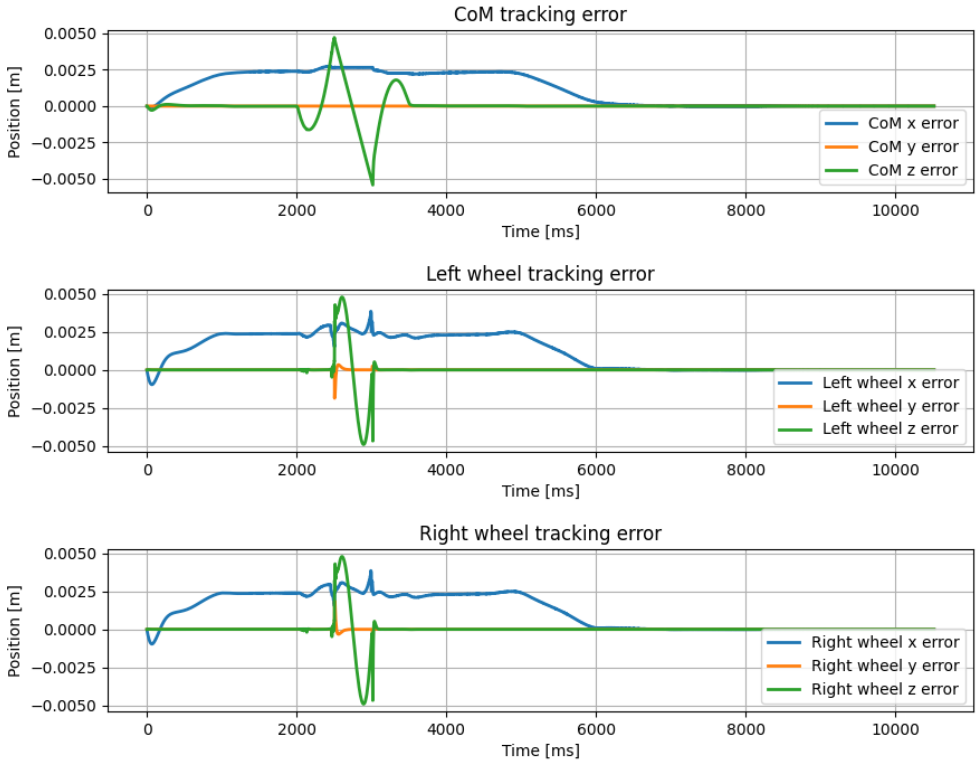


Figure 4.22. WBC tracking error with respect to the MPC desired reference trajectories during the maximum–height jump maneuver.

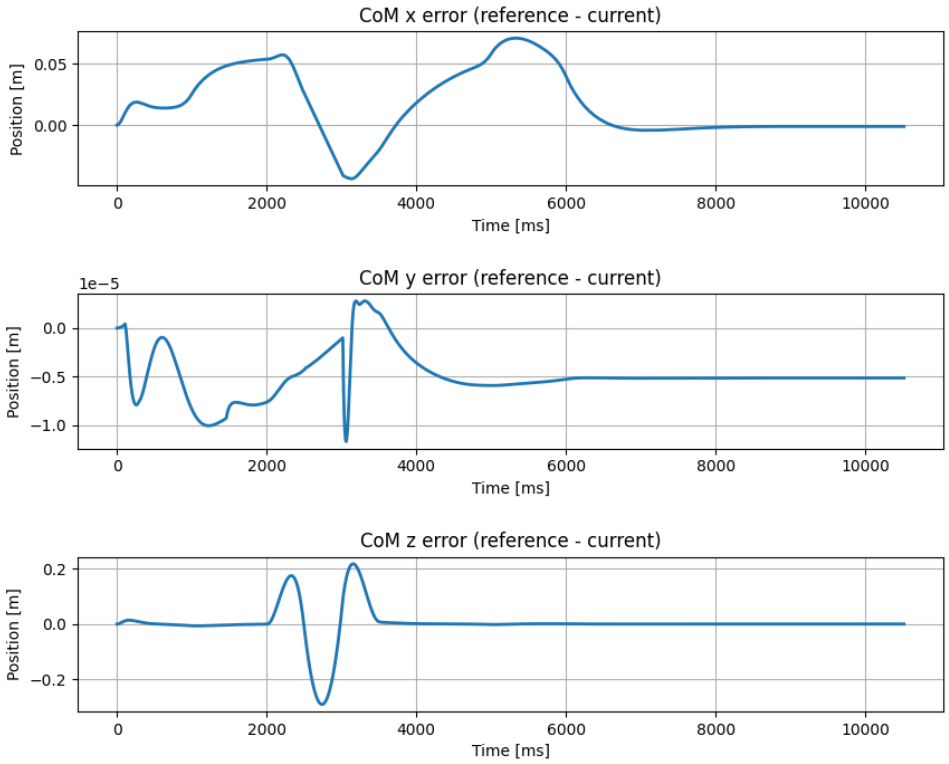


Figure 4.23. Tracking error of the CoM with respect to the generated cruise reference trajectory during the maximum–height jump maneuver.

avoided, these deviations are effectively reduced and the robot returns to the nominal motion.

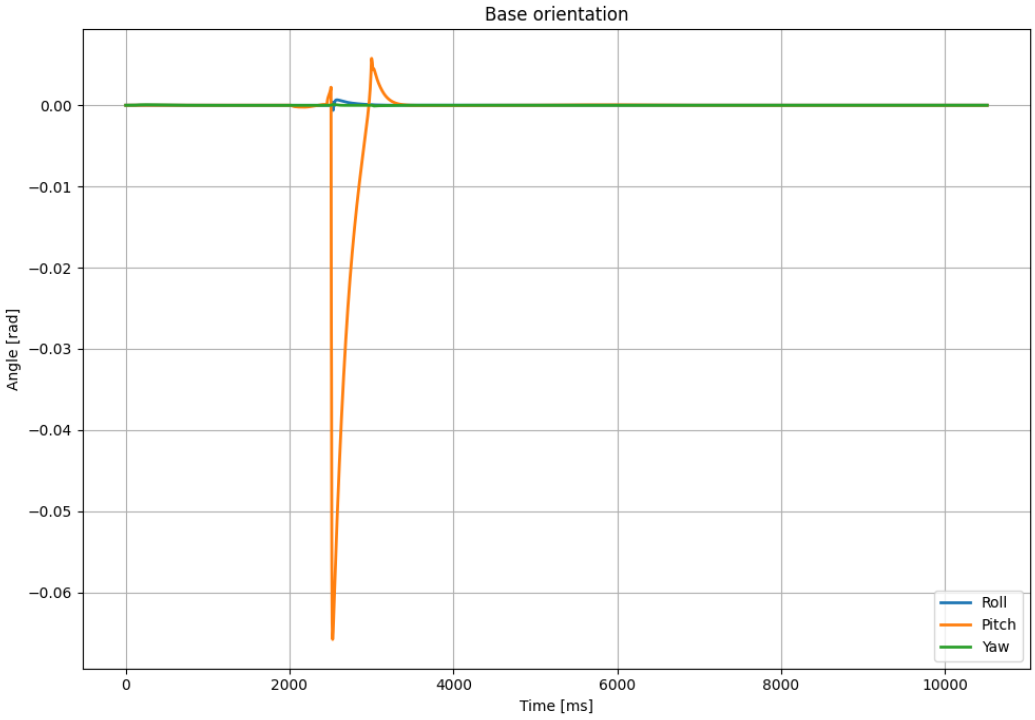


Figure 4.24. Evolution of the platform orientation, expressed in roll–pitch–yaw Euler angles, during the maximum-height jump maneuver.

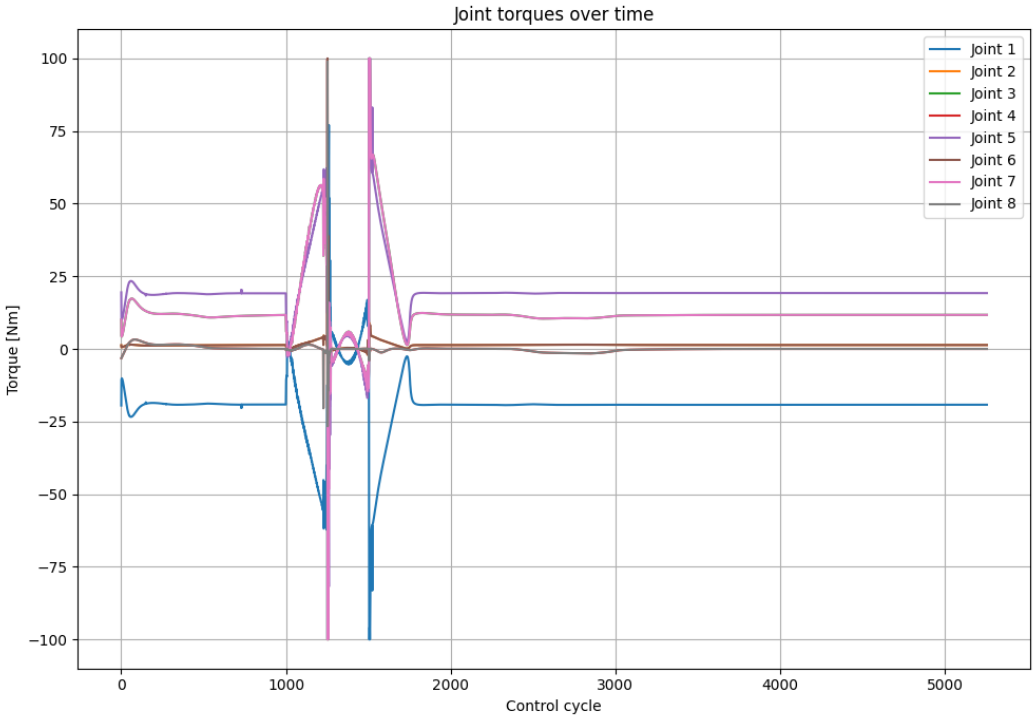


Figure 4.25. Joint torque profiles during the execution of the maximum–height jump maneuver. The largest torque peaks occur during the *takeoff* and *landing* phases.

Finally, the control effort remains contained and within the actuator limits even in this demanding condition, as shown in Figure 4.25.

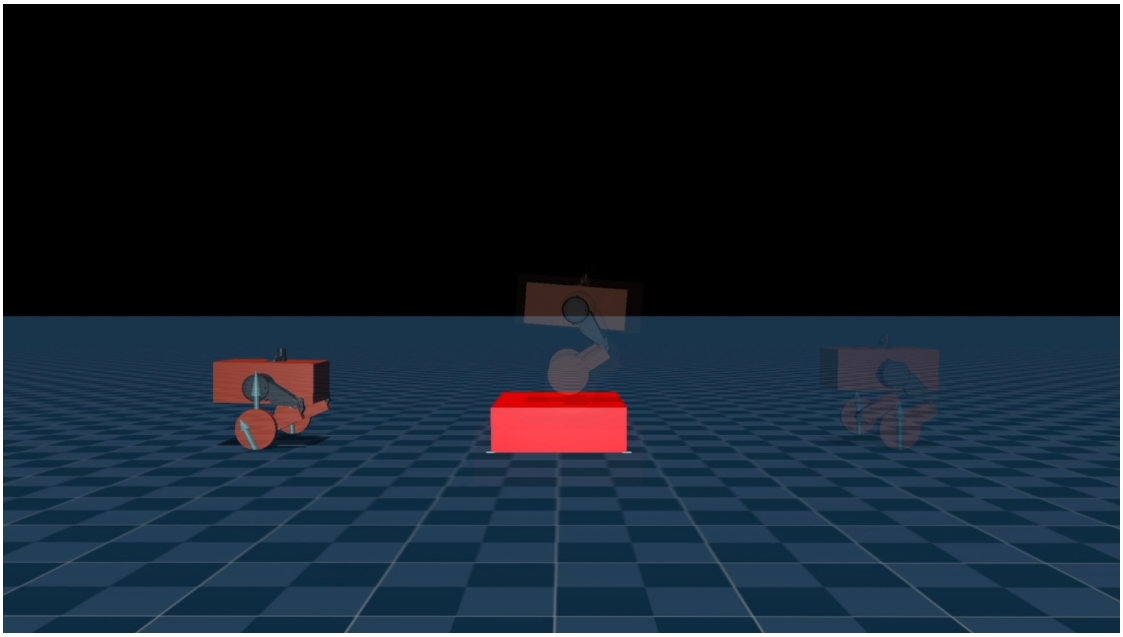


Figure 4.26. Illustration of the high–speed jump maneuver executed to avoid the obstacle.

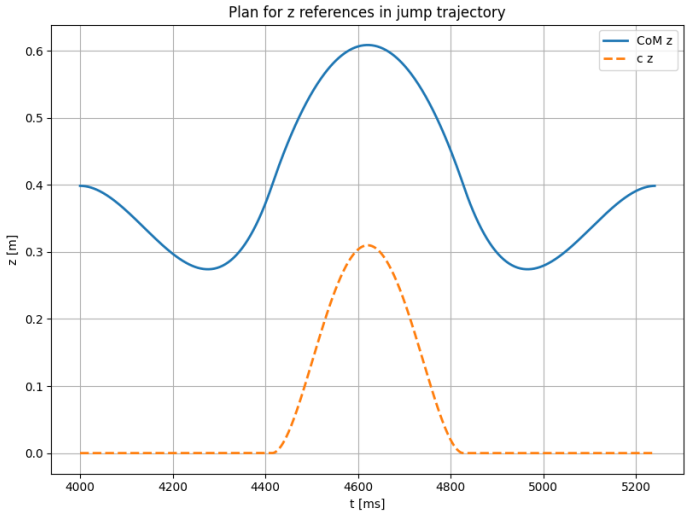
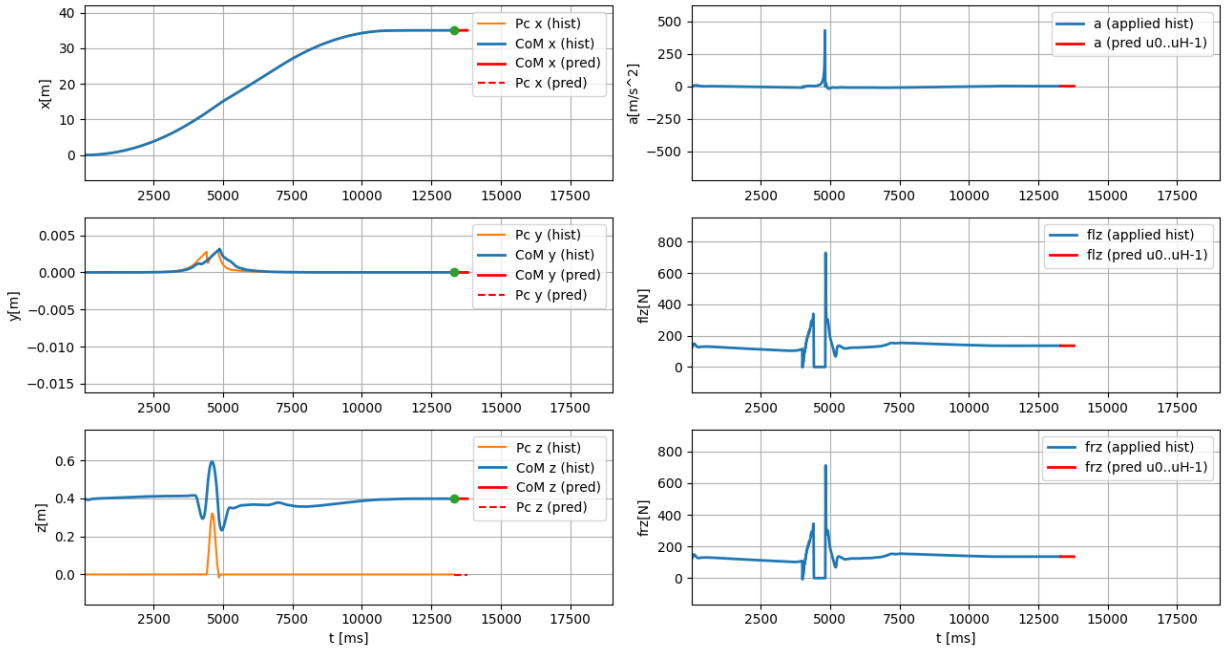


Figure 4.27. Vertical reference jump trajectory for the high–speed obstacle avoidance experiment. The maneuver is designed with a maximum CoM vertical displacement of 21 cm and a maximum leg vertical excursion of 31 cm.

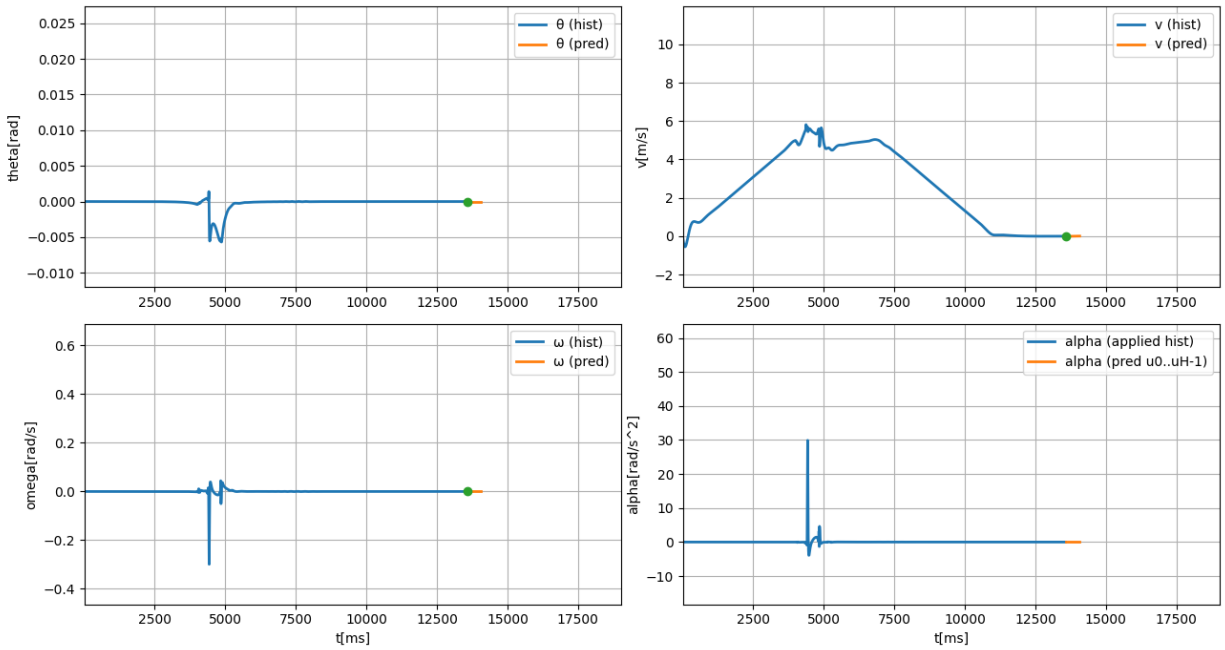
4.1.5 High–speed jump

This experiment is designed to assess the performance limits of the proposed control framework in a highly dynamic scenario. In particular, the robot is required to track a high–speed reference trajectory with a peak forward velocity of 5 m/s. While tracking this trajectory, the robot is required to avoid a suddenly appearing obstacle of dimensions $60 \times 80 \times 20$ cm^3, located on the path in correspondence when the robot reaches the peak forward velocity. This configuration represents a particularly demanding condition, since the obstacle must be overcome while the platform is moving at high speed and with limited time available to reconfigure the motion. For completeness, an illustration of the jump maneuver is reported in Figure 4.26.

The jump trajectory planned to overcome the obstacle is reported in Figure 4.27. In order to guarantee a safe clearance, the maneuver is conservatively designed with a maximum leg vertical excursion of 31 cm and a maximum CoM vertical displacement of 21 cm.



(a) Evolution of the CoM and unicycle–base states together with the corresponding MPC predictions. The right column also reports the base accelerations and the vertical components of the contact forces.



(b) Evolution of the scalar velocity, orientation, angular velocity, and angular acceleration of the unicycle base, together with the corresponding MPC predictions.

Figure 4.28. MPC state and input predictions during the execution of the high–speed jump maneuver.

The MPC predictions reported in Figures 4.28a and 4.28b confirm that the predictive controller is able to generate dynamically consistent trajectories also in this aggressive operating condition. This result is particularly relevant, since at high forward velocity the controller must simultaneously maintain longitudinal motion, regulate the vertical dynamics required for the jump, and preserve platform stability throughout the maneuver.

The tracking performance of the WBC remains accurate also in this experiment, as shown in Figure 4.29. Despite the aggressive nature of the maneuver, the whole–body controller is able to

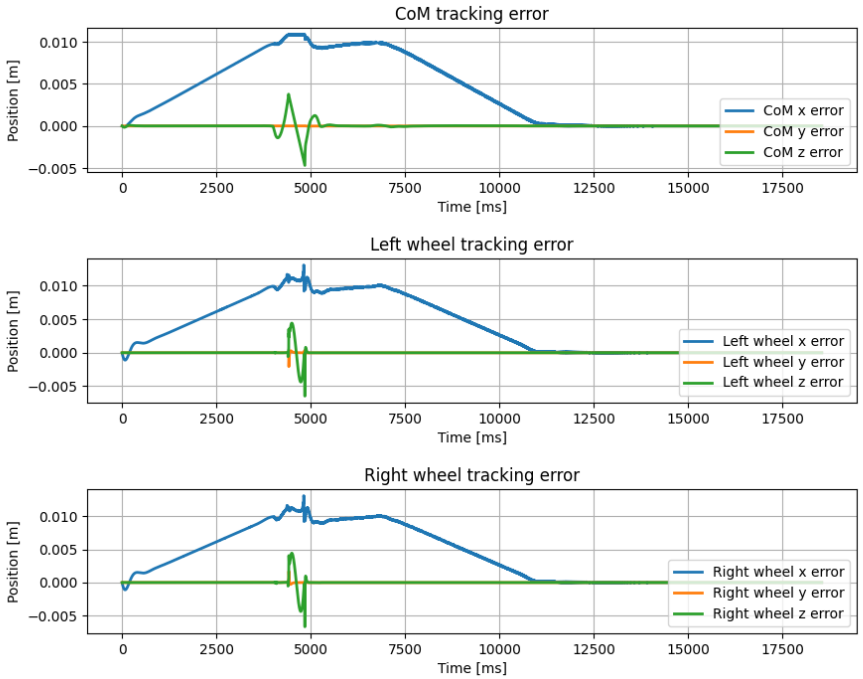


Figure 4.29. WBC tracking error with respect to the cruise reference trajectories generated from the MPC during the high–speed jump experiment.

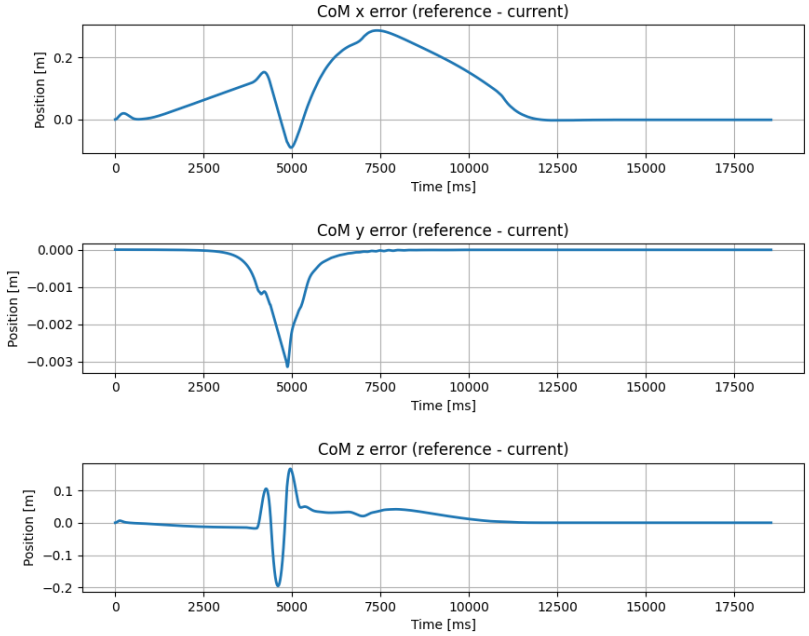


Figure 4.30. Tracking error of the CoM with respect to the nominal cruise reference trajectory during the high–speed jump experiment.

closely track the trajectories generated by the MPC.

The CoM tracking error with respect to the nominal cruise trajectory is reported in Figure 4.30, while the evolution of the platform orientation is shown in Figure 4.31. As in the previous jump experiments, the largest deviations from the nominal reference occur in correspondence with the jump maneuver, especially during the takeoff and landing phases. Nevertheless, once the maneuver is completed, the robot progressively reduces the tracking error and returns to the nominal motion, thus confirming the ability of the controller to restore a stable configuration after the maneuver.

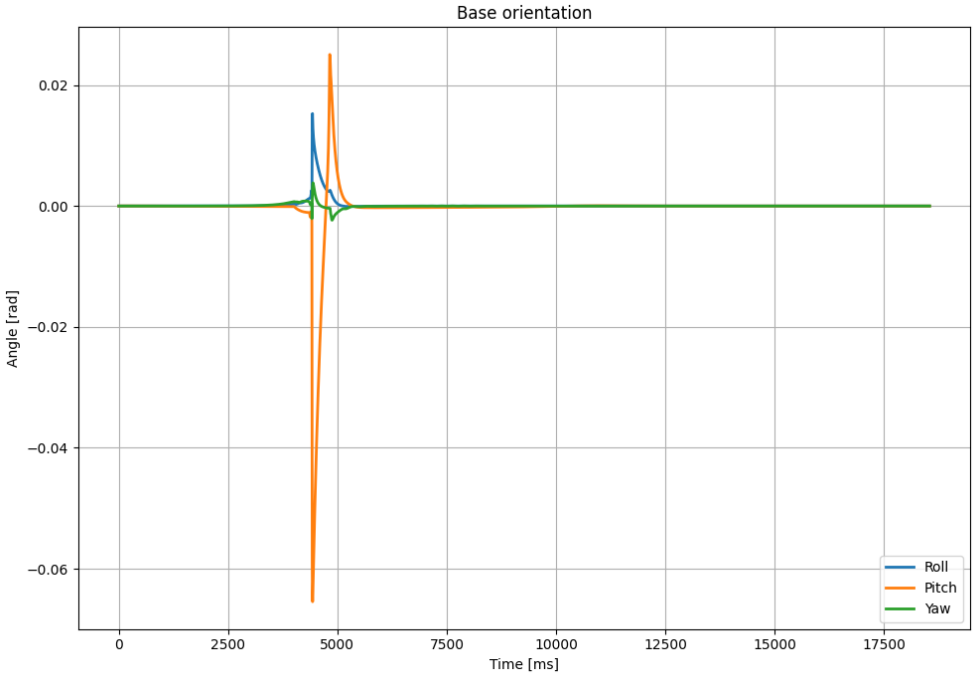


Figure 4.31. Evolution of the platform orientation, expressed in roll–pitch–yaw Euler angles, during the high–speed jump experiment.

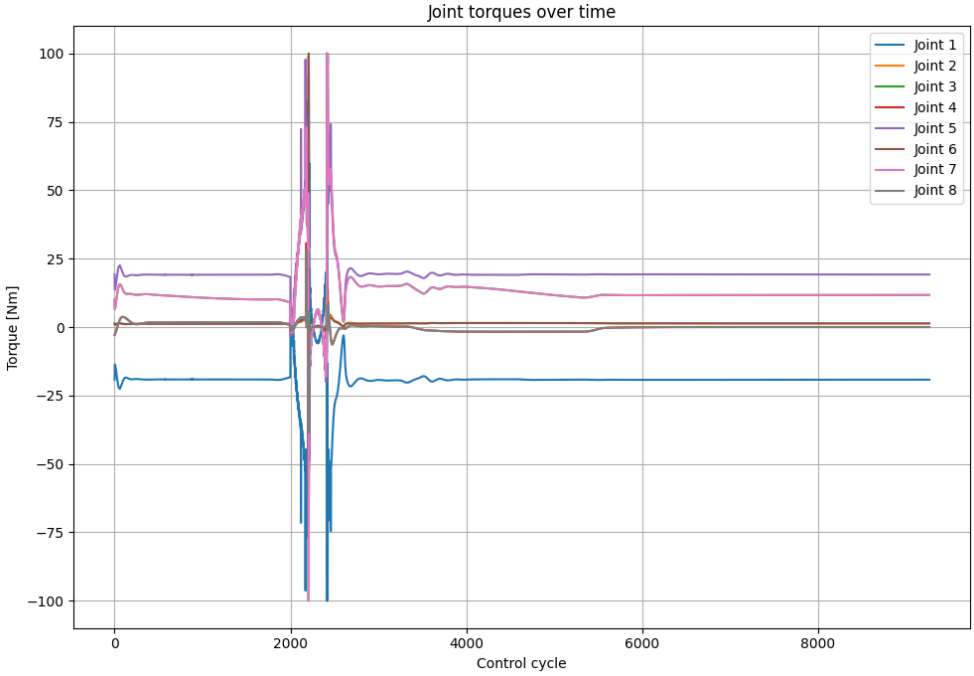


Figure 4.32. Joint torque profiles in the high–speed jump experiment. The largest torque peaks are observed during the *takeoff* and *landing* phases, while all torques remain within the actuator limits.

The control effort required by the maneuver is reported in Figure 4.32. As expected, the highest torque peaks occur during the takeoff and landing phases, when the robot must generate the vertical impulse needed to clear the obstacle and then absorb the impact at touchdown. Nevertheless, all joint torques remain within the admissible actuator limits, showing that the maneuver is dynamically demanding but still feasible for the platform.

Finally, the timing results reported in Figure 4.33 show that the entire control pipeline remains

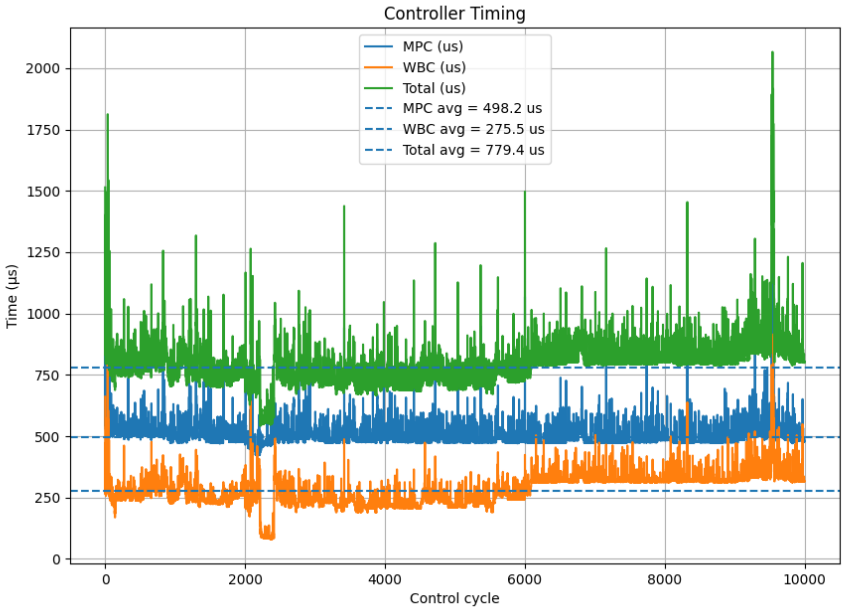


Figure 4.33. Computation times of the control framework n the high–speed jump experiment. All computations remain within the available control period, confirming the real–time feasibility of the proposed approach.

within the available computational budget also in this aggressive scenario. This confirms that the proposed framework preserves real–time feasibility not only during nominal locomotion, but also during highly dynamic maneuvers.

Appendix A

Differential Dynamic Programming

Differential Dynamic Programming (DDP), originally proposed in [23], is a trajectory optimization method for nonlinear optimal control problems. It is based on Bellman's principle of optimality and is implemented numerically by constructing local approximations of both the system dynamics and the cost function around a nominal trajectory. The method iteratively improves the control policy through two main steps: a backward pass, in which a local approximation of the optimal policy is derived, and a forward pass, in which a new trajectory is generated and evaluated.

More specifically, DDP decomposes the original finite–horizon optimal control problem into a sequence of local subproblems, one for each time step, which are solved recursively backward in time. Its main idea is that the value function can be approximated locally by a quadratic expression around the current trajectory, and that this approximation can be propagated analytically from the final time step to the initial one. By repeatedly updating the trajectory using these local quadratic approximations, the algorithm converges to a locally optimal solution.

Because of its efficiency and its suitability for nonlinear systems, DDP has become an important tool in optimal control. At the same time, it also presents some practical limitations, particularly in the presence of constraints or poor initial guesses, which have motivated the development of several extensions. For this reason, this appendix briefly reviews the main principles of the method and then discusses its main advantages, limitations, and some relevant extensions proposed in the literature.

A.1 Optimal control problem

Here is recalled the trajectory optimization problem already described in Section 2.1.1.

Consider the discrete–time dynamical system

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \tag{A.1}$$

where $\mathbf{x}_k \in \mathbb{R}^n$ is the state of the system at time step k, $\mathbf{u}_k \in \mathbb{R}^m$ is the control input at time step k, and f is the nonlinear state transition function.

The cost associated with a state sequence $\mathbf{X} = \{\mathbf{x}_0, \dots, \mathbf{x}_N\}$ and a control sequence $\mathbf{U} = \{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ is defined as

$$J(\mathbf{X}, \mathbf{U}) = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N), \tag{A.2}$$

where $\ell(\mathbf{x}, \mathbf{u}) : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$ is the running cost and $\ell_f : \mathbb{R}^n \to \mathbb{R}$ is the terminal cost.

Given the initial state $\mathbf{x}_0$ and a horizon N, the goal is to determine the control sequence $\mathbf{U}$ that minimizes (A.2) subject to the dynamics (A.1). Since the problem has a sequential structure, it can be formulated through dynamic programming. The optimal value function at time step k is

defined as the minimum cost–to–go starting from a given state $\mathbf{x}$:

$$V_k(\mathbf{x}) = \min_{\mathbf{U}} \left(\sum_{j=k}^{N-1} \ell_k(\mathbf{x}_j, \mathbf{u}_j) + \ell_f(\mathbf{x}_N) \right). \tag{A.3}$$

By Bellman's principle of optimality, the value function satisfies the recursion

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \left(\ell_k(\mathbf{x}, \mathbf{u}) + V_{k+1}(f(\mathbf{x}, \mathbf{u})) \right), \tag{A.4}$$

with terminal condition

$$V_N(\mathbf{x}) = \ell_f(\mathbf{x}). \tag{A.5}$$

DDP seeks a locally optimal trajectory by exploiting a quadratic approximation of the value function around a nominal trajectory. Accordingly, both the dynamics and the cost are locally expanded about the nominal state–control sequence $\{\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k\}$. In this setting, it is convenient to define the state and control perturbations as

$$\delta\mathbf{x}_k = \mathbf{x}_k - \bar{\mathbf{x}}_k, \qquad \delta\mathbf{u}_k = \mathbf{u}_k - \bar{\mathbf{u}}_k. \tag{A.6}$$

Using this notation, the Bellman equation can be written locally as

$$V_k(\delta\mathbf{x}_k) = \min_{\delta\mathbf{u}_k} \left[\ell_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k) + V_{k+1}(f_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k)) \right]. \tag{A.7}$$

It is then useful to define the action–value function Q_k, which represents the local cost–to–go associated with applying a control perturbation $\delta\mathbf{u}_k$ from a state perturbation $\delta\mathbf{x}_k$:

$$Q_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k) = \ell_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k) + V_{k+1}(f(\delta\mathbf{x}_k, \delta\mathbf{u}_k)). \tag{A.8}$$

With this definition, the local optimal value function becomes

$$V_k(\delta\mathbf{x}_k) = \min_{\delta\mathbf{u}_k} Q_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k). \tag{A.9}$$

A.2 Backward pass

The backward pass computes a local approximation of the optimal control law around the nominal trajectory. To do so, the function Q_k is expanded to second order with respect to $\delta\mathbf{x}_k$ and $\delta\mathbf{u}_k$:

$$\begin{aligned} Q_k(\delta\mathbf{x}_k, \delta\mathbf{u}_k) \approx\ & Q_k^0 + Q_k^{x\top}\delta\mathbf{x}_k + Q_k^{u\top}\delta\mathbf{u}_k \\ & + \frac{1}{2}\delta\mathbf{x}_k^\top Q_k^{xx}\delta\mathbf{x}_k + \delta\mathbf{u}_k^\top Q_k^{ux}\delta\mathbf{x}_k + \frac{1}{2}\delta\mathbf{u}_k^\top Q_k^{uu}\delta\mathbf{u}_k. \end{aligned} \tag{A.10}$$

All derivatives are evaluated along the nominal trajectory.

The first– and second–order derivatives of Q_k are obtained by combining the derivatives of the running cost and of the dynamics with the derivatives of the value function at the next time step:

$$\begin{aligned} Q_k^x &= \ell_k^x + f_k^{x\top} V_{k+1}^x, \\ Q_k^u &= \ell_k^u + f_k^{u\top} V_{k+1}^x, \\ Q_k^{xx} &= \ell_k^{xx} + f_k^{x\top} V_{k+1}^{xx} f_k^x + {V_{k+1}^x}^\top f_k^{xx}, \\ Q_k^{uu} &= \ell_k^{uu} + f_k^{u\top} V_{k+1}^{xx} f_k^u + {V_{k+1}^x}^\top f_k^{uu}, \\ Q_k^{ux} &= \ell_k^{ux} + f_k^{u\top} V_{k+1}^{xx} f_k^x + {V_{k+1}^x}^\top f_k^{ux}. \end{aligned} \tag{A.11}$$

The previous expressions show that the DDP recursion requires a local approximation of the cost and dynamics. In particular, the stage cost is approximated by a second-order Taylor expansion:

$$\begin{aligned} \ell(x_k, u_k) \approx\ & \ell_k + \ell_x^\top \delta x_k + \ell_u^\top \delta u_k \\ & + \frac{1}{2}\delta x_k^\top \ell_{xx}\delta x_k + \delta u_k^\top \ell_{ux}\delta x_k + \frac{1}{2}\delta u_k^\top \ell_{uu}\delta u_k, \end{aligned} \tag{A.12}$$

where all derivatives are evaluated at $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$. The nonlinear dynamics are expanded around $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$ as

$$\delta \mathbf{x}_{k+1} \approx f_x \delta \mathbf{x}_k + f_u \delta \mathbf{u}_k + \frac{1}{2} \begin{bmatrix} \delta \mathbf{x}_k \\ \delta \mathbf{u}_k \end{bmatrix}^\top \begin{bmatrix} f_{xx} & f_{xu} \\ f_{ux} & f_{uu} \end{bmatrix} \begin{bmatrix} \delta \mathbf{x}_k \\ \delta \mathbf{u}_k \end{bmatrix}, \tag{A.13}$$

where f_x, f_u, f_{xx}, f_{xu}, f_{ux}, and f_{uu} denote the first– and second–order derivatives of the dynamics evaluated along the nominal trajectory. If the second–order derivatives of the dynamics are neglected, the resulting approximation corresponds to iLQR.

Once the quadratic approximation of the Q_k function (A.10) has been constructed, the optimal perturbation is obtained by minimizing Q_k with respect to $\delta \mathbf{u}_k$. Setting the gradient to zero yields

$$\frac{\partial Q_k}{\partial \delta \mathbf{u}_k} = Q_k^u + Q_k^{ux} \delta \mathbf{x}_k + Q_k^{uu} \delta \mathbf{u}_k = 0. \tag{A.14}$$

Assuming that Q_k^{uu} is invertible, the minimizing control perturbation can be written as

$$\delta \mathbf{u}_k^\star = \mathbf{k}_k + K_k \delta \mathbf{x}_k, \tag{A.15}$$

where

$$\mathbf{k}_k = -\left(Q_k^{uu}\right)^{-1} Q_k^u, \tag{A.16}$$

$$K_k = -\left(Q_k^{uu}\right)^{-1} Q_k^{ux}. \tag{A.17}$$

The term $\mathbf{k}_k$ is the feedforward correction, whereas K_k is the feedback gain matrix. Therefore, the backward pass does not only compute an improved open–loop control sequence, but also a local state–feedback law around the nominal trajectory.

Substituting (A.15) into the quadratic approximation of Q_k, is it possible to obtain a recursive update for the derivatives of the value function as

$$V_k^x = Q_k^x - Q_k^{xu} Q_k^{uu-1} Q_k^u, \tag{A.18}$$

$$V_k^{xx} = Q_k^{xx} - Q_k^{xu} Q_k^{uu-1} Q_k^{ux}. \tag{A.19}$$

The updated expressions for V_k^x and V_k^{xx} allow the value function approximation to be propagated recursively from one time step to the previous one. Hence, the backward pass is carried out from $k = N - 1$ down to $k = 0$. The recursion is initialized at the terminal time by the terminal cost.

$$V_N(\mathbf{x}_N) = \ell_f(\mathbf{x}_N), \tag{A.20}$$

$$V_x(N) = \ell_f^x, \qquad V_{xx}(N) = \ell_f^{xx}. \tag{A.21}$$

A.3 Forward pass

After the backward pass has produced the feedforward and feedback terms $\{\mathbf{k}_k, K_k\}$, a forward rollout is performed to generate a new candidate trajectory. Starting from the initial state $\mathbf{x}_0$, the control is updated according to

$$\mathbf{u}_k = \bar{\mathbf{u}}_k + \alpha \mathbf{k}_k + K_k(\mathbf{x}_k - \bar{\mathbf{x}}_k), \qquad \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \tag{A.22}$$

where $\alpha \in (0, 1]$ is a step size selected through line search. This step is necessary because the quadratic approximation used in the backward pass is only accurate in a neighborhood of the nominal trajectory.

The line search is used to guarantee a decrease in cost when the full Newton–like step is too aggressive. Instead of applying the feedforward correction k_k with full magnitude, DDP scales it by a coefficient α. Typical values of α are chosen from a decreasing sequence in the interval $(0, 1]$. For

Algorithm 1 Discrete-time Differential Dynamic Programming (DDP)

Require: Dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, running cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$, terminal cost $l_f(\mathbf{x}_N)$
Require: Initial state $\mathbf{x}_0$, nominal control sequence $\{\bar{\mathbf{u}}_0, \bar{\mathbf{u}}_1, \ldots, \bar{\mathbf{u}}_{N-1}\}$
Ensure: Locally optimal state and control trajectories
Roll out the nominal dynamics to obtain $\{\bar{\mathbf{x}}_0, \bar{\mathbf{x}}_1, \ldots, \bar{\mathbf{x}}_N\}$
while not converged **do**
    Backward pass
    $V_N^x = l_f^x(\bar{\mathbf{x}}_N)$
    $V_N^{xx} = l_f^{xx}(\bar{\mathbf{x}}_N)$
    for $k = N-1, \ldots, 0$ **do**
        Compute the local derivatives of f and ℓ along $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$
        $Q_k^x = \ell_k^x + (f_k^x)^\top V_{k+1}^x$
        $Q_k^u = \ell_k^u + (f_k^u)^\top V_{k+1}^x$
        $Q_k^{xx} = \ell_k^{xx} + (f_k^x)^\top V_{k+1}^{xx} f_k^x + V_{k+1}^x f_k^{xx}$
        $Q_k^{uu} = \ell_k^{uu} + (f_k^u)^\top V_{k+1}^{xx} f_k^u + V_{k+1}^x f_k^{uu}$
        $Q_k^{ux} = \ell_k^{ux} + (f_k^u)^\top V_{k+1}^{xx} f_k^x + V_{k+1}^x f_k^{ux}$
        $\mathbf{k}_k = -(Q_k^{uu})^{-1} Q_k^u$
        $K_k = -(Q_k^{uu})^{-1} Q_k^{ux}$
        $V_k^x = Q_k^x - (Q_k^{ux})^\top (Q_k^{uu})^{-1} Q_k^u$
        $V_k^{xx} = Q_k^{xx} - (Q_k^{ux})^\top (Q_k^{uu})^{-1} Q_k^{ux}$
    end for
    Forward pass
    Choose step size $\alpha \in (0, 1]$
    $\mathbf{x}_0^{\text{new}} = \mathbf{x}_0$
    for $k = 0, \ldots, N-1$ **do**
        $\mathbf{u}_k^{\text{new}} = \bar{\mathbf{u}}_k + \alpha \mathbf{k}_k + K_k(\mathbf{x}_k^{\text{new}} - \bar{\mathbf{x}}_k)$
        $\mathbf{x}_{k+1}^{\text{new}} = f(\mathbf{x}_k^{\text{new}}, \mathbf{u}_k^{\text{new}})$
    end for
    Evaluate the new cost J^{new}
    if $J^{\text{new}} < J$ **then**
        Accept the new trajectory:
        $\bar{\mathbf{x}}_k \leftarrow \mathbf{x}_k^{\text{new}}, \quad \bar{\mathbf{u}}_k \leftarrow \mathbf{u}_k^{\text{new}}$
    else
        Reduce α or increase regularization
    end if
end while

each candidate value, the trajectory is rolled out forward and the corresponding cost is evaluated. The first value that provides a sufficient reduction in the cost is accepted.

This mechanism is analogous to line search in Newton–type optimization methods. Since the quadratic local approximation may lose accuracy far from the nominal trajectory, reducing the step size can prevent divergence and improve numerical stability. For this reason, line search is an essential component of DDP in practical nonlinear problems.

For clarity, the overall DDP procedure is summarized in Algorithm 1.

In summary, DDP can be interpreted as a second–order dynamic programming method for nonlinear trajectory optimization. Unlike direct approaches, which typically solve a large nonlinear program over the entire trajectory, DDP exploits the sequential structure of the system and propagates local optimality information backward in time.

From a control viewpoint, the result of the algorithm is not limited to an improved nominal input sequence. The feedforward term $\mathbf{k}_k$ corrects the open–loop command, while the feedback gain K_k stabilizes deviations from the nominal trajectory during the forward rollout. As a consequence,

DDP provides both a locally improved trajectory and a time–varying local feedback policy.

A.4 Extensions

Several extensions of DDP have been proposed to overcome some of the limitations of the classical unconstrained single–shooting formulation. In particular, modern implementations such as *Crocoddyl* [26] extend DDP to multi–contact optimal control problems by exploiting sparse analytical derivatives and system geometry. Within this framework, the authors also propose Feasibility–driven Differential Dynamic Programming (FDDP), which allows an initial guess not only for the controls but also for the state trajectory, temporarily accepting dynamic defects in a way that is conceptually similar to multiple shooting. Another important line of work concerns the treatment of constraints. For example, constrained variants of DDP explicitly incorporate state and control constraints within the backward–pass optimization, improving the applicability of the method to realistic control problems [27].

A.5 Advantages and limitations of DDP

DDP has several important advantages. Since it exploits second–order information, it typically achieves fast local convergence when initialized sufficiently close to a local optimum. It also naturally provides not only a nominal control sequence, but also a local feedback policy, which is particularly valuable in optimal control applications where robustness to small perturbations is important.

Another major strength is that the computations are propagated through forward and backward passes along the horizon, so the computational cost scales linearly with the horizon length: if the horizon is doubled, the amount of computation is approximately doubled as well. This is an important advantage compared to methods such as SQP, whose complexity depends on the QP subproblem and on the solver used, and is therefore generally less predictable. It is also well suited to nonlinear systems and nonlinear cost functions, and can produce high–quality trajectories in relatively few iterations when the local approximations are accurate.

At the same time, DDP also has several limitations. First, it is fundamentally a local optimization method, so its performance depends strongly on the initial trajectory and it may converge to a poor local minimum. In addition, as a single–shooting method, DDP integrates the dynamics starting from the initial state and the initial control guess; if this guess is poor, the resulting trajectory may diverge significantly, which can make the optimization difficult. The method also requires first and second derivatives of the cost and, in full DDP, also second derivatives of the dynamics, which may be difficult to derive and computationally expensive to evaluate. From a numerical point of view, DDP can be sensitive to ill–conditioning, especially when Q_{uu} is nearly singular or indefinite, so regularization and line search strategies are often necessary.

Another important drawback is that standard DDP does not directly handle hard state and input constraints, whereas in methods such as SQP the inclusion of constraints is more straightforward. Although constrained variants of DDP exist, they increase the algorithmic complexity.

Appendix B

Wheel–ground contact constraint

This Appendix presents the mathematical derivation of the wheel–ground contact constraint adopted in this work for a non–steerable fixed wheel. As a preliminary step, we first analyse the spherical case, which provides a convenient simplification thanks to the sphere's rotational symmetry: the rolling without slipping condition applies in all directions: a compatible angular velocity can be chosen that produce any desired center velocity. In this sense the pure rolling constraint is isotropic for the sphere. We then extend the same reasoning to the wheel case, where the pure rolling holds only along a privileged direction that lies in the wheel plane, in particular, the rolling along the lateral direction is prohibited, a restriction not present in the spherical case. In addition the interaction between the wheel and the ground introduces an additional condition: the wheel plane may tilt while preserving the contact point. The derivation follows the treatment in [17].

B.1 Pure rolling of a sphere

Consider a rigid sphere rolling on a planar ground under pure rolling conditions.

For a generic point P fixed to the rigid body, rigid body kinematics yields

$$\mathbf{v}_P = \mathbf{v}_w + \boldsymbol{\omega} \times \mathbf{r}_{wP}, \tag{B.1}$$

where $\mathbf{v}_w$ is the linear velocity of the sphere center, $\boldsymbol{\omega}$ is the angular velocity of the sphere, and $\mathbf{r}_{wP}$ is the position vector from the sphere center to point P.

Let C denote the material point of the sphere that is in contact with the ground at time t. Pure rolling imposes that the velocity of the contact point vanishes:

$$\mathbf{v}_{C|t} = \mathbf{v}_{w|t} + \boldsymbol{\omega}_{|t} \times \mathbf{r}_{wC} = \mathbf{0}. \tag{B.2}$$

Equivalently,

$$\mathbf{v}_{w|t} = -\boldsymbol{\omega}_{|t} \times \mathbf{r}_{wC}. \tag{B.3}$$

Now consider two consecutive instants t and $t + \delta t$, and denote by C and C' the sphere material points in contact with the ground at these instants, respectively. Figure B.1 illustrates this scenario.

The difference between the sphere center velocity at these two time steps reads

$$\mathbf{v}_{w|t+\delta t} - \mathbf{v}_{w|t} = -\boldsymbol{\omega}_{|t+\delta t} \times \mathbf{r}_{wC'} + \boldsymbol{\omega}_{|t} \times \mathbf{r}_{wC} \tag{B.4}$$

Taking the limit as $\delta t \to 0$ yields the acceleration of the sphere center

$$\mathbf{a}_w = \lim_{\delta t \to 0} \frac{\mathbf{v}_{w|t+\delta t} - \mathbf{v}_{w|t}}{\delta t} = \frac{d}{dt}(-\boldsymbol{\omega} \times \mathbf{r}_{\mathrm{SPV}}), \tag{B.5}$$



(a) Pure rolling motion at time t.

(b) Pure rolling motion at time $t + \delta t$.

Figure B.1. Schematic of pure rolling for a sphere: contact point changes from C to C'.

where $\mathbf{r}_{\text{SPV}} \in \mathbb{R}^3$ denotes the *Support Polygon Vertex* (SPV), i.e., a vector that describes a set of consecutive positions of the contact with the ground with respect to the sphere center. In particular, for the contact at time t,

$$\mathbf{r}_{\text{SPV}} \triangleq \mathbf{r}_{wC}. \tag{B.6}$$

A crucial subtle point is that $\dot{\mathbf{r}}_{\text{SPV}}$ does not coincide in general with the time derivative of $\mathbf{r}_{wC}$ for a fixed material point, because C and C' can be distinct material points on the sphere surface. Consequently,

$$\dot{\mathbf{r}}_{\text{SPV}} = \lim_{\delta t \to 0} \frac{\mathbf{r}_{wC'} - \mathbf{r}_{wC}}{\delta t} \neq \dot{\mathbf{r}}_{wC}. \tag{B.7}$$

Indeed, for a point C rigidly attached to the body, the derivative of $\mathbf{r}_{wC}$ is given by the Poisson formula,

$$\dot{\mathbf{r}}_{wC} = \boldsymbol{\omega} \times \mathbf{r}_{wC}, \tag{B.8}$$

Note that (B.5) is obtained directly from the constraint in (B.2); therefore, it remains valid in all situations where this constraint applies, including, for instance, both wheel rolling and tilting motions.

For a sphere subject to pure rolling motion on a plane, the vector from the center to the contact point always has constant magnitude equals to the sphere radius and constant direction in the inertial frame (it points along the ground normal). Therefore, $\mathbf{r}_{\text{SPV}}$ is constant and

$$\dot{\mathbf{r}}_{\text{SPV}}(t) = \mathbf{0} \qquad \forall t. \tag{B.9}$$

Substituting this property into (B.5) yields

$$\mathbf{a}_w = -\dot{\boldsymbol{\omega}} \times \mathbf{r}_{wC}. \tag{B.10}$$

Finally, it is worth stressing that under pure rolling the acceleration of the material point in contact with the ground is not zero. For the contact point C at time t, rigid body kinematics gives

$$\mathbf{a}_C = \mathbf{a}_w + \dot{\boldsymbol{\omega}} \times \mathbf{r}_{wC} + \boldsymbol{\omega} \times \boldsymbol{\omega} \times \mathbf{r}_{wC}. \tag{B.11}$$

B.2 Rolling without slipping for a fixed wheel

For a fixed wheel in contact with the ground, the *pure rolling* assumption restricts the admissible motion by introducing a local mobility limitation. In addition to spinning about its axle, the wheel may also undergo rotations (e.g., steering or tilting) that reorient the wheel plane while maintaining contact. To derive a general contact kinematic constraint at the acceleration level, both contributions must be incorporated. Before presenting the complete constraint that combines these effects, we discuss the two cases separately.

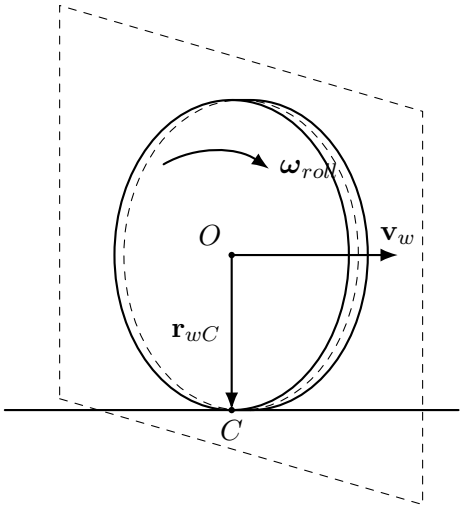


Figure B.2. Kinematic sketch of a wheel in pure rolling: the wheel center O moves with velocity $\mathbf{v}_w$, the wheel rotates with angular velocity $\boldsymbol{\omega}_{roll}$, and $\mathbf{r}_{wC}$ denotes the vector from O to the contact point C. The wheel plane is shown for reference.

First, we consider the wheel undergoing pure rolling only, e.g., the wheel rigid body is subject to a rolling angular velocity $\boldsymbol{\omega}_{roll}$ expressed in the inertial reference frame as illustrated in Figure B.2. In this scenario, no steering or tilting occurs during rolling.

$$\mathbf{v}_w = -\boldsymbol{\omega}_{roll} \times \mathbf{r}_{wC} \tag{B.12}$$

Since no steering or tilting occurs, the wheel plane orientation is constant over time: there is no angular velocity that would change it, in other words the SPV vector verifies

$$\dot{\mathbf{r}}_{\mathrm{SPV}}(t) = \mathbf{0} \qquad \forall t \tag{B.13}$$

Therefore, from (B.5), the wheel center acceleration satisfies

$$\mathbf{a}_w = -\dot{\boldsymbol{\omega}}_{roll} \times \mathbf{r}_{wC} \tag{B.14}$$

This situation is representative, for example, of unicycle–like robots in which the wheel plane is constrained to remain vertical with respect to the ground, e.g. $\mathbf{r}_{SPV} = \begin{pmatrix} 0 & 0 & -r_{wheel} \end{pmatrix}^T = const.$

The second illustrative case considers a wheel that undergoes tilting while preserving the same ground contact point. Specifically, an angular velocity that reorients the wheel plane, denoted by $\boldsymbol{\omega}_{\mathrm{tilt}}$ and expressed in the inertial frame, is applied to the wheel rigid body, as shown in Figure B.3.

The wheel center velocity is

$$\mathbf{v}_w = -\boldsymbol{\omega}_{tilt} \times \mathbf{r}_{wC} \tag{B.15}$$

This time, the wheel plane orientation is no longer constant: it evolves over time according to the tilting angular velocity $\boldsymbol{\omega}_{\mathrm{tilt}}$. Accordingly, one can write

$$\dot{\mathbf{r}}_{SPV} = \boldsymbol{\omega}_{tilt} \times \mathbf{r}_{wC}, \tag{B.16}$$

Hence, from (B.5), the wheel center acceleration can be expressed as

$$\mathbf{a}_w = -\dot{\boldsymbol{\omega}}_{tilt} \times \mathbf{r}_{wC} - \boldsymbol{\omega}_{tilt} \times \boldsymbol{\omega}_{tilt} \times \mathbf{r}_{wC} \tag{B.17}$$

We note that the essential difference between the two cases can be interpreted in terms of the acceleration of the material point of the wheel in contact with the ground, indeed when the wheel is subject to only $\boldsymbol{\omega}_{roll}$ the acceleration is

$$\mathbf{a}_{C|roll} = \boldsymbol{\omega}_{roll} \times \boldsymbol{\omega}_{roll} \times \mathbf{r}_{wC} \tag{B.18}$$

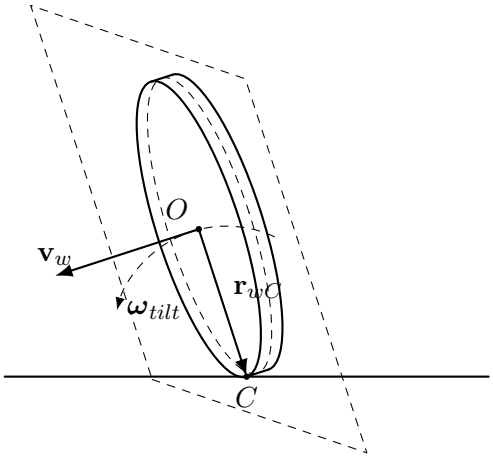


Figure B.3. Kinematic sketch of a wheel tilting: the wheel center O moves with velocity $\mathbf{v}_w$, the wheel rotates with angular velocity $\boldsymbol{\omega}_{tilt}$, and $\mathbf{r}_{wC}$ denotes the vector from O to the contact point C. The wheel plane is shown for reference.

i.e., a purely centripetal contribution associated with the wheel rolling. This corresponds to straight line rolling with no steering/tilting. In the general case (e.g., non straight motion), the contact point acceleration is $\mathbf{a}_C = \mathbf{a}_w + \dot{\boldsymbol{\omega}} \times \mathbf{r}_{wC} + \boldsymbol{\omega} \times \boldsymbol{\omega} \times \mathbf{r}_{wC}$ which is not purely centripetal.

In contrast, for the tilting case, the contact point remains fixed, hence

$$\mathbf{a}_{C|tilt} = \mathbf{a}_w + \dot{\boldsymbol{\omega}}_{tilt} \times \mathbf{r}_{wC} + \boldsymbol{\omega}_{tilt} \times \dot{\mathbf{r}}_{wC} = \mathbf{0} \tag{B.19}$$

This expresses the fact that the wheel reorient its plane through tilting without detaching and without changing the contact material point.

After introducing the two limiting cases, we now derive a constraint that captures the most general situation, namely a wheel that continues to roll while its plane is reoriented through steering and/or tilting, without losing ground contact.

Consider a wheel moving on the ground plane with a general angular velocity $\boldsymbol{\omega}_{\text{wheel}}$ expressed in the inertial frame. To formulate the kinematic constraint, it is convenient to introduce a *virtual* frame

$$R_{\text{virtual}} = \begin{bmatrix} \mathbf{t} & \mathbf{n} & \mathbf{s} \end{bmatrix}, \tag{B.20}$$

where $\mathbf{t}$ is tangent to the rolling direction, $\mathbf{n}$ is normal to the wheel plane (e.g., aligned with the wheel rolling axis), and $\mathbf{s}$ lies in the wheel plane and is chosen as the axis closest to the ground normal, see Figure B.4. By construction, this virtual frame follows the wheel reorientation but excludes the spin associated with rolling. Hence, its angular velocity is obtained by subtracting from $\boldsymbol{\omega}_{\text{wheel}}$ the component along $\mathbf{n}$

$$\boldsymbol{\omega}_{\text{virtual}} = \left(I - \mathbf{n}\mathbf{n}^T\right) \boldsymbol{\omega}_{\text{wheel}}. \tag{B.21}$$

Where $I \in \mathbb{R}^{3\times3}$ represents the identity matrix.

Due to the construction of the virtual frame the $\mathbf{r}_{SPV}$ vector is always aligned with $\mathbf{s}$, more precisely

$$\mathbf{r}_{SPV} = -||\mathbf{r}_{SPV}|| \cdot \mathbf{s} = -r_{wheel} \cdot \mathbf{s} \tag{B.22}$$

Exploiting the Poisson formula for the derivative of $\mathbf{s}$, yelds

$$\dot{\mathbf{s}} = \boldsymbol{\omega}_{virtual} \times \mathbf{s} \tag{B.23}$$

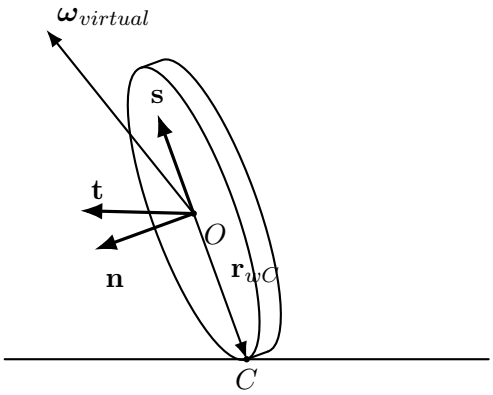


Figure B.4. Kinematic sketch of a general motion of a wheel: the wheel center O moves with velocity $\mathbf{v}_w$, the wheel rotates with angular velocity $\boldsymbol{\omega}_{wheel}$. The virtual frame is defined as follows: $\mathbf{t}$ is the unit vector tangent to the rolling direction, $\mathbf{n}$ is the unit vector orthogonal to the wheel plane (e.g., aligned with the wheel axle), and $\mathbf{s}$ lies in the wheel plane and is selected as the in-plane axis most closely aligned with the ground normal. The angular velocity of the virtual frame, $\boldsymbol{\omega}_{virtual}$ lies in the plane formed by $\mathbf{t}$ and $\mathbf{s}$.

Thus we can relate the angular velocity of the virtual frame to the derivative of $\mathbf{r}_{SPV}$ in the inertial frame through

$$\dot{\mathbf{r}}_{SPV} = \boldsymbol{\omega}_{virtual} \times \mathbf{r}_{SPV} \tag{B.24}$$

Finally, from (B.5) we can derive wheel–ground contact kinematic constraint

$$\mathbf{a}_w = -\dot{\boldsymbol{\omega}}_{\text{wheel}} \times \mathbf{r}_{wC} - \boldsymbol{\omega}_{\text{wheel}} \times \boldsymbol{\omega}_{\text{virtual}} \times \mathbf{r}_{wC} \tag{B.25}$$

Bibliography

[1] X. Liu, Y. Sun, S. Wen, K. Cao, Q. Qi, X. Zhang, H. Shen, G. Chen, J. Xu, and A. Ji, "Development of wheel-legged biped robots: A review," *Journal of Bionic Engineering*, vol. 21, no. 2, pp. 607–634, 2024. Published online 27 February 2024.

[2] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *Proceedings of the 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.

[3] Google DeepMind, "Mujoco: Advanced physics simulation." `https://mujoco.org/`.

[4] Google DeepMind, "Mujoco documentation." `https://mujoco.readthedocs.io/`.

[5] S. Kajita and K. Tani, "Study of dynamic biped locomotion on rugged terrain: Derivation and application of the linear inverted pendulum mode," in *Proceedings of the 1991 IEEE International Conference on Robotics and Automation*, vol. 2, pp. 1405–1411, IEEE, Apr. 1991.

[6] S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Harada, K. Yokoi, and H. Hirukawa, "The 3d linear inverted pendulum mode: A simple modeling for a biped walking pattern generation," in *Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems*, vol. 1, pp. 239–246, IEEE, 2001.

[7] S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Harada, K. Yokoi, and H. Hirukawa, "Biped walking pattern generation by using preview control of zero-moment point," in *Proceedings of the 2003 IEEE International Conference on Robotics and Automation*, pp. 1620–1626, IEEE, 2003.

[8] T. Koolen, M. Posa, and R. Tedrake, "Balance control using center of mass height variation: Limitations imposed by unilateral contact," in *2016 IEEE-RAS International Conference on Humanoid Robots*, pp. 8–15, 2016.

[9] N. Scianca, D. De Simone, L. Lanari, and G. Oriolo, "Intrinsically stable mpc for humanoid gait generation," in *2016 IEEE-RAS 16th International Conference on Humanoid Robots*, pp. 601–606, IEEE, 2016.

[10] N. Scianca, D. De Simone, L. Lanari, and G. Oriolo, "Mpc for humanoid gait generation: Stability and feasibility," *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1171–1188, 2020.

[11] M. Bjelonic, C. D. Bellicoso, Y. de Viragh, D. Sako, F. D. Tresoldi, F. Jenelten, and M. Hutter, "Keep rollin': Whole-body motion control and planning for wheeled quadrupedal robots," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 2116–2123, 2019.

[12] H. Zhao, L. Yu, S. Qin, G. Jin, and Y. Chen, "Design and control of a bio-inspired wheeled bipedal robot," *IEEE/ASME Transactions on Mechatronics*, vol. 30, no. 4, pp. 2461–2472, 2025.

[13] D. Liu, F. Yang, X. Liao, and X. Lyu, "Diablo: A 6-dof wheeled bipedal robot composed entirely of direct-drive joints," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3605–3612, 2024.

[14] S. Xin and S. Vijayakumar, "Online dynamic motion planning and control for wheeled biped robots," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3892–3899, IEEE, 2020.

[15] J. An, J. Y. Kim, M.-T. Lim, and Y. Oh, "High-speed and enhanced motion control for a wheeled-legged humanoid robot using a two-wheeled inverted pendulum with roll joint," *IEEE Access*, vol. 13, pp. 33330–33340, 2025.

[16] V. Klemm, A. Morra, L. Gulich, D. Mannhart, D. Rohr, M. Kamel, Y. de Viragh, and R. Siegwart, "Lqr-assisted whole-body control of a wheeled bipedal robot with kinematic loops," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3745–3752, 2020.

[17] M. K. Kameduła, *Support Polygon in the Hybrid Legged-Wheeled CENTAURO Robot: Modelling and Control*. Ph.d. thesis, Università degli Studi di Genova, Genoa, Italy, Dec. 2019. Doctor of Philosophy, 32nd cycle.

[18] H. Yu, X. Li, Z. Tao, H. Feng, S. Zhang, and Y. Fu, "Hierarchical jumping optimization for hydraulic biped wheel-legged robots," *Control Engineering Practice*, vol. 141, p. 105721, 2023.

[19] M. Hosseini, D. Rodriguez, and S. Behnke, "Dynamic hybrid locomotion and jumping for wheeled-legged quadrupeds," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 793–799, 2023.

[20] N. Rotella, M. Blösch, L. Righetti, and S. Schaal, "State estimation for a humanoid robot," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, (Chicago, IL, USA), pp. 952–958, IEEE, Sept. 2014.

[21] N. Scianca, "Course slides: Trajectory optimization, dynamic programming, legged robots models and model predictive control." Lecture slides, Sapienza University of Rome, 2024. Course: Underactuated Robots, Department of Computer, Control and Management Engineering.

[22] R. Bellman, "The theory of dynamic programming," *Bulletin of the American Mathematical Society*, vol. 60, pp. 503–515, Nov. 1954.

[23] D. Q. Mayne, "A second-order gradient method of optimizing non-linear discrete time systems," *International Journal of Control*, vol. 3, no. 1, pp. 85–95, 1966.

[24] H. Li, W. Yu, T. Zhang, and P. M. Wensing, "A unified perspective on multiple shooting in differential dynamic programming," Sept. 2023.

[25] S. Caron, Q.-C. Pham, and Y. Nakamura, "Stability of surface contacts for humanoid robots: Closed-form formulae of the contact wrench cone for rectangular support areas," in *IEEE International Conference on Robotics and Automation*, 2015.

[26] C. Mastalli, R. Budhiraja, W. Merkt, G. Saurel, B. Hammoud, M. Naveau, J. Carpentier, L. Righetti, S. Vijayakumar, and N. Mansard, "Crocoddyl: An efficient and versatile framework for multi-contact optimal control," in *IEEE International Conference on Robotics and Automation*, 2020.

[27] Y. Aoyama, G. Boutselis, A. Patel, and E. A. Theodorou, "Constrained differential dynamic programming revisited," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.